

UNIVERSITÀ
degli STUDI
di CATANIA

DIPARTIMENTO DI INGEGNERIA
ELETTRICA ELETTRONICA E INFORMATICA

CORSO DI LAUREA IN INGEGNERIA INFORMATICA

Stefano Caramagno

L'INTELLIGENZA ARTIFICIALE
COME MOTORE
PER L'EVOLUZIONE DEI ROBOT ANTROPOMORFI

Relatore:

Chiar.ma Prof.ssa Daniela Giordano

Correlatori:

Dott. Salvatore Calcagno

Dott. Raffaele Mineo

Anno Accademico 2023-2024

INDICE

INTRODUZIONE	1
1. FONDAMENTI STORICI DELLA ROBOTICA E DELL'INTELLIGENZA ARTIFICIALE.....	5
1.1. CENNI DI STORIA E SVILUPPO DELLA ROBOTICA	6
1.1.1. Origini della robotica	6
1.1.2. Evoluzione dei robot antropomorfi.....	8
1.1.3. Applicazioni moderne della robotica.....	9
1.2. CENNI DI STORIA E SVILUPPO DELL'INTELLIGENZA ARTIFICIALE.....	11
1.2.1. Prime ricerche e scoperte	11
1.2.2. Evoluzione degli algoritmi di apprendimento	13
1.2.3. Large language models (LLMs)	16
1.3. LA SINERGIA TRA ROBOTICA E INTELLIGENZA ARTIFICIALE NELL'EVOLUZIONE STORICA E NELLE APPLICAZIONI FUTURE	18
1.3.1. L'integrazione della robotica con l'intelligenza artificiale.....	18
1.3.2. Impatti e potenziale applicazione	20
2. POLLEN ROBOTICS E L'INNOVATIVO ROBOT ANTROPOMORFO REACHY	22
2.1. POLLEN ROBOTICS	22
2.1.1. Storia e fondazione	23
2.1.2. Missione e visione	24
2.2. IL ROBOT ANTROPOMORFO REACHY.....	26
2.2.1. Descrizione generale di Reachy.....	26
2.2.2. Struttura e componenti tecnici	27
2.2.3. Funzionalità e applicazioni	41

3. SETUP DELL'AMBIENTE DI SVILUPPO E TEST PRELIMINARI DEL SIMULATORE DI REACHY	43
3.1. COMPONENTI E RISORSE NECESSARIE	44
3.1.1. Installazione dei componenti	44
3.1.2. Downloading delle risorse	45
3.2. CONFIGURAZIONE E AVVIAMENTO DEL SIMULATORE DI REACHY CON UNITY 3D	46
3.3.1. Configurazione di Unity per Reachy	47
3.3.2. Importazione del progetto	48
3.3. VALUTAZIONE DELL'EFFETTIVO FUNZIONAMENTO DEL SIMULATORE.....	50
3.4.1. Esecuzione di script di test forniti da Pollen Robotics	50
3.4.2. Analisi dei risultati dei test	63
4. SVILUPPO DI SCRIPT PYTHON PER IL ROBOT ANTROPOMORFO REACHY	67
4.1. SVILUPPO DI SCRIPT PYTHON PER L'ESECUZIONE DEI MOVIMENTI.....	68
4.1.1. Pianificazione dei movimenti	68
4.1.2. Implementazione degli script di movimento	70
4.1.3. Test dei movimenti	75
4.2. SVILUPPO DI SCRIPT PYTHON PER IL RICONOSCIMENTO DI OGGETTI.....	78
4.2.1. Introduzione al riconoscimento di oggetti	79
4.2.2. Implementazione degli script di riconoscimento	80
4.2.3. Test del riconoscimento di oggetti.....	85
5. ADDESTRAMENTO DI ChatGPT-4o E VALUTAZIONE DEGLI SCRIPT GENERATI PER REACHY	89

5.1.	ADDESTRAMENTO DI ChatGPT-4o CON SCRIPT PYTHON	90
5.1.1.	Processo di addestramento tramite prompt e dati in input.....	90
5.1.2.	Configurazione dell'ambiente di addestramento	92
5.2.	TEST PILOTA E VALUTAZIONE INZIALE	96
5.2.1.	Formulazione delle richiesta del task specifico	96
5.2.2.	Test degli script generati.....	98
5.3.	TEST SISTEMATICI CON INCREMENTO DI COMPLESSITA' E DEI DATI DI INPUT	111
5.3.1.	Movimenti semplici	111
5.3.2.	Movimenti complessi.....	114
5.3.3.	Movimenti complessi con input visivo	116
5.4.	ANALISI DELLE DIFFERENZE E DELLE LIMITAZIONI DEI RISULTATI.....	120
5.4.1.	Differenze osservate nei risultati dei test.....	120
5.4.2.	Analisi delle limitazioni e dei vincoli riscontrati.....	121
	CONCLUSIONI	124
	RINGRAZIAMENTI.....	125
	INDICE DELLE FIGURE.....	126
	INDICE DELLE TABELLE	127
	BIBLIOGRAFIA	128

INTRODUZIONE

L'Intelligenza Artificiale (IA) e la robotica rappresentano due dei settori più dinamici e innovativi della tecnologia contemporanea.

L'integrazione tra queste due discipline sta trasformando radicalmente il modo in cui i robot interagiscono con il mondo e con gli esseri umani.

Questa tesi sperimentale, intitolata "L'Intelligenza Artificiale come Motore per l'Evoluzione dei Robot Antropomorfi," esplora questa sinergia, focalizzandosi sul robot antropomorfo Reachy sviluppato da Pollen Robotics e sull'impiego di modelli avanzati di IA per migliorare le sue capacità operative.

Motivazioni dello studio

La scelta di studiare l'evoluzione dei robot antropomorfi guidata dall'intelligenza artificiale nasce dalla consapevolezza dell'enorme potenziale che questa integrazione può offrire.

I robot antropomorfi, progettati per imitare l'aspetto e i movimenti umani, trovano applicazione in numerosi settori, dalla medicina all'educazione, dalla manifattura all'assistenza personale.

Tuttavia, per raggiungere un livello di interazione e autonomia paragonabile a quello umano, è essenziale dotare questi robot di sistemi di IA avanzati.

Un robot antropomorfo come Reachy può beneficiare enormemente dall'integrazione di modelli di IA avanzati, come GPT-4o, che permettono di migliorare la capacità di comprensione e generazione di linguaggio naturale.

Questo non solo potenzia l'interazione con gli utenti, ma rende anche il robot più versatile e capace di adattarsi a una vasta gamma di situazioni operative.

L'obiettivo principale della tesi è esplorare come l'IA, attraverso l'addestramento di large language models (LLMs) come ChatGPT-4o, possa essere utilizzata per generare script Python capaci di eseguire task complessi con il robot Reachy.

Questo approccio sperimentale mira a valutare l'efficacia degli script generati dall'IA rispetto a quelli creati manualmente, analizzando miglioramenti in termini di efficienza, precisione e versatilità.

Obiettivi della tesi

Gli obiettivi specifici della tesi sono:

1. **Sviluppare un ambiente di simulazione:** Configurare un simulatore basato su Unity 3D per testare gli script Python su Reachy.
2. **Addestrare ChatGPT-4o per la generazione di script Python:** Utilizzare script esistenti per addestrare il modello linguistico a generare nuovi script per task specifici.
3. **Valutare e confrontare gli script generati:** Testare gli script generati dall'IA nel simulatore di Reachy, confrontandoli con quelli manuali per identificare miglioramenti e inefficienze.

Struttura della tesi

La tesi è strutturata in cinque capitoli principali:

1. **Introduzione e fondamenti Storici della robotica e dell'intelligenza artificiale:** Questo capitolo fornisce una panoramica della storia e dell'evoluzione della robotica e dell'IA, descrivendo le principali innovazioni e i progressi tecnologici.

Saranno analizzate le origini della robotica, l'evoluzione dei robot antropomorfi e le applicazioni moderne della robotica in vari settori.

Inoltre, si esploreranno le tappe fondamentali della ricerca sull'intelligenza artificiale, dall'inizio alle innovazioni contemporanee come i large language models (LLMs).

2. **Pollen Robotics e l'innovativo robot antropomorfo Reachy:** Viene presentata Pollen Robotics, la storia dell'azienda, la sua missione e visione, e una descrizione dettagliata di Reachy, il robot antropomorfo oggetto dello studio.

Questo capitolo descrive i progetti principali dell'azienda, fornendo un quadro completo del contesto in cui è nato Reachy.

3. **Setup dell'ambiente di sviluppo e test preliminari del simulatore di Reachy:** Questo capitolo descrive l'installazione e la configurazione degli strumenti necessari per sviluppare e testare gli script Python nel simulatore di Reachy.

Include istruzioni dettagliate su come installare Python, Unity 3D, Visual Studio e altre librerie e strumenti essenziali. Inoltre, vengono trattati i passaggi per configurare il simulatore e verificarne il corretto funzionamento.

4. **Sviluppo di script Python per il robot antropomorfo Reachy:** Dettaglia il processo di sviluppo degli script per i movimenti e il riconoscimento di oggetti, inclusi i test e la validazione.

Questo capitolo fornisce una guida pratica su come pianificare e implementare movimenti complessi e sviluppare script per il riconoscimento di oggetti, utilizzando librerie Python specifiche.

5. **Addestramento di ChatGPT-4o e valutazione degli script generati per Reachy:** Descrive il processo di addestramento del modello IA, la generazione di nuovi script, e l'analisi comparativa tra gli script manuali e quelli generati dall'IA.

Viene esaminata l'efficacia del processo di addestramento, le prestazioni degli script generati e le aree di miglioramento identificabili per ottimizzare ulteriormente il sistema.

Impatto e conclusioni

L'integrazione tra intelligenza artificiale e robotica antropomorfa rappresenta una frontiera affascinante e promettente.

La capacità di generare script Python attraverso modelli linguistici avanzati non solo facilita lo sviluppo di nuove funzionalità per i robot, ma apre anche nuove possibilità per l'interazione uomo-macchina.

I risultati di questo studio sperimentale forniranno un contributo al campo della robotica, evidenziando il potenziale delle tecnologie AI nell'evoluzione dei robot antropomorfi e offrendo spunti per ulteriori ricerche e applicazioni future.

1. FONDAMENTI STORICI DELLA ROBOTICA E DELL'INTELLIGENZA ARTIFICIALE

In questo capitolo, viene fornita una panoramica delle basi storiche e dello sviluppo di robotica e intelligenza artificiale, due discipline che hanno rivoluzionato il panorama tecnologico moderno e rappresentano i pilastri della ricerca scientifica contemporanea. Verranno analizzati i principali eventi storici, le innovazioni tecnologiche e le applicazioni che hanno caratterizzato l'evoluzione di queste due aree, con particolare attenzione alla loro convergenza e alle implicazioni future.

La robotica, fin dai primi automi meccanici delle antiche civiltà, ha catturato l'immaginazione dell'uomo con la possibilità di creare macchine capaci di replicare movimenti e compiti umani. Da semplici dispositivi meccanici, come quelli progettati durante il Rinascimento, ai complessi robot antropomorfi contemporanei, la disciplina ha registrato un'evoluzione straordinaria, sostenuta dai progressi in meccanica, elettronica e informatica. Saranno esaminati i momenti chiave di questa evoluzione, dalle origini alla moderna applicazione della robotica in settori avanzati quali la manifattura, la medicina e l'assistenza personale.

Parallelamente, l'intelligenza artificiale ha percorso un cammino altrettanto affascinante, partendo dai primi tentativi di riprodurre l'intelligenza umana attraverso modelli computazionali, fino a giungere ai sofisticati algoritmi di apprendimento automatico di oggi. Verranno illustrate le principali tappe dello sviluppo dell'IA, inclusi i contributi teorici iniziali, le innovazioni negli algoritmi di machine learning e deep learning, e l'emergere dei large language models (LLMs) come ChatGPT-4, che hanno rivoluzionato il campo del trattamento del linguaggio naturale.

L'analisi proseguirà esplorando le sinergie tra robotica e intelligenza artificiale, con un focus su come queste discipline si integrano per creare sistemi robotici intelligenti, in grado di interagire in modo naturale con l'ambiente e con gli esseri umani. Verranno infine presentate le attuali applicazioni e le prospettive future di

questa interazione, evidenziando il ruolo cruciale che essa riveste nel panorama tecnologico contemporaneo.

1.1. CENNI DI STORIA E SVILUPPO DELLA ROBOTICA

La robotica è una disciplina che ha affascinato l'umanità per secoli, evolvendosi da semplici automi meccanici a sofisticati sistemi robotici in grado di svolgere compiti complessi. Comprendere le origini e lo sviluppo storico della robotica è fondamentale per apprezzare le tecnologie avanzate che abbiamo oggi e per capire le potenzialità future di questo campo. La storia della robotica è caratterizzata da continue innovazioni e scoperte che hanno trasformato la nostra capacità di progettare macchine intelligenti.

1.1.1. Origini della robotica

Le origini della robotica affondano le radici nell'antichità, con i primi tentativi umani di creare dispositivi meccanici in grado di svolgere compiti specifici. Le civiltà antiche, come quella egizia e greca, sono note per aver sviluppato alcune delle prime macchine automatiche.

Il termine "robotica" deriva dal termine "robot," che a sua volta proviene dalla parola ceca "robota," che significa lavoro forzato o servitù. Il termine fu introdotto per la prima volta dallo scrittore ceco Karel Čapek nel suo dramma del 1920 "R.U.R." (Rossum's Universal Robots), in cui immaginava una fabbrica di esseri artificiali creati per svolgere lavori pesanti e ripetitivi. Questo concetto letterario pose le basi per lo sviluppo della moderna robotica, suscitando interesse e ispirazione tra scienziati e ingegneri.

Durante il Rinascimento, inventori come Leonardo da Vinci contribuirono ulteriormente allo sviluppo di idee robotiche. Da Vinci progettò diversi automi, tra cui un cavaliere meccanico capace di muoversi autonomamente, un precursore dei moderni robot antropomorfi. Questi progetti, sebbene mai realizzati

completamente, dimostrarono la fattibilità e il potenziale delle macchine automatiche.

Il vero progresso nella robotica iniziò con la Rivoluzione Industriale nel XVIII secolo, quando l'automazione divenne una componente chiave nei processi di produzione. I telai meccanici, come il telaio Jacquard, introdussero il concetto di controllo programmato, utilizzando schede perforate per gestire le operazioni delle macchine tessili. Questa innovazione rappresentò un passo cruciale verso lo sviluppo di sistemi automatizzati complessi.

Nel XX secolo, la robotica cominciò a prendere forma come disciplina scientifica e tecnologica. Uno degli sviluppi più significativi fu la creazione del primo robot programmabile da parte di George Devol e Joseph Engelberger negli anni '50. Il loro robot, chiamato Unimate, fu utilizzato per la prima volta in una linea di assemblaggio della General Motors nel 1961, rivoluzionando il settore manifatturiero. Unimate rappresentò il primo esempio di un robot industriale, capace di svolgere compiti ripetitivi con precisione e affidabilità.

Parallelamente, la scienza dell'automazione e del controllo iniziò a svilupparsi rapidamente. I progressi nella teoria dei controlli, nell'elettronica e nell'informatica permisero la creazione di robot sempre più sofisticati. La nascita dei primi computer digitali giocò un ruolo fondamentale, consentendo lo sviluppo di algoritmi complessi per il controllo e la programmazione dei robot.

Un altro passo importante fu l'introduzione della cibernetica, un campo interdisciplinare che studia i sistemi di controllo e comunicazione negli esseri viventi e nelle macchine. Fondato da Norbert Wiener negli anni '40, la cibernetica gettò le basi teoriche per comprendere e progettare sistemi automatici complessi, influenzando profondamente la robotica moderna.

Negli anni '70 e '80, la robotica fece ulteriori progressi con lo sviluppo dei robot mobili e dei robot di servizio. Progetti come il robot mobile Shakey, sviluppato dallo Stanford Research Institute (SRI), dimostrarono la capacità dei robot di navigare e interagire con ambienti non strutturati, aprendo nuove possibilità per applicazioni al di fuori del contesto industriale.

Oggi, la robotica è un campo vasto e diversificato, che abbraccia numerosi settori, tra cui la manifattura, la medicina, l'assistenza domestica, l'esplorazione spaziale e la difesa. La ricerca continua a spingere i limiti della tecnologia, esplorando nuove frontiere come la robotica autonoma, l'intelligenza artificiale e la bio-robotica.

1.1.2. Evoluzione dei robot antropomorfi

L'evoluzione dei robot antropomorfi è una testimonianza del progresso tecnologico e scientifico, che ha portato allo sviluppo di macchine sempre più sofisticate e capaci di imitare l'aspetto e i movimenti umani. La storia dei robot antropomorfi è ricca di innovazioni e scoperte, che hanno contribuito a trasformare idee fantascientifiche in realtà pratiche.

Le prime idee di robot antropomorfi risalgono all'antichità, con miti e leggende che descrivevano figure meccaniche dotate di sembianze umane. Tuttavia, le prime realizzazioni pratiche di automi umanoidi si ebbero durante il Rinascimento. Leonardo da Vinci progettò un cavaliere meccanico, capace di muovere braccia e gambe, rappresentando uno dei primi tentativi di creare una macchina con movimenti simili a quelli umani.

Nel XVIII secolo, l'inventore svizzero Pierre Jaquet-Droz costruì diversi automi, tra cui "The Writer", un automa capace di scrivere su carta. Questi primi automi erano azionati da complessi meccanismi di ingranaggi e molle, mostrando un'abilità ingegneristica notevole per l'epoca, ma erano limitati nelle loro capacità di interazione e autonomia.

Il XX secolo ha visto un'accelerazione significativa nello sviluppo dei robot antropomorfi, grazie ai progressi nell'elettronica, nella meccanica e nell'informatica. Nel 1927, il film "Metropolis" di Fritz Lang presentava uno dei primi esempi di robot antropomorfo nella cultura popolare, influenzando l'immaginario collettivo riguardo ai robot umanoidi.

Negli anni '60, WABOT-1, sviluppato dall'Università di Waseda in Giappone, è stato uno dei primi robot antropomorfi funzionanti. WABOT-1 era in grado di camminare, afferrare oggetti e persino dialogare in modo rudimentale. Questo

progetto segnò un passo importante nella robotica, dimostrando la fattibilità di creare macchine che potevano imitare alcune delle funzioni umane di base.

Negli anni '70 e '80, la robotica antropomorfa continuò a evolversi con lo sviluppo di robot più avanzati. ASIMO, sviluppato da Honda, è uno dei robot antropomorfi più noti e rappresenta un significativo avanzamento tecnologico. Introdotto per la prima volta nel 2000, ASIMO è in grado di camminare su due gambe, correre, salire scale e interagire con gli esseri umani in modo complesso. La sua progettazione ha richiesto decenni di ricerca e sviluppo, incorporando tecnologie avanzate di sensori, attuatori e controllo dell'equilibrio.

Un altro esempio rilevante è il robot umanoide iCub, sviluppato dall'Istituto Italiano di Tecnologia (IIT). iCub è progettato per studiare la cognizione e l'apprendimento nei robot, simulando il comportamento di un bambino. Dotato di un alto grado di articolazione e di capacità di percezione avanzate, iCub è utilizzato in numerosi progetti di ricerca per esplorare l'interazione uomo-robot e l'apprendimento automatico.

La combinazione di robotica e intelligenza artificiale ha aperto nuove frontiere per i robot antropomorfi. Oggi, i robot umanoidi non solo imitano l'aspetto umano, ma sono anche dotati di capacità cognitive che permettono loro di apprendere e adattarsi a nuovi compiti. Algoritmi di machine learning e reti neurali artificiali sono utilizzati per migliorare la percezione, la pianificazione e il controllo dei robot.

Sophia, sviluppata da Hanson Robotics, è un esempio di robot antropomorfo avanzato dotato di intelligenza artificiale. Sophia è in grado di riconoscere volti, comprendere e rispondere a domande, e imitare espressioni facciali umane. La sua capacità di interazione sociale rappresenta un importante passo avanti nella robotica antropomorfa, dimostrando il potenziale per l'integrazione di robot umanoidi in contesti quotidiani.

1.1.3. Applicazioni moderne della robotica

Le applicazioni moderne della robotica spaziano in diversi settori, evidenziando l'impatto significativo che questa tecnologia ha nella società contemporanea. Originariamente confinata all'industria manifatturiera, la robotica è oggi integrata

in molteplici contesti, contribuendo all'aumento dell'efficienza e della sicurezza, oltre che al miglioramento della qualità della vita umana.

Nell'industria manifatturiera, i robot automatizzano operazioni complesse e ripetitive, come saldatura e assemblaggio, migliorando la produttività e la precisione. I cobot lavorano a fianco degli operatori umani per compiti che richiedono destrezza, riducendo i costi e aumentando la sicurezza.

In ambito sanitario, i robot chirurgici, come il sistema da Vinci, consentono interventi con maggiore precisione e tempi di recupero più rapidi per i pazienti. I robot assistivi supportano la riabilitazione e svolgono compiti di assistenza in ospedali e case di cura, migliorando la gestione dei pazienti e l'efficienza operativa.

L'esplorazione spaziale vede l'impiego di rover robotici, come quelli della NASA, che esplorano altri pianeti in ambienti estremi, raccogliendo dati cruciali per l'avanzamento scientifico e preparando il terreno per future missioni umane.

In agricoltura, i robot ottimizzano la gestione delle colture attraverso compiti come semina e raccolta, utilizzando sensori e algoritmi di intelligenza artificiale per migliorare la resa e ridurre l'impatto ambientale, rendendo l'agricoltura più sostenibile.

Nel settore della logistica, i robot automatizzano processi di stoccaggio e recupero delle merci nei magazzini, mentre droni e veicoli autonomi migliorano l'efficienza nelle consegne e nel trasporto di merci, con potenziali benefici per la mobilità urbana e la sicurezza stradale.

Nel campo dei servizi e dell'intrattenimento, i robot offrono nuove esperienze interattive e migliorano l'interazione con i clienti in contesti come hotel e ristoranti. Nell'educazione, i robot facilitano l'insegnamento della programmazione e della robotica, rendendo l'apprendimento pratico e interattivo.

Infine, nella sfera dell'assistenza domiciliare, i robot assistivi supportano persone anziane o con disabilità, svolgendo compiti quotidiani e promuovendo l'indipendenza, rispondendo alle esigenze di una popolazione in continua crescita.

1.2. CENNI DI STORIA E SVILUPPO DELL'INTELLIGENZA ARTIFICIALE

L'intelligenza artificiale (IA) ha rivoluzionato il modo in cui affrontiamo problemi complessi e interagiamo con le tecnologie moderne. Le radici dell'IA risalgono a teorie pionieristiche e scoperte che hanno gettato le basi per lo sviluppo di algoritmi sofisticati e sistemi intelligenti. Esaminare la storia dell'IA ci consente di comprendere le sue evoluzioni, dai primi concetti teorici alle applicazioni pratiche che oggi influenzano numerosi aspetti della nostra vita quotidiana.

1.2.1. Prime ricerche e scoperte

Le prime ricerche e scoperte nel campo dell'intelligenza artificiale (IA) hanno gettato le basi per una disciplina che oggi è al centro dell'innovazione tecnologica. L'IA ha avuto origine come campo di studio formale nella metà del XX secolo, ma le idee fondamentali che ne hanno guidato lo sviluppo risalgono a molto tempo prima.

Le radici concettuali dell'intelligenza artificiale possono essere fatte risalire ai filosofi antichi e medievali che esploravano la natura dell'intelligenza, della ragione e della mente. Ad esempio, Aristotele sviluppò il sillogismo, una forma di ragionamento logico che ha influenzato il pensiero successivo sulla razionalità. Nel XVII secolo, il filosofo e matematico René Descartes propose che gli animali (inclusi gli esseri umani) potrebbero essere visti come macchine complesse, prefigurando in qualche modo l'idea di una mente artificiale.

Una figura centrale nelle prime ricerche sull'intelligenza artificiale è Alan Turing, un matematico britannico che durante la Seconda Guerra Mondiale contribuì significativamente alla crittografia e alla teoria computazionale. Nel 1950, Turing pubblicò il celebre articolo "Computing Machinery and Intelligence," in cui propose il famoso "Test di Turing" come criterio per determinare se una macchina possa essere considerata intelligente. Turing suggerì che, se una macchina potesse sostenere una conversazione indistinguibile da quella di un essere umano, allora

potrebbe essere considerata intelligente. Questo articolo è considerato uno dei fondamenti teorici dell'IA.

Il vero inizio dell'IA come campo di studio formale è spesso attribuito alla Dartmouth Conference del 1956, organizzata da John McCarthy, Marvin Minsky, Nathaniel Rochester e Claude Shannon. Durante questa conferenza, il termine "intelligenza artificiale" fu coniato e i partecipanti delinearono un programma di ricerca per creare macchine che potessero "pensare" come esseri umani. Questo evento segnò l'inizio di un periodo di entusiasmo e finanziamenti significativi per la ricerca sull'IA.

Negli anni '50 e '60, i ricercatori svilupparono i primi sistemi di intelligenza artificiale, molti dei quali erano basati su algoritmi di ricerca e logica. Ad esempio, Allen Newell e Herbert A. Simon crearono il programma Logic Theorist, che fu in grado di dimostrare teoremi matematici. Successivamente, svilupparono il General Problem Solver (GPS), progettato per risolvere una vasta gamma di problemi attraverso la ricerca euristica.

Un altro sistema influente fu ELIZA, sviluppato da Joseph Weizenbaum negli anni '60. ELIZA era un programma di elaborazione del linguaggio naturale che simulava una conversazione con uno psicoterapeuta. Sebbene molto semplice rispetto agli standard odierni, ELIZA dimostrò il potenziale dell'interazione uomo-macchina e gettò le basi per futuri sviluppi nel campo del linguaggio naturale.

Durante le prime ricerche sull'IA, emersero due approcci principali: l'IA simbolica e l'IA connessionista. L'IA simbolica, o GOFAI (Good Old-Fashioned Artificial Intelligence), si basava sulla manipolazione di simboli e regole logiche per rappresentare la conoscenza e risolvere problemi. Questo approccio dominò la ricerca sull'IA per diversi decenni e portò allo sviluppo di sistemi esperti, che utilizzavano regole di produzione per emulare il processo decisionale umano in domini specifici.

L'IA connessionista, d'altra parte, si ispirava al funzionamento del cervello umano e utilizzava reti neurali artificiali per apprendere dai dati. Sebbene inizialmente meno popolare a causa delle limitazioni computazionali dell'epoca, questo

approccio guadagnò terreno negli anni '80 e '90 grazie ai progressi nell'hardware e agli algoritmi di apprendimento.

Nonostante l'entusiasmo iniziale, la ricerca sull'IA incontrò numerose sfide, tra cui la difficoltà di creare sistemi veramente intelligenti e la mancanza di potenza computazionale sufficiente. Negli anni '70 e '80, ci fu un periodo noto come "inverno dell'IA," caratterizzato da un calo dei finanziamenti e dell'interesse per la disciplina. Le promesse non mantenute e le aspettative disattese portarono a una fase di riflessione e ripensamento degli approcci utilizzati.

A partire dagli anni '90, l'IA ha conosciuto una rinascita grazie ai progressi nell'hardware, alla disponibilità di grandi quantità di dati e allo sviluppo di nuovi algoritmi. Le deep neural networks hanno rivoluzionato il campo, permettendo la creazione di sistemi altamente performanti in compiti come il riconoscimento delle immagini, il linguaggio naturale e i giochi strategici.

Oggi, l'IA è una disciplina dinamica e in continua evoluzione, con applicazioni che spaziano dalla medicina alla finanza, dalla robotica all'intrattenimento. Le prime ricerche e scoperte hanno fornito le fondamenta teoriche e pratiche su cui si basa l'IA moderna, e il campo continua a spingere i limiti della tecnologia e della conoscenza.

1.2.2. Evoluzione degli algoritmi di apprendimento

L'evoluzione degli algoritmi di apprendimento automatico rappresenta uno dei progressi più significativi nel campo dell'intelligenza artificiale. Dalla nascita degli algoritmi simbolici ai moderni approcci di deep learning, lo sviluppo degli algoritmi di apprendimento ha trasformato il modo in cui i sistemi intelligenti elaborano e comprendono i dati, portando a innovazioni in numerosi settori.

Negli anni '50 e '60, l'apprendimento automatico era dominato dagli approcci simbolici, basati sulla manipolazione di simboli e regole logiche. Gli algoritmi simbolici, come quelli utilizzati nel programma Logic Theorist di Newell e Simon, erano progettati per risolvere problemi attraverso la ricerca euristica e la logica formale. Questo periodo vide anche lo sviluppo dei primi sistemi esperti, che

utilizzavano regole di produzione per emulare il processo decisionale umano in domini specifici.

Alla fine degli anni '50, Frank Rosenblatt introdusse il perceptron, una delle prime reti neurali artificiali. Il perceptron era un modello semplice che poteva apprendere a classificare input lineari attraverso un processo di addestramento supervisionato. Tuttavia, le limitazioni matematiche del perceptron, evidenziate dal libro "Perceptrons" di Minsky e Papert nel 1969, rallentarono lo sviluppo delle reti neurali per diversi anni.

Negli anni '70 e '80, l'apprendimento induttivo divenne una delle principali aree di ricerca. Gli alberi di decisione, come l'algoritmo ID3 sviluppato da Ross Quinlan, emersero come potenti strumenti per la classificazione e la regressione. Gli alberi di decisione costruiscono modelli gerarchici che rappresentano le decisioni in un formato comprensibile, facilitando l'interpretazione e l'analisi dei dati.

Il rilancio delle reti neurali avvenne negli anni '80 con lo sviluppo delle reti neurali multi-strato e l'introduzione dell'algoritmo di backpropagation. Questo algoritmo, popolarizzato da Rumelhart, Hinton e Williams, permise l'addestramento efficace di reti neurali con più strati nascosti, superando le limitazioni dei perceptron. Le reti neurali multi-strato aprirono nuove possibilità per l'apprendimento automatico, consentendo la modellazione di funzioni non lineari complesse.

Negli anni '90, le Support Vector Machines (SVM) sviluppate da Vladimir Vapnik e i suoi colleghi divennero uno degli strumenti più efficaci per la classificazione e la regressione. Le SVM si basano sulla teoria delle funzioni di margine massimo, che consente di trovare un iperpiano ottimale che separa le classi di dati. La loro capacità di gestire dati ad alta dimensionalità e di trovare soluzioni robuste le ha rese estremamente popolari.

Parallelamente, l'apprendimento bayesiano guadagnò importanza come metodo probabilistico per l'inferenza e la previsione. Gli approcci bayesiani, come i modelli di Markov nascosti (HMM) e le reti bayesiane, offrono una struttura solida per incorporare incertezze e conoscenze a priori nei modelli di apprendimento. Questi

metodi sono particolarmente utili in applicazioni come il riconoscimento vocale e la modellazione temporale.

Il XXI secolo ha visto l'ascesa del deep learning, una sottocategoria dell'apprendimento automatico basata su deep neural networks. Le deep neural networks, composte da molteplici strati di nodi (neuroni), sono in grado di apprendere rappresentazioni gerarchiche complesse dai dati. Il deep learning ha rivoluzionato vari campi, tra cui il riconoscimento delle immagini, il riconoscimento vocale e il trattamento del linguaggio naturale.

L'avanzamento del deep learning è stato facilitato da tre fattori principali: l'aumento della potenza computazionale (grazie alle GPU), la disponibilità di grandi quantità di dati (big data) e lo sviluppo di nuovi algoritmi di ottimizzazione e architetture di reti neurali. Le architetture di maggior successo sono le reti convoluzionali (CNN) e i Transformers per il linguaggio naturale e il modellamento delle sequenze temporali.

L'apprendimento per rinforzo è un altro approccio significativo nell'evoluzione degli algoritmi di apprendimento. In questo paradigma, un agente apprende a compiere azioni in un ambiente al fine di massimizzare una ricompensa cumulativa. Algoritmi come Q-learning e metodi basati su policy gradient hanno dimostrato di essere efficaci in vari contesti, dai giochi strategici alla robotica. Il successo dell'algoritmo AlphaGo di DeepMind, che ha sconfitto campioni umani nel gioco del Go, è un esempio emblematico della potenza dell'apprendimento per rinforzo.

Negli ultimi anni, gli algoritmi di apprendimento automatico continuano a evolversi con l'introduzione di tecniche innovative come l'apprendimento federato, che consente di addestrare modelli su dati distribuiti senza centralizzare le informazioni, e il meta-apprendimento, che mira a creare modelli che possano apprendere nuove attività con pochi esempi.

Inoltre, l'attenzione si è concentrata su aspetti come l'interpretabilità e la trasparenza degli algoritmi di apprendimento automatico, con lo sviluppo di tecniche per spiegare e visualizzare le decisioni dei modelli complessi. Questo è particolarmente

importante in settori sensibili come la medicina e la finanza, dove la comprensione e la fiducia nelle decisioni del modello sono cruciali.

1.2.3. Large language models (LLMs)

I Large Language Models rappresentano uno dei progressi più rivoluzionari nel campo dell'intelligenza artificiale e del trattamento del linguaggio naturale (NLP). Questi modelli, addestrati su enormi quantità di dati testuali, hanno dimostrato capacità sorprendenti nella generazione, comprensione e traduzione del linguaggio umano. La loro evoluzione ha portato a innovazioni significative in vari settori, dalla ricerca scientifica alla produzione di contenuti, fino all'interazione uomo-macchina.

I LLMs si basano su architetture di deep neural networks, in particolare sui Transformers, introdotti da Vaswani et al. nel 2017. I Transformers utilizzano meccanismi di attenzione che consentono al modello di dare peso diverso alle parole nel contesto di una frase, migliorando la capacità di catturare le dipendenze a lungo termine nel testo. Questo ha rappresentato un notevole passo avanti rispetto ai modelli sequenziali tradizionali, come le reti neurali ricorrenti (RNN) e le reti a memoria a lungo termine (LSTM).

Uno dei primi LLMs di grande impatto è stato BERT (Bidirectional Encoder Representations from Transformers), sviluppato da Google nel 2018. BERT utilizza un approccio bidirezionale per comprendere il contesto delle parole in una frase, consentendo al modello di ottenere prestazioni all'avanguardia in molteplici task di NLP, come named entity recognition e sentiment analysis.

Successivamente, OpenAI ha sviluppato la serie di modelli GPT (Generative Pre-trained Transformer), con GPT-3 come modello di punta. GPT-3, rilasciato nel 2020, è stato addestrato su 175 miliardi di parametri, rendendolo uno dei modelli di linguaggio più grandi mai creati. La capacità di GPT-3 di generare testo coerente e contestualmente rilevante ha aperto nuove possibilità per l'automazione della scrittura, la creazione di chatbot avanzati e la produzione di contenuti creativi.

I LLMs come BERT e GPT sono addestrati utilizzando tecniche di pre-training e fine-tuning. Durante il pre-training, il modello viene esposto a una vasta gamma di dati testuali per imparare rappresentazioni generiche del linguaggio.

Successivamente, il modello viene adattato su compiti specifici utilizzando dataset più piccoli e mirati.

L'addestramento dei LLMs richiede risorse computazionali significative, inclusi potenti processori grafici (GPU) e accesso a grandi quantità di dati. Le tecniche di addestramento parallelo e distribuito sono spesso utilizzate per accelerare il processo, permettendo di sfruttare l'infrastruttura cloud e i supercomputer.

I LLMs hanno avuto un impatto significativo su numerosi settori. In ambito aziendale, vengono utilizzati per automatizzare il servizio clienti attraverso chatbot intelligenti, migliorare la ricerca di informazioni nei documenti aziendali e generare rapporti e riassunti automatici. Nella sanità, i LLMs aiutano nella generazione di documentazione clinica, nell'analisi dei dati dei pazienti e nella consulenza diagnostica basata su letteratura medica aggiornata.

Nel settore educativo, i LLMs supportano l'apprendimento personalizzato fornendo spiegazioni dettagliate, rispondendo a domande complesse e generando materiali didattici. Inoltre, i LLMs sono utilizzati per migliorare gli strumenti di traduzione automatica, rendendo la comunicazione multilingue più accessibile e precisa.

Nonostante i notevoli progressi, i LLMs presentano alcune sfide e considerazioni etiche. La generazione di testo può talvolta includere bias presenti nei dati di addestramento, portando a risultati discriminatori o inaccurati. Inoltre, l'uso dei LLMs solleva preoccupazioni riguardo alla privacy dei dati e alla possibilità di generare contenuti malevoli o fuorvianti.

Per affrontare queste sfide, la comunità di ricerca sta sviluppando metodi per rendere i LLMs più trasparenti e interpretabili. Tecniche di debiasing e meccanismi di controllo per la generazione di contenuti sono in fase di esplorazione per mitigare i rischi associati. Inoltre, l'adozione di principi etici nella progettazione e

nell'implementazione dei LLMs è fondamentale per garantire un uso responsabile della tecnologia.

Il futuro dei LLMs è promettente, con continui progressi nelle architetture di rete, negli algoritmi di addestramento e nelle applicazioni pratiche. La ricerca futura si concentrerà probabilmente sulla creazione di modelli più efficienti dal punto di vista computazionale, in grado di operare con meno risorse pur mantenendo alte prestazioni.

Inoltre, si prevede che i LLMs diventeranno sempre più integrati nei sistemi di intelligenza artificiale multimodale, combinando testo, immagini e altri tipi di dati per fornire una comprensione più completa e contestuale. Questa integrazione potrebbe portare a nuovi livelli di automazione e assistenza intelligente in vari ambiti, migliorando ulteriormente l'interazione uomo-macchina.

1.3. LA SINERGIA TRA ROBOTICA E INTELLIGENZA ARTIFICIALE NELL'EVOLUZIONE STORICA E NELLE APPLICAZIONI FUTURE

La fusione tra robotica e intelligenza artificiale rappresenta uno degli sviluppi più promettenti e rivoluzionari della tecnologia moderna. La sinergia tra questi due campi consente di creare robot intelligenti in grado di apprendere, adattarsi e interagire con il mondo in modi sempre più sofisticati. L'integrazione della robotica con l'IA sta trasformando la nostra capacità di automatizzare compiti complessi, migliorare l'efficienza operativa e innovare in settori come la medicina, l'industria e l'assistenza quotidiana.

1.3.1. L'integrazione della robotica con l'intelligenza artificiale

La combinazione tra robotica e IA ha avuto inizio con i primi sviluppi nel campo della cibernetica e dell'informatica negli anni '50 e '60. I pionieri della robotica iniziarono a esplorare l'uso di algoritmi di controllo e logica per migliorare le capacità dei robot. L'avvento dei computer digitali permise l'implementazione di

algoritmi complessi, che migliorarono il controllo dei movimenti e la gestione dei dati sensoriali.

Negli anni '80, con l'introduzione delle reti neurali e dei primi sistemi di apprendimento automatico, la robotica iniziò a incorporare tecniche di IA per migliorare l'autonomia e l'adattabilità dei robot. L'uso di sensori avanzati e tecnologie di visione artificiale permise ai robot di percepire e interpretare il loro ambiente in modo più sofisticato.

L'unione tra robotica e IA è diventata fondamentale per diversi motivi. Prima di tutto, l'IA consente ai robot di apprendere dall'esperienza, migliorando le loro prestazioni nel tempo senza bisogno di essere riprogrammati manualmente per ogni nuovo compito. Questo rende i sistemi robotici più flessibili e adattabili a vari contesti operativi.

Inoltre, l'integrazione con l'IA permette ai robot di prendere decisioni in modo autonomo basandosi sull'analisi dei dati in tempo reale. Ciò è particolarmente utile in ambienti dinamici e complessi, dove è necessario reagire rapidamente ai cambiamenti e agli imprevisti. La capacità di riconoscere oggetti, volti e comportamenti umani migliora significativamente l'interazione uomo-robot, rendendo i robot più utili e intuitivi da utilizzare.

L'applicazione dell'IA nei sistemi robotici ha portato a numerosi benefici. Tra i principali vi sono:

- **Autonomia e adattabilità:** I robot dotati di IA possono operare autonomamente in ambienti non strutturati, adattandosi ai cambiamenti senza interventi umani.
- **Efficienza operativa:** L'IA ottimizza i processi robotici, riducendo i tempi di inattività e migliorando l'efficienza delle operazioni.
- **Interazione naturale:** I robot con IA possono comprendere e rispondere al linguaggio naturale, riconoscere espressioni facciali e gesti, e interagire in modo più umano.

- **Sicurezza:** L'IA aiuta a prevenire incidenti attraverso il monitoraggio continuo e la previsione dei rischi, migliorando la sicurezza nelle operazioni robotiche.

Diversi progressi tecnologici hanno reso possibile la sinergia tra robotica e IA. L'aumento della potenza computazionale e lo sviluppo di algoritmi avanzati di machine learning hanno permesso ai robot di elaborare grandi quantità di dati e apprendere in modo più efficiente. I sensori avanzati, come le telecamere 3D, i lidar e i sensori di forza, forniscono ai robot una percezione dettagliata del loro ambiente, migliorando le capacità di navigazione e manipolazione.

L'introduzione delle deep neural networks ha ulteriormente potenziato le capacità dei robot, consentendo loro di riconoscere modelli complessi nei dati visivi e sonori. Questo ha portato a notevoli miglioramenti in applicazioni come la visione artificiale, il riconoscimento vocale e la robotica cognitiva.

1.3.2. Impatti e potenziale applicazione

L'integrazione della robotica con l'intelligenza artificiale (IA) ha generato impatti significativi in vari settori, rivoluzionando applicazioni e tecnologie e ampliando le prospettive future. In ambito manifatturiero, i robot intelligenti stanno migliorando l'efficienza e la flessibilità delle linee di produzione, ottimizzando i processi grazie all'apprendimento automatico e alla manutenzione predittiva. I cobot, inoltre, permettono un'interazione sicura tra robot e operatori umani, aumentando la produttività.

Nel settore sanitario, l'IA ha potenziato i robot chirurgici, consentendo operazioni più precise e meno invasive. I robot assistivi e di riabilitazione offrono terapie personalizzate e monitoraggio continuo, mentre i robot domestici supportano persone anziane o con disabilità nelle attività quotidiane.

In ambito educativo e formativo, i robot dotati di IA sono impiegati per insegnare concetti complessi in modo interattivo, adattandosi ai diversi stili di apprendimento. I simulatori robotici consentono agli studenti di acquisire competenze pratiche in ambienti controllati, risultando particolarmente utili in ingegneria, medicina e aviazione.

La logistica e i trasporti beneficiano dell'uso di robot autonomi nei magazzini e di veicoli a guida autonoma per il trasporto di merci e persone, migliorando l'efficienza e riducendo gli errori. I droni, dotati di IA, sono utilizzati per la consegna rapida, il monitoraggio ambientale e la sorveglianza delle infrastrutture, aprendo nuove opportunità per la gestione delle risorse.

In agricoltura, i robot intelligenti ottimizzano la gestione delle colture, migliorando la resa e riducendo l'impatto ambientale. I droni agricoli forniscono dati dettagliati per identificare problemi nelle colture e supportare una gestione più precisa.

Nel settore dei servizi e dell'intrattenimento, i robot interattivi creano esperienze coinvolgenti in musei e centri commerciali, mentre i chatbot con IA automatizzano l'assistenza ai clienti, migliorando l'efficienza e la qualità del servizio.

Le prospettive future prevedono robot sempre più autonomi e intelligenti, capaci di collaborare efficacemente con gli esseri umani e adattarsi rapidamente a nuovi contesti. L'integrazione con tecnologie emergenti, come IoT, AR e blockchain, potrà ulteriormente potenziare le capacità dei robot, aprendo nuove opportunità per innovazione e progresso.

2. POLLEN ROBOTICS E L'INNOVATIVO ROBOT ANTROPOMORFO REACHY

Questo capitolo si focalizza su Pollen Robotics, l'azienda che ha sviluppato Reachy, un robot antropomorfo all'avanguardia, e sulle caratteristiche e potenzialità di questo robot. Pollen Robotics è un'azienda pionieristica nel campo della robotica che ha rapidamente guadagnato rilevanza grazie alla sua missione innovativa e alle sue collaborazioni strategiche.

Inizieremo con una panoramica della storia e della crescita di Pollen Robotics, esplorando le motivazioni dei fondatori, la missione e la visione che guidano le attività aziendali, e i principali progetti che hanno caratterizzato il loro percorso.

Successivamente, ci concentreremo su Reachy, fornendo una descrizione dettagliata delle sue caratteristiche, della sua struttura tecnica, delle sue funzionalità e delle sue capacità.

Reachy rappresenta un significativo passo avanti nella robotica antropomorfa grazie alla sua abilità di interagire in modo naturale con l'ambiente e gli esseri umani.

Esploreremo la composizione tecnica di Reachy, per comprendere come questi componenti lavorino insieme per creare un robot versatile e avanzato.

Inoltre, esamineremo le applicazioni pratiche di Reachy in vari settori, come l'educazione, l'industria, la sanità e altri ambiti, dimostrando come questo robot potrebbe trasformare le operazioni e aprire nuove possibilità.

2.1. POLLEN ROBOTICS

Pollen Robotics è una start-up innovativa nel campo della robotica avanzata, con una particolare enfasi sullo sviluppo di robot antropomorfi. Fondata con la visione di creare robot capaci di interagire in modo naturale con l'ambiente umano,

l'azienda si distingue per l'approccio multidisciplinare che combina ingegneria, intelligenza artificiale e design.

2.1.1. Storia e fondazione

Pollen Robotics è stata fondata nel 2016 con l'obiettivo di sviluppare tecnologie robotiche avanzate che potessero migliorare l'interazione uomo-macchina. La storia dell'azienda inizia con un gruppo di ingegneri e ricercatori appassionati di robotica e intelligenza artificiale, provenienti da prestigiose istituzioni accademiche francesi, tra cui l'INRIA (Institut National de Recherche en Informatique et en Automatique) e l'Université de Bordeaux.

I fondatori di Pollen Robotics sono Matthieu Lapeyre, Jonas Viens, e Pierre Rouanet. Con una solida esperienza in progetti di ricerca e sviluppo tecnologico, questi ingegneri hanno deciso di unire le loro competenze per creare un'azienda dedicata alla robotica antropomorfa.

L'idea di creare Pollen Robotics è nata dalla volontà dei fondatori di colmare il divario tra la ricerca accademica e le applicazioni pratiche della robotica. Con una visione chiara e innovativa, l'azienda si è posta l'obiettivo di sviluppare robot che non solo potessero eseguire compiti complessi, ma che fossero anche in grado di interagire in modo naturale ed efficace con gli esseri umani.

Il percorso di fondazione di Pollen Robotics è stato caratterizzato da alcune tappe fondamentali. Nel 2015, prima della fondazione ufficiale, i fondatori hanno lavorato su progetti indipendenti nel campo della robotica presso diversi istituti di ricerca. Questa esperienza ha permesso loro di acquisire una conoscenza approfondita delle tecnologie robotiche e delle sfide associate al loro sviluppo.

La svolta decisiva è avvenuta nel 2016, quando Matthieu Lapeyre, Jonas Viens e Pierre Rouanet hanno deciso di unire le proprie competenze e risorse per creare un'azienda dedicata allo sviluppo di robot antropomorfi avanzati. Grazie al supporto di finanziamenti iniziali ottenuti attraverso grant di ricerca e investimenti privati, Pollen Robotics è stata ufficialmente fondata. La sede principale è stata stabilita a Bordeaux, una città con un forte ecosistema tecnologico, facilitando l'accesso a talenti, risorse e infrastrutture necessarie per la crescita dell'azienda.

Uno degli elementi distintivi di Pollen Robotics fin dalla sua fondazione è stata la filosofia dell'open-source. I fondatori credevano fermamente che la condivisione delle conoscenze e delle tecnologie fosse fondamentale per accelerare l'innovazione nel campo della robotica. Pertanto, hanno deciso di rendere i progetti dell'azienda accessibili alla comunità scientifica e agli sviluppatori di tutto il mondo, promuovendo la collaborazione e lo scambio di idee.



Figura 2.1. Logo di Pollen Robotics

Negli anni successivi alla fondazione, Pollen Robotics ha avviato numerosi progetti di ricerca e sviluppo, collaborando con università, istituti di ricerca e altre aziende tecnologiche. Queste collaborazioni hanno portato alla creazione di prototipi innovativi e al consolidamento della reputazione dell'azienda come leader nel settore della robotica antropomorfa.

2.1.2. Missione e visione

La missione e la visione di Pollen Robotics sono i pilastri fondamentali che guidano tutte le attività dell'azienda e orientano il suo sviluppo strategico. Fin dalla sua fondazione, Pollen Robotics ha mirato a rivoluzionare il campo della robotica attraverso l'innovazione, la collaborazione e l'impegno verso l'eccellenza.

La missione di Pollen Robotics è quella di creare robot intelligenti e interattivi che possano migliorare la vita delle persone e contribuire al progresso tecnologico della società. Questo obiettivo si traduce in una serie di impegni chiave:

- **Innovazione tecnologica:** Pollen Robotics si dedica allo sviluppo di tecnologie robotiche all'avanguardia. Attraverso una costante attività di ricerca e sviluppo, l'azienda punta a creare soluzioni innovative che

superino i limiti attuali della robotica e aprano nuove possibilità di applicazione.

- **Accessibilità e open-source:** Un principio fondamentale della missione di Pollen Robotics è rendere la tecnologia robotica accessibile a tutti. L'azienda adotta un approccio open-source, rilasciando software e hardware sviluppati internamente per permettere a ricercatori, sviluppatori e appassionati di contribuire e beneficiare delle innovazioni tecnologiche.
- **Interazione uomo-robot:** Pollen Robotics si impegna a sviluppare robot che possano interagire in modo naturale ed efficace con gli esseri umani. L'obiettivo è creare macchine che non siano solo strumenti, ma veri e propri compagni in grado di comprendere e rispondere alle esigenze umane, migliorando la qualità della vita e facilitando attività quotidiane e professionali.
- **Sostenibilità e impatto sociale:** L'azienda pone grande enfasi sull'impatto sociale e ambientale delle sue tecnologie. Pollen Robotics lavora per sviluppare robot che possano contribuire a soluzioni sostenibili, ridurre gli sprechi e migliorare l'efficienza in vari settori. Inoltre, l'azienda è impegnata a considerare le implicazioni etiche della robotica, promuovendo uno sviluppo responsabile e consapevole.

La visione di Pollen Robotics è quella di diventare un leader globale nel campo della robotica antropomorfa, creando una nuova era in cui i robot intelligenti sono integrati nella società come partner fidati e indispensabili. La visione dell'azienda si articola in diversi punti chiave:

- **Eccellenza nella ricerca:** Pollen Robotics aspira a essere un punto di riferimento per l'innovazione scientifica e tecnologica. L'azienda investe continuamente in ricerca e sviluppo per spingere avanti i confini della conoscenza nel campo della robotica e dell'intelligenza artificiale.
- **Collaborazione globale:** Pollen Robotics vede la collaborazione come un elemento essenziale per il progresso. L'azienda promuove partnership con università, istituti di ricerca, industrie e comunità open-source a livello globale, al fine di condividere conoscenze, risorse e idee innovative.

- **Applicazioni pervasive:** La visione dell'azienda include l'integrazione dei robot in vari aspetti della vita quotidiana e professionale. Pollen Robotics immagina un futuro in cui i robot antropomorfi sono comuni in ambito domestico, industriale, educativo e sanitario, migliorando l'efficienza, la sicurezza e il benessere delle persone.
- **Formazione e istruzione:** Un altro elemento centrale della visione di Pollen Robotics è l'educazione. L'azienda si impegna a fornire risorse educative e piattaforme di apprendimento per formare la prossima generazione di ingegneri, programmatori e ricercatori nel campo della robotica.

In sintesi, la missione e la visione di Pollen Robotics riflettono un impegno profondo verso l'innovazione, l'accessibilità, la collaborazione e l'impatto positivo sulla società. Attraverso lo sviluppo di robot avanzati e interattivi, l'azienda mira a ridefinire il ruolo della robotica nel mondo moderno, creando un futuro in cui macchine intelligenti lavorano al fianco degli esseri umani per affrontare le sfide globali e migliorare la qualità della vita.

2.2. IL ROBOT ANTROPOMORFO REACHY

Il robot antropomorfo Reachy rappresenta uno dei progetti più ambiziosi e innovativi di Pollen Robotics. Progettato per interagire in modo naturale con gli esseri umani, Reachy è dotato di un'architettura avanzata che integra componenti meccanici, sensori sofisticati e potenti algoritmi di intelligenza artificiale. Questo robot non solo esemplifica l'eccellenza ingegneristica di Pollen Robotics, ma anche il loro impegno nel creare soluzioni robotiche che possano trovare applicazione in vari settori, dall'educazione alla sanità, fino alla ricerca e all'intrattenimento.

2.2.1. Descrizione generale di Reachy

Progettato per emulare in maniera realistica le caratteristiche fisiche e i movimenti degli esseri umani, Reachy si distingue per il suo design modulare e flessibile. Questo robot è caratterizzato da un aspetto esteticamente gradevole, con arti

articolati e una testa dotata di espressioni facciali, rendendolo particolarmente adatto per interazioni sociali e applicazioni di assistenza.

Reachy è stato concepito con l'obiettivo principale di facilitare la ricerca e lo sviluppo nel campo della robotica interattiva, fornendo una piattaforma accessibile e versatile. Le sue dimensioni sono progettate per essere compatibili con un ambiente umano standard, permettendo al robot di operare efficacemente in vari contesti, come laboratori di ricerca, istituzioni educative, e ambienti domestici.

Una delle caratteristiche che distingue Reachy dagli altri robot antropomorfi è la sua architettura open-source. Questa scelta progettuale consente a ricercatori, sviluppatori e appassionati di robotica di personalizzare e migliorare il robot in base alle loro esigenze specifiche. La documentazione completa e il supporto della comunità facilitano l'integrazione di nuove funzionalità e l'adattamento a diversi scenari applicativi.

Il design di Reachy è studiato per combinare robustezza e leggerezza, utilizzando materiali avanzati che garantiscono una struttura resistente senza compromettere la mobilità. Le articolazioni sono dotate di motori ad alta precisione, che permettono movimenti fluidi e naturali. Inoltre, il robot è equipaggiato con sensori avanzati per la percezione dell'ambiente, tra cui telecamere stereoscopiche e sensori di forza, che migliorano la sua capacità di interazione con gli esseri umani e l'ambiente circostante.

L'innovatività di Reachy risiede anche nella sua capacità di apprendimento e adattamento. Grazie agli algoritmi di intelligenza artificiale integrati, il robot può analizzare e interpretare i dati sensoriali in tempo reale, adattando il suo comportamento in risposta alle variazioni dell'ambiente e alle interazioni con gli utenti. Questa capacità rende Reachy particolarmente utile in applicazioni di assistenza personalizzata, dove è necessario un alto grado di adattabilità e comprensione del contesto.

2.2.2. Struttura e componenti tecnici

L'architettura tecnica di Reachy è un esempio avanzato di ingegneria robotica che combina hardware sofisticato e software di intelligenza artificiale per creare un

sistema versatile e altamente funzionale. Questa sezione esamina in dettaglio i componenti principali e le specifiche tecniche che costituiscono Reachy, mettendo in evidenza i progressi tecnologici che ne fanno una piattaforma ideale per la ricerca e le applicazioni pratiche.

Torso di Reachy

Il torso di Reachy ospita diversi componenti critici che garantiscono il funzionamento del robot:

- **Fronte:**
 1. **Alloggiamento motori e ventola per il collo orbita:** Questo alloggiamento contiene i motori e la ventola che raffredda il collo sferico di Reachy.
 2. **Scheda di alimentazione:** Fornisce energia a tutti i componenti del robot.
 3. **Computer integrato Intel NUC:** Un NUC Intel i5 quad-core 1.6GHz con 16GB DDR4 e 256GB SSD gestisce l'elaborazione centrale.
 4. **Microfono:** Seeed Studio ReSpeaker Mic Array v2.0 per input audio.
 5. **Hub USB:** Consente la connessione di vari dispositivi USB.
 6. **TPU Google Coral:** Acceleratore hardware per il machine learning.
 7. **Altoparlante destro:** Tectonic TEBM36S12 per l'output audio stereo.
 8. **Altoparlante sinistro:** Tectonic TEBM36S12 per l'output audio stereo.

9. **Pulsante volume altoparlanti:** Per regolare il volume degli altoparlanti.

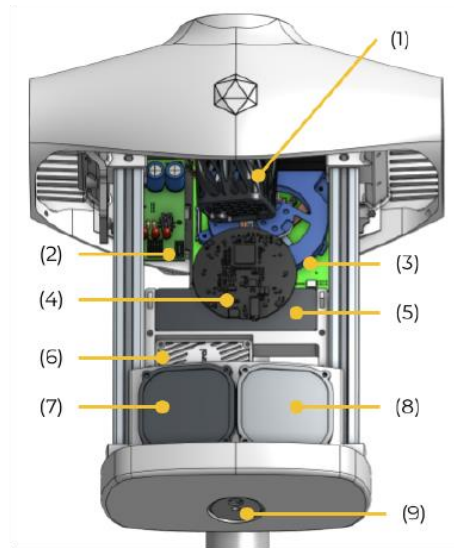


Figura 2.2. Torso frontale di Reachy

- **Retro:**
 - 10. **Interfaccia hardware:** Include diverse porte per connettività e controllo.
 - 11. **Interfaccia di fissaggio:** Per montare il robot in diverse configurazioni.

12. **Griglia di raffreddamento Coral:** Per dissipare il calore generato dall'acceleratore TPU.

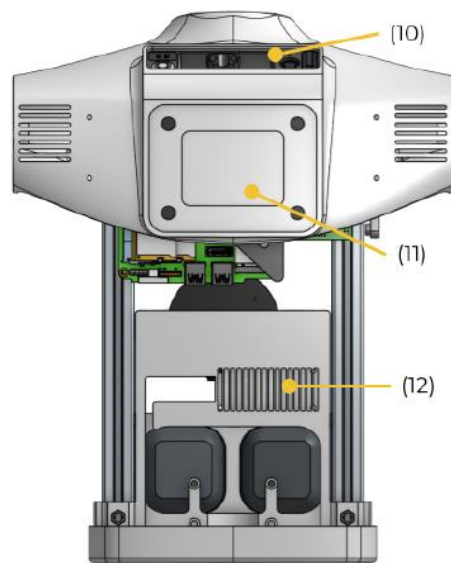


Figura 2.3. Torso posteriore di Reachy

L'interfaccia hardware comprende:

- a. Interruttore computer integrato (NUC)
- b. Porta Ethernet
- c. Porta USB-a 3.0 (per telecamere)
- d. Porta USB-a 2.0
- e. Porta USB-c
- f. Porta HDMI
- g. Porta di alimentazione
- h. Interruttore alimentazione motori

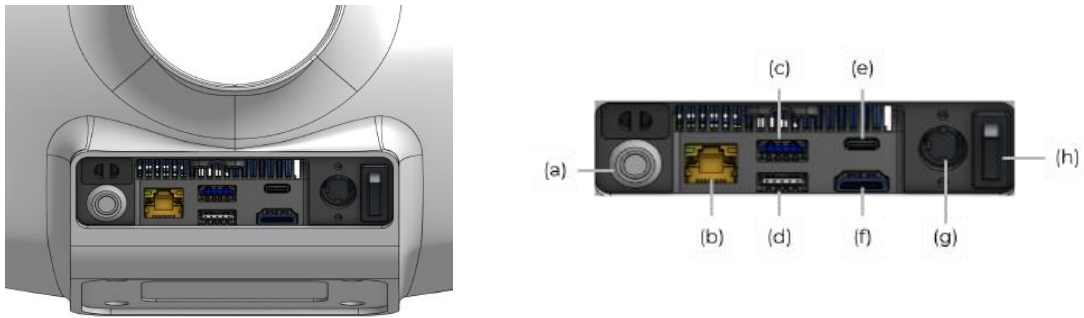


Figura 2.4. Interfaccia hardware di Reachy

Testa di Reachy

La testa di Reachy è dotata di caratteristiche avanzate per la percezione e l'interazione:

13. **Telecamere motorizzate:** Due telecamere 1080p@30fps con zoom motorizzato (FOV 65°-125°).
14. **Fissazioni motorizzate delle antenne:** Consentono movimenti dinamici per esprimere emozioni.
15. **Foro per l'instradamento dei cavi della testa:** Per organizzare i cavi di connessione.

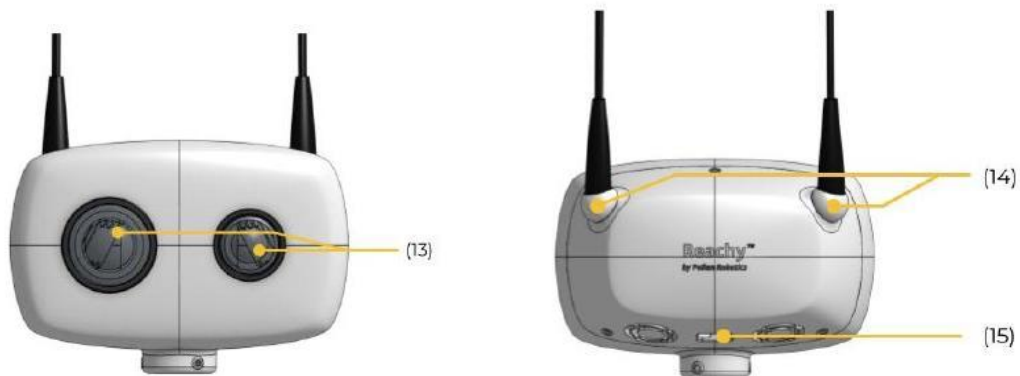


Figura 2.5. Testa di Reachy

Braccia di Reachy

L'Arm Kit di Reachy comprende:

1. **Scheda di alimentazione 12V (SMPS2Dynamixel):** Alimenta i motori delle braccia.
2. **Scheda USB-seriale:** Per il controllo delle braccia tramite il computer.

Le braccia di Reachy hanno sette gradi di libertà, consentendo movimenti fluidi e naturali, con servomotori ad alta precisione per il controllo della posizione e della forza applicata.



Figura 2.6. Braccia di Reachy

Reachy con base mobile

La configurazione di Reachy con la base mobile comprende il kit completo di Reachy, costituito da due braccia, un torso e una testa, e una base mobile omnidirezionale che permette al robot di muoversi in tutte le direzioni. La base mobile comunica con Reachy tramite un cavo USB collegato alla parte posteriore del robot, offrendo una maggiore versatilità e mobilità.



Figura 2.7. Reachy con base mobile

Specifiche generali

Le specifiche generali del kit completo e dello starter kit di Reachy 2023 includono:

- **Costruzione:** Stampa 3D MJF Verniciata, PU flessibile modellato, Alluminio
- **Consumo massimo di potenza:** 180W
- **Dimensioni del kit completo:** 670x450x200mm (670x1800x200mm con braccia estese)
- **Dimensioni dello starter kit:** 670x380x200mm (670x1008x200mm con braccia estese)
- **Peso:** 6.2 kg senza fissaggio metallico

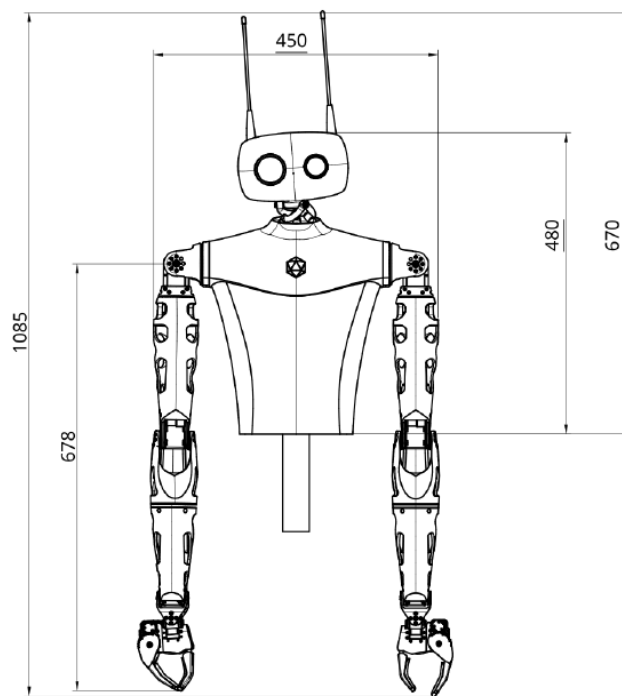


Figura 2.8. Specifiche generali di Rechy

Specifiche del braccio

Il braccio di Rechy è costruito con materiali di alta qualità e progettato per garantire movimenti fluidi e precisi. Le specifiche includono:

- **Materiali:** Stampa 3D MJF Verniciata, cuscinetti a rulli assiali
- **Consumo di energia:** 51W
- **Dimensioni:** 662x88x73mm

Ripartizione del Peso

- **Peso complessivo:** 1670g
- **Spalla:** 240g
- **Braccio superiore:** 610g
- **Avambraccio:** 590g
- **Pinza:** 230g

- **Carico massimo:** 500g (può variare in base alla configurazione di tenuta e durata)

Gradi di libertà

- **Braccio:** 7 gradi di libertà + 1 per la pinza
- **Motori:** 3 Dynamixel MX-106T, 1 Dynamixel MX-64AT, 4 Dynamixel MX-28AT

Ventole

- **Ventole:** 3 ventole (spalla, gomito e polso) per il raffreddamento dei motori.

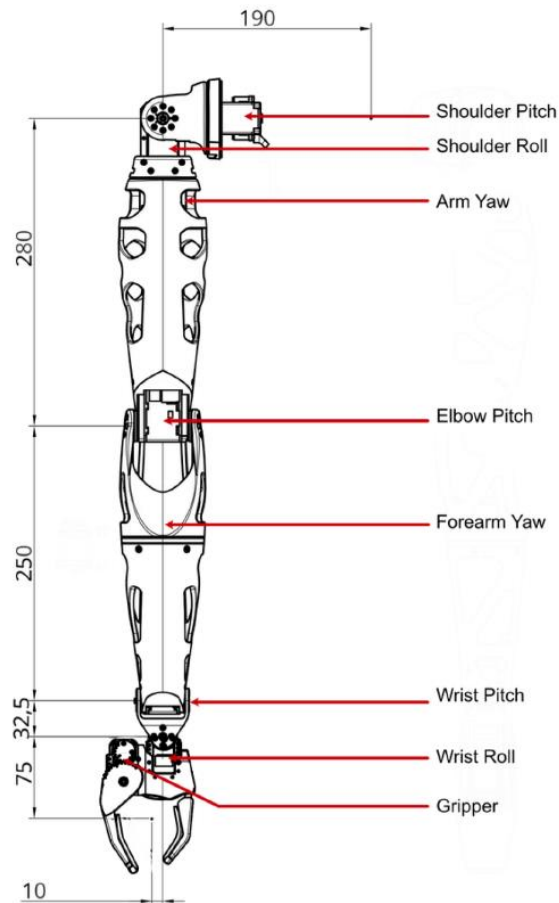


Figura 2.9. Specifiche del braccio di Reachy

Specifiche della pinza

La pinza di Reachy ha le seguenti caratteristiche:

- **Animazione:** Dynamixel MX-28AT
- **Sensore di carico:** Micro load cell da 0.78 Kg
- **Costruzione:** Stampa 3D MJF Verniciata e PU flessibile modellato
- **Consumo di energia:** 5.3W
- **Dimensioni:** 117.3x84x51.4mm
- **Peso:** 0.3Kg

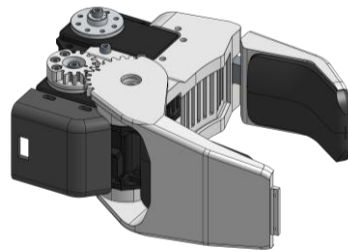


Figura 2.10. Specifiche della pinza di Reachy

Specifiche della testa

La testa di Reachy presenta le seguenti specifiche tecniche:

- **Costruzione:** Stampa 3D MJF Verniciata
- **Consumo di energia:** 15W
- **Dimensioni:** 175x253x108mm
- **Peso:** 0.675Kg

Caratteristiche principali

- **Telecamere:** Due telecamere motorizzate di alta qualità, dual 1080p@30fps con zoom motorizzato (FOV 65° a 125°) e due lenti ottiche.
- **Giunto del collo orbita:** Attuatore a giunto sferico motorizzato da 3 motori brushless Maxon con encoder magnetici assoluti.

- **Antenne:** Motorizzate da un motore Dynamixel, rimovibili, attaccate tramite un sistema di 3 magneti.

La tecnologia Orbita sviluppata da Pollen Robotics consente movimenti dinamici multi-direzionali, mentre le antenne animate permettono a Reachy di esprimere emozioni.



Figura 2.11. Specifiche della testa di Reachy

Specifiche del Torso

Il torso include:

- **Costruzione:** Stampa 3D MJF verniciata e alluminio
- **Consumo di energia:** 36W
- **Dimensioni:** 300x350x150mm
- **Peso:** 1.7Kg

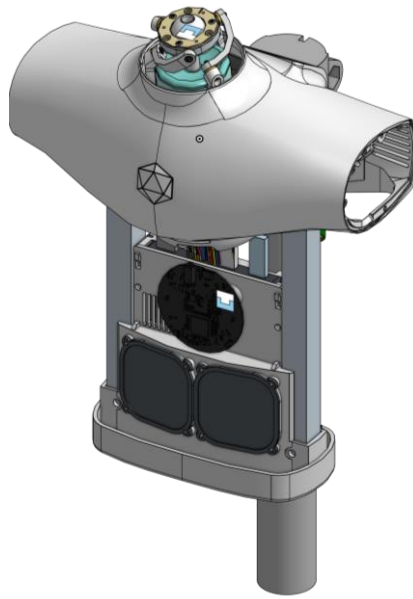


Figura 2.12. Specifiche del torso di Rechy

Specifiche di Orbita

Orbita è il giunto sferico che permette movimenti dinamici della testa:

- **Costruzione:** Parti lavorate in alluminio, ingranaggi in acciaio, cuscinetti metallici
- **Consumo di energia:** 8W
- **Tensione:** 12V
- **Velocità a vuoto:** 25 rpm
- **Coppia massima continua:** 1.7 Nm
- **Rapporto di riduzione:** 4.2666
- **Dimensioni:** 70mm di diametro, 170mm di altezza
- **Peso:** 850g
- **Gradi di libertà:** Roll (-46° a 46°), Pitch (-46° a 46°), Yaw (360°)

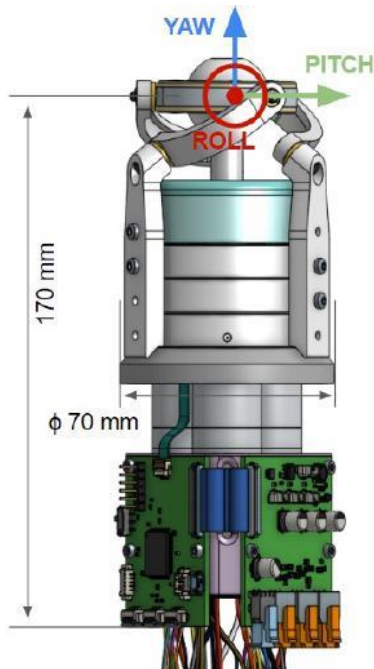


Figura 2.13. Specifiche di orbita di Reachy

Specifiche della Base Mobile

La base mobile consente a Reachy di muoversi autonomamente:

- **Costruzione:** Forma cilindrica, alluminio, parti in plastica stampate in 3D MJF, acciaio
- **Dimensioni:** 50x25 cm
- **Peso:** 25 kg
- **Capacità di carico:** 80 kg
- **Sensori:** Sensori di Hall e IMU su ogni ruota, Lidar RP S2 con raggio di 30 m e 32k misurazioni/s
- **Ruote:** Tre ruote omnidirezionali con potenza massima di 300W
- **Batteria OlenBox M:** 24V, 35Ah con BMS, dimensioni 19,5x17,2x13,4 cm, peso 6,5 kg

La base mobile è dotata di un sistema di arresto di emergenza che può essere posizionato sul robot o tenuto dall'utente, garantendo un arresto immediato in caso di necessità.

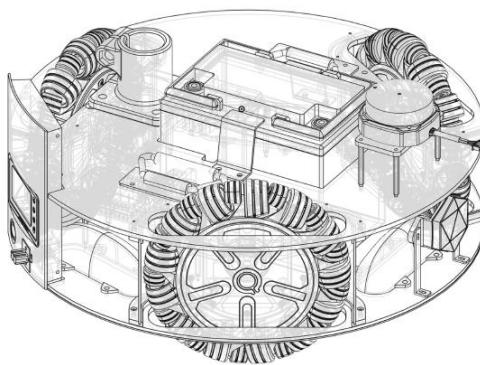


Figura 2.14. Base mobile di Rechy

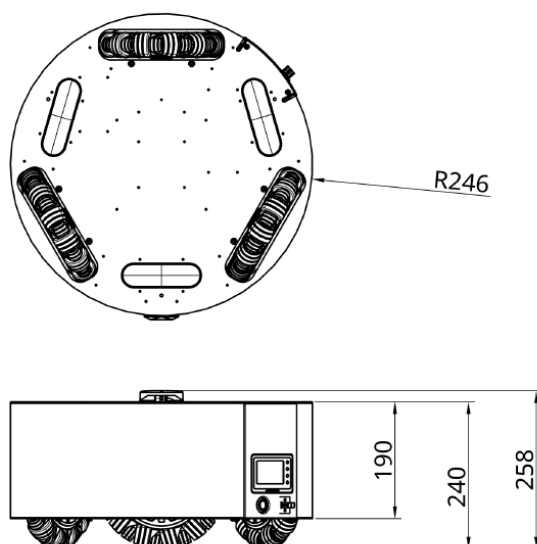


Figura 2.15. Specifiche della base mobile di Rechy

In sintesi, l'architettura tecnica di Rechy combina componenti hardware avanzati e software di intelligenza artificiale per creare un sistema robotico versatile e altamente funzionale, adatto a una vasta gamma di applicazioni. La modularità, la capacità di apprendimento e l'interfaccia user-friendly rendono Rechy una piattaforma ideale per promuovere l'innovazione e la ricerca nel campo della robotica moderna.

2.2.3. Funzionalità e applicazioni

Reachy è una piattaforma robotica e di intelligenza artificiale umanoide versatile, progettata per integrare manipolazione di oggetti, espressività, apprendimento automatico e interazione uomo-robot in un unico sistema modulare e facilmente installabile. Dotato di un'ampia gamma di funzionalità, Reachy è adatto a numerosi ambiti applicativi, dalla ricerca scientifica all'assistenza sanitaria, fino all'intrattenimento e alla formazione.

Una delle principali caratteristiche è la possibilità di essere controllato da remoto tramite realtà virtuale, permettendo all'utente di muovere le braccia, le mani e la testa di Reachy in tempo reale grazie a un'interfaccia intuitiva e alle telecamere integrate. Questa capacità di teleoperazione lo rende ideale per attività che richiedono interazioni precise e sicure, come il supporto in ambienti complessi.

La base robotica omnidirezionale, dotata di tre omniwheel e un sistema LiDAR, consente a Reachy di navigare agilmente in diversi contesti operativi, espandendo il suo raggio d'azione e permettendogli di interagire efficacemente con l'ambiente circostante. La capacità di movimento, combinata con i sensori avanzati, offre un alto livello di adattabilità in molteplici applicazioni.

L'espressività è un'altra caratteristica distintiva di Reachy: la tecnologia Orbita© consente alla testa del robot di imitare espressioni umane e di utilizzare le antenne per trasmettere emozioni, facilitando la creazione di un legame empatico con gli utenti. Questa capacità di comunicazione non verbale è particolarmente utile in ambiti come l'assistenza sociale e l'educazione.

Le braccia robotiche di Reachy, con sette gradi di libertà, sono progettate per la manipolazione avanzata, sollevando fino a 500 grammi con precisione e destrezza. Grazie alla pinza adattabile, le braccia possono essere configurate per diverse applicazioni, rendendo Reachy un valido supporto sia in contesti di ricerca che in attività produttive leggere.

La piattaforma supporta avanzati strumenti di apprendimento automatico, come TensorFlow, grazie a un'accelerazione hardware interna. Con le sue telecamere e il sistema audio integrato, Reachy può eseguire compiti complessi di rilevamento e

interazione, consentendo un'ampia gamma di applicazioni di visione artificiale e dialogo.

Reachy è particolarmente utile in settori come l'assistenza sanitaria, dove può monitorare i pazienti, somministrare farmaci e assistere persone con disabilità. La sua capacità di operare in modo autonomo e sicuro lo rende adatto a ospedali, case di riposo e strutture mediche.

Nel contesto educativo, Reachy offre un'opportunità unica per insegnare programmazione, robotica e intelligenza artificiale attraverso un'interazione pratica e coinvolgente. Le università e i centri di formazione possono utilizzarlo per progetti multidisciplinari, consentendo agli studenti di sperimentare con tecnologie avanzate in un ambiente sicuro e controllato.

Nel settore dei servizi e dell'intrattenimento, Reachy può essere impiegato come assistente in centri commerciali, hotel e showroom, interagendo con il pubblico in modo naturale e coinvolgente. La sua capacità di eseguire compiti ripetitivi lo rende ideale per l'automazione di mansioni come la distribuzione di documenti o la preparazione di bevande.

La modularità e l'adattabilità di Reachy permettono di espanderne continuamente le capacità con nuovi moduli e aggiornamenti hardware, rendendolo una piattaforma flessibile e innovativa per esplorare le frontiere dell'intelligenza artificiale e della robotica. Reachy rappresenta, quindi, una combinazione di tecnologia avanzata e accessibilità, in grado di soddisfare esigenze diverse e contribuire allo sviluppo di soluzioni innovative in numerosi campi.

3. SETUP DELL'AMBIENTE DI SVILUPPO E TEST PRELIMINARI DEL SIMULATORE DI REACHY

In questo capitolo viene illustrato il processo di configurazione dell'ambiente di sviluppo e testing necessario per lavorare con il robot antropomorfo Reachy. La predisposizione di un'infrastruttura tecnologica completa e funzionale è un requisito fondamentale per garantire l'efficacia e la precisione delle attività di ricerca e sperimentazione. Attraverso l'installazione e l'integrazione di diversi strumenti software, si creano le condizioni per sviluppare e testare script complessi, ottimizzando il controllo e le capacità operative del robot.

Vengono descritte in dettaglio le principali componenti software utilizzate, tra cui linguaggi di programmazione, ambienti di sviluppo integrati e librerie specializzate, che permettono di scrivere e gestire efficacemente gli script necessari al funzionamento di Reachy. Parallelamente, sono presentate le risorse aggiuntive essenziali, come repository GitHub e pacchetti di configurazione, che consentono di sincronizzare il simulatore Unity 3D con l'ambiente di sviluppo, fornendo un supporto completo per la simulazione e la verifica preliminare delle funzionalità del robot.

La corretta configurazione di questi strumenti e risorse è cruciale non solo per garantire un workflow fluido e privo di interruzioni, ma anche per assicurare la replicabilità e l'affidabilità dei risultati sperimentali ottenuti. Un ambiente di sviluppo ben strutturato permette infatti di minimizzare i potenziali errori e di affrontare in maniera sistematica le sfide poste dalle interazioni complesse tra il software e l'hardware di Reachy.

3.1. COMPONENTI E RISORSE NECESSARIE

La configurazione di un ambiente di sviluppo adeguato richiede l'installazione di specifiche componenti software e l'accesso a risorse mirate, indispensabili per garantire il corretto funzionamento e la gestione del robot Reachy. Vengono descritti gli strumenti e le librerie necessari per la scrittura, l'esecuzione e il testing degli script, nonché le risorse esterne che supportano l'integrazione e l'ottimizzazione del simulatore e dell'ambiente di sviluppo.

3.1.1. Installazione dei componenti

Per realizzare uno sviluppo efficace degli script per il robot antropomorfo Reachy, è fondamentale predisporre un ambiente di sviluppo dotato di strumenti e librerie specifici. Le componenti software descritte di seguito costituiscono la base tecnica necessaria per la scrittura, l'editing e l'esecuzione degli script, garantendo un workflow ottimale e un'interfaccia intuitiva per l'interazione con il robot.

Tabella 3.1. Componenti necessari per l'ambiente di sviluppo

Componente	Utilizzo
Python	Linguaggio di programmazione principale per la realizzazione degli script che controllano le funzionalità di Reachy.
Visual Studio Code	Ambiente di sviluppo integrato (IDE) utilizzato per la scrittura e la modifica del codice Python.
Plugin di Visual Studio Code: Python	Estensione che abilita il supporto per la sintassi Python, facilitando l'esecuzione e il debugging del codice all'interno dell'IDE.
Plugin di Visual Studio Code: Jupyter Notebooks	Strumento che consente l'esecuzione interattiva dei blocchi di codice Python in formato notebook, utile per l'esplorazione rapida e la visualizzazione dei risultati.

Libreria Python: <i>'reachy-sdk==0.5.4'</i>	SDK ufficiale sviluppato da Pollen Robotics per l'interfacciamento e il controllo completo del robot Reachy.
Libreria Python: <i>'pollen-vision'</i>	Libreria per l'elaborazione delle immagini, essenziale per implementare funzionalità di riconoscimento visivo avanzato in Reachy.
Libreria Python: <i>'opencv'</i>	Libreria utilizzata per l'elaborazione di immagini e video, fondamentale per implementare algoritmi di visione artificiale e riconoscimento di oggetti.
Libreria Python: <i>'openai'</i>	Utilizzata per integrare le funzionalità di intelligenza artificiale basate su modelli di linguaggio negli script Python, migliorando l'interazione del robot con l'ambiente e gli utenti.
Git	Sistema di controllo di versione indispensabile per tracciare le modifiche al codice, collaborare con altri sviluppatori e gestire il ciclo di vita del software.
Unity	Piattaforma di simulazione 3D utilizzata per creare ambienti virtuali realistici in cui testare e ottimizzare gli script di Reachy, senza la necessità di accedere direttamente al robot.

Questi componenti, integrati e configurati correttamente, rappresentano gli strumenti essenziali per lo sviluppo e la gestione del codice, offrendo un supporto completo dall'editing iniziale fino al testing e al debugging degli script.

3.1.2. Downloading delle risorse

Oltre alle componenti software, è necessario accedere a risorse aggiuntive per completare l'ambiente di sviluppo e abilitare funzionalità avanzate nel simulatore di Reachy. Le risorse elencate di seguito, comprese le repository GitHub e pacchetti specifici, forniscono codice sorgente, documentazione e strumenti di configurazione indispensabili per l'integrazione e l'utilizzo ottimale del robot.

Tabella 3.2. Risorse necessarie per l'ambiente di sviluppo

Risorse	Utilizzo
Repository GitHub: <i>'reachy-sdk'</i>	Contiene il codice sorgente e la documentazione dell'SDK di Reachy, fondamentali per il controllo e la gestione del robot.
Repository GitHub: <i>'reachy2021-unity-package'</i>	Include il pacchetto necessario per l'integrazione e la simulazione di Reachy all'interno dell'ambiente Unity, permettendo di testare gli script in un contesto virtuale.
Repository GitHub: <i>'pollen-vision'</i>	Fornisce librerie e strumenti per il riconoscimento visivo, essenziali per implementare capacità di percezione avanzata in Reachy.
Risorsa: <i>'grpc_unity_package.2.47.0-dev202204190851.zip'</i>	Pacchetto di configurazione gRPC per stabilire la comunicazione tra Unity e Reachy, garantendo una trasmissione fluida dei dati tra i due ambienti.

Queste risorse, adeguatamente scaricate e configurate, completano l'infrastruttura necessaria per lo sviluppo e il testing avanzato, facilitando la connessione tra il simulatore Unity e il robot Reachy e potenziando le capacità operative del sistema.

3.2. CONFIGURAZIONE E AVVIAMENTO DEL SIMULATORE DI REACHY CON UNITY 3D

Con i componenti installati e le risorse necessarie scaricate, il prossimo passo fondamentale è configurare e avviare il simulatore di Reachy utilizzando Unity 3D. Unity è una piattaforma potente e versatile per lo sviluppo di applicazioni interattive in 3D, che permette di creare simulazioni realistiche e dettagliate dei robot antropomorfi come Reachy. Questa fase del processo è cruciale per assicurarsi che

il simulatore funzioni correttamente e che sia pronto per eseguire e testare gli script Python sviluppati.

3.3.1. Configurazione di Unity per Reachy

L'installazione di Unity Hub e la configurazione dell'Editor Unity sono passaggi fondamentali per preparare l'ambiente di sviluppo necessario per lavorare con il simulatore di Reachy. Unity Hub è un'applicazione di gestione che facilita l'installazione, l'aggiornamento e la gestione delle diverse versioni del Unity Editor, nonché dei progetti associati.

Configurazione di Unity Hub

1. Primo avvio di Unity Hub:

- Al primo avvio di Unity Hub, l'applicazione aprirà la pagina di benvenuto.
- È necessario effettuare il login con il proprio Unity ID o creare un nuovo account se non se ne possiede già uno.
- Questo passaggio è essenziale per sbloccare tutte le funzionalità di Unity Hub e per sincronizzare le installazioni con il proprio account.

2. Gestione delle installazioni del Unity Editor:

- Unity Hub permette di installare e gestire diverse versioni del Unity Editor, garantendo la compatibilità con vari progetti.
- Per installare una nuova versione del Unity Editor, navigare alla scheda "Installs".
- Cliccare sul pulsante "Add" per selezionare la versione desiderata del Unity Editor.
- Seguire le istruzioni a schermo per completare il processo di installazione.

3. Installazione dell'Unity Editor:

- Selezione della Versione dell'Editor (2021.3.24f1):
- In Unity Hub, andare alla scheda "Installs" e cliccare su "Add".
- Selezionare la versione specifica dell'Editor (ad esempio, 2021.3.24f1) che è stata testata e raccomandata per lavorare con il simulatore di Reachy.

- È possibile scegliere tra versioni ufficiali, beta e alpha a seconda delle esigenze del progetto e della stabilità desiderata.

4. Aggiunta di componenti all'installazione:

- Durante il processo di installazione dell'Editor, sarà possibile selezionare componenti aggiuntivi.
- Questi componenti includono supporto per piattaforme specifiche (ad esempio, Android, iOS) e strumenti aggiuntivi come il supporto per il debugging e le estensioni di sviluppo.
- Selezionare i componenti necessari e cliccare su "Done" per completare l'installazione.

5. Gestione delle installazioni:

- Unity Hub consente di gestire facilmente tutte le installazioni del Unity Editor.
- È possibile impostare una versione preferita che verrà utilizzata come predefinita per nuovi progetti.
- Inoltre, Unity Hub permette di aggiungere o rimuovere componenti aggiuntivi in qualsiasi momento, facilitando l'aggiornamento dell'ambiente di sviluppo in base alle necessità del progetto.

La configurazione di Unity tramite Unity Hub è essenziale per preparare l'ambiente di sviluppo necessario per lavorare con il simulatore di Reachy. Seguendo i passaggi dettagliati, dall'installazione di Unity Hub all'aggiunta e gestione delle diverse versioni dell'Unity Editor, è possibile assicurarsi che l'ambiente sia ottimizzato per le esigenze del progetto.

3.3.2. Importazione del progetto

L'importazione del progetto Reachy in Unity è una procedura che richiede precisione e attenzione per garantire una corretta configurazione e gestione del progetto. I seguenti passaggi descrivono in dettaglio il processo, con l'obiettivo di fornire un ambiente di sviluppo organizzato ed efficiente.

1. Preparazione delle risorse:

- **Estrazione del pacchetto:** In primo luogo, è necessario estrarre il pacchetto `'grpc_unity_package.2.47.0-dev202204190851.zip'`, precedentemente scaricato. Questo pacchetto contiene la cartella `'Plugin'`, che è essenziale per l'integrazione delle funzionalità di gRPC in Unity.
- **Clonazione della repository:** La repository `'reachy2021-unity-package'`, clonata dal profilo GitHub di Pollen Robotics, deve essere già disponibile sul sistema locale. Questa repository contiene la base del progetto Reachy in Unity.

2. Organizzazione dei file:

- **Estrazione della repository:** è necessario estrarre la repository `'reachy2021-unity-package.zip'`, precedentemente scaricata.
- **Copia della cartella plugin:** La cartella `'Plugin'` estratta dal pacchetto deve essere copiata nella cartella `'Assets'` all'interno della directory `'reachy2021-unity-package'`. La struttura risultante dovrebbe essere `reachy2021-unity-package/Assets/Plugin`.
- **Rinominare la cartella del progetto:** Per migliorare la chiarezza e l'organizzazione, rinominare la cartella `'reachy2021-unity-package'` in `'Reachy in OfficeScene'`. Questo nuovo nome riflette meglio il contenuto e l'obiettivo del progetto.

3. Apertura del progetto con UnityHub:

- **Configurazione dell'apertura:** Utilizzare UnityHub per aprire il progetto rinominato `'Reachy in OfficeScene'`. UnityHub è uno strumento che facilita la gestione di progetti Unity, permettendo di selezionare facilmente versioni specifiche di Unity Editor compatibili con il progetto.
- **Caricamento della scena:** All'apertura del progetto in Unity, dovrebbe apparire la scena predefinita con Reachy all'interno di un ufficio. Questa scena è stata creata da Pollen Robotics per consentire agli sviluppatori di testare i loro script in un ambiente controllato e realistico.

4. Descrizione della scena:

- **Ambiente di lavoro:** La scena rappresenta un ufficio con pareti di mattoni chiari, pavimento in legno e vari oggetti di scena che simulano un ambiente di lavoro reale.
- **Oggetti e posizionamento:** Al centro della scena, c'è un tavolo su cui è posizionato Reachy, il robot antropomorfo. Sul tavolo sono presenti diversi oggetti, come una lampada, una tazza, una pianta e una piccola palla da basket. Questi oggetti possono essere utilizzati per interazioni e test delle funzionalità di Reachy.
- **Struttura della scena:** Nell'angolo destro, si trova una scaffalatura vuota e un cestino per i rifiuti. Questi elementi aggiuntivi offrono ulteriori opportunità per sperimentare il movimento, la manipolazione e il riconoscimento degli oggetti da parte di Reachy.

Seguendo questi, è possibile importare e gestire efficacemente il progetto Reachy in Unity, creando un ambiente di sviluppo strutturato e produttivo.

3.3. VALUTAZIONE DELL'EFFETTIVO FUNZIONAMENTO DEL SIMULATORE

Dopo aver configurato e avviato il simulatore di Reachy, è essenziale valutare il suo funzionamento per assicurarsi che tutte le componenti operino correttamente. Questa fase di valutazione è cruciale per identificare e risolvere eventuali problemi, garantendo così che il simulatore sia pronto per supportare lo sviluppo e il test degli script Python in modo efficace. Una corretta valutazione del simulatore permette di ottenere un ambiente di test affidabile e accurato, fondamentale per la validazione delle funzionalità di Reachy.

3.4.1. Esecuzione di script di test forniti da Pollen Robotics

In questo paragrafo, analizzeremo i risultati ottenuti dai test eseguiti sui due script Jupyter Notebook forniti da Pollen Robotics per verificare il funzionamento del simulatore di Reachy. Gli script, denominati *'Testing the head.ipynb'* e *'Testing the*

`arm.ipynb`’, sono stati prelevati dalla repository GitHub ‘*reachy-sdk*’ nel percorso `reachy-sdk/reachy_sdk/examples`.

Dettagli del codice ‘*Testing the head.ipynb*’

Obiettivo dello script

Questo script è progettato per testare il funzionamento della testa di Reachy, verificando la capacità del robot di orientare la testa verso diverse direzioni e di utilizzare le antenne per esprimere emozioni.

Analisi del codice

1. Importazione delle librerie

Importa la classe `ReachySDK` dal modulo `reachy_sdk`, necessaria per interagire con il robot Reachy.

```
from reachy_sdk import ReachySDK
```

2. Connessione con Reachy

Crea un'istanza della classe `ReachySDK` e stabilisce una connessione con Reachy utilizzando l'host locale.

```
reachy = ReachySDK(host='localhost')
```

3. Rilevamento della testa

Verifica la presenza della testa di Reachy, assicurandosi che sia correttamente rilevata.

```
reachy.head
```

4. Accensione della testa

Accende la testa di Reachy, rendendo operativi i motori e i sensori associati.

```
reachy.turn_on('head')
```


5. Orientamento della testa verso un punto specifico

- Comanda alla testa di Reachy di orientarsi verso le coordinate (x=0.5, y=0, z=0) in un tempo di 1 secondo, partendo dalle posizioni attuali di neck_roll, neck_pitch e neck_yaw.
- Importa il modulo time per gestire le pause nei movimenti.

```
reachy.head.look_at(  
    x=0.5,  
    y=0,  
    z=0,  
    duration=1.0,  
    starting_positions={  
        reachy.head.neck_roll:  
reachy.head.neck_roll.goal_position,  
        reachy.head.neck_pitch:  
reachy.head.neck_pitch.goal_position,  
        reachy.head.neck_yaw:  
reachy.head.neck_yaw.goal_position  
    })  
  
import time
```

6. Movimenti della testa (destra, giù, sinistra, davanti)

Serie di comandi per far orientare la testa di Reachy verso destra, giù, sinistra e infine davanti, con pause di 0.5 secondi tra un movimento e l'altro.

```
look_right = reachy.head.look_at(  
    x=0.5,  
    y=-0.5,  
    z=0.1,  
    duration=1.0,  
    starting_positions={  
        reachy.head.neck_roll:  
reachy.head.neck_roll.goal_position,  
        reachy.head.neck_pitch:  
reachy.head.neck_pitch.goal_position,  
        reachy.head.neck_yaw:  
reachy.head.neck_yaw.goal_position  
    })  
  
time.sleep(0.5)  
  
look_down = reachy.head.look_at(  
    x=0.5,  
    y=0,  
    z=-0.4,  
    duration=1.0,  
    starting_positions={  
        reachy.head.neck_roll:  
reachy.head.neck_roll.goal_position,
```

```

        reachy.head.neck_pitch:
reachy.head.neck_pitch.goal_position,
        reachy.head.neck_yaw:
reachy.head.neck_yaw.goal_position
    })

time.sleep(0.5)

look_left = reachy.head.look_at(
    x=0.5,
    y=0.3,
    z=-0.3,
    duration=1.0,
    starting_positions={
        reachy.head.neck_roll:
reachy.head.neck_roll.goal_position,
        reachy.head.neck_pitch:
reachy.head.neck_pitch.goal_position,
        reachy.head.neck_yaw:
reachy.head.neck_yaw.goal_position
    })

time.sleep(0.5)

look_front = reachy.head.look_at(
    x=0.5,
    y=0,
    z=0,
    duration=1.0,
    starting_positions={
        reachy.head.neck_roll:
reachy.head.neck_roll.goal_position,
        reachy.head.neck_pitch:
reachy.head.neck_pitch.goal_position,
        reachy.head.neck_yaw:
reachy.head.neck_yaw.goal_position
    })

```

7. Seguimento del braccio destro

Comanda la testa di Reachy a seguire il movimento del braccio destro utilizzando la cinematica diretta. Se Reachy non ha un braccio destro, stampa un messaggio di errore.

```

try:
    x, y, z = reachy.r_arm.forward_kinematics()[ :3,-1]
    reachy.head.look_at(x=x, y=y, z=z-0.05, duration=1.0)

    time.sleep(0.5)

    while True:
        x, y, z = reachy.r_arm.forward_kinematics()[ :3,-1]
        gp_dic = reachy.head._look_at(x, y, z - 0.05)
        reachy.head.neck_roll.goal_position =
gp_dic[reachy.head.neck_roll]

```

```

        reachy.head.neck_pitch.goal_position =
gp_dic[reachy.head.neck_pitch]
        reachy.head.neck_yaw.goal_position =
gp_dic[reachy.head.neck_yaw]
        time.sleep(0.01)
except AttributeError:
    print('Reachy has no right arm!')

```

8. Seguimento del braccio sinistro

Simile alla settima cella, ma per il braccio sinistro di Reachy.

```

try:
    x, y, z = reachy.l_arm.forward_kinematics()[ :3,-1]
    reachy.head.look_at(x=x, y=y, z=z-0.05, duration=1.0)

    time.sleep(0.5)

    while True:
        x, y, z = reachy.l_arm.forward_kinematics()[ :3,-1]
        gp_dic = reachy.head._look_at(x, y, z - 0.05)
        reachy.head.neck_roll.goal_position =
gp_dic[reachy.head.neck_roll]
        reachy.head.neck_pitch.goal_position =
gp_dic[reachy.head.neck_pitch]
        reachy.head.neck_yaw.goal_position =
gp_dic[reachy.head.neck_yaw]
        time.sleep(0.01)
except AttributeError:
    print('Reachy has no left arm!')

```

9. Movimento dell'antennina sinistra

```
reachy.head.l_antenna.goal_position = -40
```

10. Reimpostazione dell'antennina sinistra

```
reachy.head.l_antenna.goal_position = 0
```

11. Movimento dell'antennina destra

```
reachy.head.r_antenna.goal_position = -40
```

12. Reimpostazione dell'antennina destra

```
reachy.head.r_antenna.goal_position = 0
```

13. Definizione di funzioni per le antenne

Definisce due funzioni `happy_antennas` e `sad_antennas` per animare le antenne di Reachy, esprimendo emozioni di felicità e tristezza.

```

def happy_antennas():
    reachy.head.l_antenna.speed_limit = 0.0
    reachy.head.r_antenna.speed_limit = 0.0

    for _ in range(9):
        reachy.head.l_antenna.goal_position = 10.0
        reachy.head.r_antenna.goal_position = -10.0

        time.sleep(0.1)

        reachy.head.l_antenna.goal_position = -10.0
        reachy.head.r_antenna.goal_position = 10.0

        time.sleep(0.1)

    reachy.head.l_antenna.goal_position = 0.0
    reachy.head.r_antenna.goal_position = 0.0

def sad_antennas():
    reachy.head.l_antenna.speed_limit = 70.0
    reachy.head.r_antenna.speed_limit = 70.0

    reachy.head.l_antenna.goal_position = 140.0
    reachy.head.r_antenna.goal_position = -140.0

    time.sleep(5.0)

    reachy.head.l_antenna.goal_position = 0.0
    reachy.head.r_antenna.goal_position = 0.0

```

14. Esecuzione della funzione happy_antennas

```
happy_antennas()
```

15. Reimpostazione della posizione della testa

```
reachy.head.look_at(x=0.5, y=0, z=0, duration=1.0)
```

16. Spegnimento della testa

```
reachy.turn_off('head')
```

17. Importazione della libreria PIL per la visualizzazione delle immagini

```
from PIL import Image
```

18. Visualizzazione dell'immagine dalla camera sinistra

```
Image.fromarray(reachy.left_camera.last_frame[:, :, ::-1])
```

19. Visualizzazione dell'immagine dalla camera destra

Le ultime tre celle importano la libreria PIL e visualizzano le immagini catturate dalle telecamere sinistra e destra di Reachy.

```
Image.fromarray(reachy.right_camera.last_frame[:, :, ::-1])
```

Conclusione

L'analisi del codice di *'Testing the head.ipynb'* mostra un'implementazione dettagliata e strutturata per testare il funzionamento della testa di Reachy. Ogni cella del notebook svolge una funzione specifica per verificare i movimenti e le capacità della testa e delle antenne del robot, assicurandosi che tutte le componenti siano operative e rispondano correttamente ai comandi.

Dettagli del codice *'Testing the arm.ipynb'*

Obiettivo dello script:

Questo script è progettato per testare il funzionamento delle braccia di Reachy, verificando la capacità del robot di eseguire movimenti complessi utilizzando la cinematica inversa.

Analisi del codice:

1. Importazione delle librerie

- Importa `numpy` per operazioni matematiche.
- Importa la classe `ReachySDK` dal modulo `reachy_sdk` per interagire con il robot Reachy.
- Importa la funzione `goto` e la classe `InterpolationMode` dal modulo `reachy_sdk.trajectory` per gestire i movimenti delle articolazioni.

```
import numpy as np

from reachy_sdk import ReachySDK

from reachy_sdk.trajectory import goto
from reachy_sdk.trajectory.interpolation import
InterpolationMode
```

2. Connessione con Reachy

Crea un'istanza della classe `ReachySDK` e stabilisce una connessione con Reachy utilizzando l'host locale.

```
reachy = ReachySDK('localhost')
```

3. Verifica di Reachy

Verifica la connessione e la configurazione di Reachy.

```
reachy
```

4. Accensione di Reachy

Accende Reachy, rendendo operativi tutti i motori e i sensori.

```
reachy.turn_on('reachy')
```

5. Spegnimento di Reachy

Spegne Reachy, disattivando tutti i motori e i sensori.

```
reachy.turn_off('reachy')
```

6. Riaccensione di Reachy

Riaccende Reachy, ripristinando l'operatività dei motori e dei sensori.

```
reachy.turn_on('reachy')
```

7. Spegnimento graduale di Reachy

Spegne gradualmente Reachy per evitare movimenti bruschi.

```
reachy.turn_off_smoothly('reachy')
```

8. Posizioni angolari delle braccia

Definisce le posizioni angolari delle articolazioni delle braccia destra e sinistra. Se Reachy non ha un braccio, l'errore viene gestito.

```
try:
    right_angled_position_right = {
        reachy.r_arm.r_shoulder_pitch: 0,
        reachy.r_arm.r_shoulder_roll: 0,
        reachy.r_arm.r_arm_yaw: 0,
        reachy.r_arm.r_elbow_pitch: -90,
        reachy.r_arm.r_forearm_yaw: 0,
```

```

        reachy.r_arm.r_wrist_pitch: 0,
        reachy.r_arm.r_wrist_roll: 0,
    }
except AttributeError:
    pass

try:
    right_angled_position_left = {
        reachy.l_arm.l_shoulder_pitch: 0,
        reachy.l_arm.l_shoulder_roll: 0,
        reachy.l_arm.l_arm_yaw: 0,
        reachy.l_arm.l_elbow_pitch: -90,
        reachy.l_arm.l_forearm_yaw: 0,
        reachy.l_arm.l_wrist_pitch: 0,
        reachy.l_arm.l_wrist_roll: 0,
    }
except AttributeError:
    pass

```

9. Riaccensione di Reachy

Riaccende Reachy.

```
reachy.turn_on('reachy')
```

10. Movimento della braccia destra alla posizione angolare specificata

Muove il braccio destro di Reachy alla posizione angolare specificata utilizzando un'interpolazione di tipo `MINIMUM_JERK`. Se il braccio destro non è presente, stampa un messaggio di errore.

```

try:
    goto(
        goal_positions=right_angled_position_right,
        duration=1.0,
        interpolation_mode=InterpolationMode.MINIMUM_JERK
    )
except NameError:
    print('Reachy has no right arm!')

```

11. Movimento della braccia sinistra alla posizione angolare specificata

Muove il braccio sinistro di Reachy alla posizione angolare specificata utilizzando un'interpolazione di tipo `MINIMUM_JERK`. Se il braccio sinistro non è presente, stampa un messaggio di errore.

```

try:
    goto(
        goal_positions=right_angled_position_left,
        duration=1.0,

```

```

        interpolation_mode=InterpolationMode.MINIMUM_JERK
    )

except NameError:
    print('Reachy has no left arm!')

```

12. Spegnimento graduale di Reachy

```
reachy.turn_off_smoothly('reachy')
```

13. Definizione di matrici di trasformazione per la cinematica inversa

Definisce quattro matrici di trasformazione per la cinematica inversa delle articolazioni del braccio.

```

A = np.array([
    [0, 0, -1, 0.3],
    [0, 1, 0, -0.4],
    [1, 0, 0, -0.3],
    [0, 0, 0, 1],
])

B = np.array([
    [0, 0, -1, 0.3],
    [0, 1, 0, -0.4],
    [1, 0, 0, 0.0],
    [0, 0, 0, 1],
])

C = np.array([
    [0, 0, -1, 0.3],
    [0, 1, 0, -0.1],
    [1, 0, 0, 0.0],
    [0, 0, 0, 1],
])

D = np.array([
    [0, 0, -1, 0.3],
    [0, 1, 0, -0.1],
    [1, 0, 0, -0.3],
    [0, 0, 0, 1],
])

```

14. Calcolo della cinematica inversa per il braccio destro

Calcola le posizioni articolari necessarie per raggiungere le pose definite dalle matrici di trasformazione A, B, C e D per il braccio destro. Se il braccio destro non è presente, stampa un messaggio di errore.

```

try:
    joint_pos_A = reachy.r_arm.inverse_kinematics(A)
    joint_pos_B = reachy.r_arm.inverse_kinematics(B)

```



```

        joint_pos_C = reachy.r_arm.inverse_kinematics(C)
        joint_pos_D = reachy.r_arm.inverse_kinematics(D)
    except AttributeError:
        print('Reachy has no right arm!')

```

15. Movimento del braccio destro tra le posi calcolate

- Importa il modulo `time` per gestire le pause.
- Accende il braccio destro e lo muove tra le posizioni calcolate, con pause di 0.5 secondi tra un movimento e l'altro.
- Spegne gradualmente il braccio destro per evitare movimenti bruschi.

```

import time
try:
    reachy.turn_on('r_arm')

    goto({joint: pos for joint,pos in
zip(reachy.r_arm.joints.values(), joint_pos_A)},
duration=1.0)
    time.sleep(0.5)
    goto({joint: pos for joint,pos in
zip(reachy.r_arm.joints.values(), joint_pos_B)},
duration=1.0)
    time.sleep(0.5)
    goto({joint: pos for joint,pos in
zip(reachy.r_arm.joints.values(), joint_pos_C)},
duration=1.0)
    time.sleep(0.5)
    goto({joint: pos for joint,pos in
zip(reachy.r_arm.joints.values(), joint_pos_D)},
duration=1.0)

    reachy.turn_off_smoothly('r_arm')

except AttributeError:
    print('Reachy has no right arm!')

```

16. Definizione di nuove matrici di trasformazione per la cinematica inversa del braccio sinistro

Definisce nuove matrici di trasformazione per la cinematica inversa delle articolazioni del braccio sinistro.

```

A = np.array([
    [0, 0, -1, 0.3],
    [0, 1, 0, 0.1],
    [1, 0, 0, -0.3],
    [0, 0, 0, 1],
])

```

```

B = np.array([
    [0, 0, -1, 0.3],
    [0, 1, 0, 0.1],
    [1, 0, 0, 0.0],
    [0, 0, 0, 1],
])

C = np.array([
    [0, 0, -1, 0.3],
    [0, 1, 0, 0.3],
    [1, 0, 0, 0.0],
    [0, 0, 0, 1],
])

D = np.array([
    [0, 0, -1, 0.3],
    [0, 1, 0, 0.3],
    [1, 0, 0, -0.3],
    [0, 0, 0, 1],
])

```

17. Calcolo della cinematica inversa per il braccio sinistro

Calcola le posizioni articolari necessarie per raggiungere le pose definite dalle matrici di trasformazione A, B, C e D per il braccio sinistro. Se il braccio sinistro non è presente, stampa un messaggio di errore.

```

try:
    joint_pos_A = reachy.l_arm.inverse_kinematics(A)
    joint_pos_B = reachy.l_arm.inverse_kinematics(B)
    joint_pos_C = reachy.l_arm.inverse_kinematics(C)
    joint_pos_D = reachy.l_arm.inverse_kinematics(D)
except AttributeError:
    print('Reachy has no left arm')

```

18. Movimento del braccio sinistro tra le posi calcolate

Accende il braccio sinistro e lo muove tra le posizioni calcolate, con pause di 0.5 secondi tra un movimento e l'altro.

Spegne gradualmente il braccio sinistro per evitare movimenti bruschi.

```

try:
    reachy.turn_on('l_arm')

    goto({joint: pos for joint, pos in
zip(reachy.l_arm.joints.values(), joint_pos_A)},
duration=1.0)
    time.sleep(0.5)
    goto({joint: pos for joint, pos in
zip(reachy.l_arm.joints.values(), joint_pos_B)},
duration=1.0)
    time.sleep(0.5)

```

```

        goto({joint: pos for joint,pos in
zip(reachy.l_arm.joints.values(), joint_pos_C)},
duration=1.0)
        time.sleep(0.5)
        goto({joint: pos for joint,pos in
zip(reachy.l_arm.joints.values(), joint_pos_D)},
duration=1.0)

reachy.turn_off_smoothly('l_arm')
except AttributeError:
    print('Reachy has no left arm!')

```

19. Controllo dei sensori di forza di Reachy

Verifica la presenza e lo stato dei sensori di forza di Reachy.

```
reachy.force_sensors
```

20. Disattivazione della modalità compliant del gripper sinistro

Disattiva la modalità compliant del gripper sinistro, rendendolo rigido per la manipolazione degli oggetti. Se il braccio sinistro non è presente, stampa un messaggio di errore.

```

try:
    reachy.l_arm.l_gripper.compliant = False
except AttributeError:
    print('Reachy has no left arm.')

```

21. Verifica del sensore di forza del gripper destro

Verifica la presenza e lo stato del sensore di forza del gripper destro.

```
reachy.force_sensors.r_force_gripper
```

22. Verifica del sensore di forza del gripper sinistro

Verifica la presenza e lo stato del sensore di forza del gripper sinistro.

```
reachy.force_sensors.l_force_gripper
```

23. Controllo del sensore di forza del gripper destro

```
reachy.force_sensors.r_force_gripper
```

24. Controllo del sensore di forza del gripper sinistro

```
reachy.force_sensors.l_force_gripper
```

25. Attivazione della modalità compliant del gripper sinistro

Riattiva la modalità compliant del gripper sinistro, permettendo movimenti più flessibili.

```
try:
    reachy.l_arm.l_gripper.compliant = True
except AttributeError:
    pass
```

26. Attivazione della modalità compliant del gripper destro

Riattiva la modalità compliant del gripper destro, permettendo movimenti più flessibili.

```
try:
    reachy.r_arm.r_gripper.compliant = True
except AttributeError:
    pass
```

Conclusione

L'analisi del codice di *'Testing the arm.ipynb'* mostra un'implementazione dettagliata e strutturata per testare il funzionamento delle braccia di Reachy. Ogni cella del notebook svolge una funzione specifica per verificare i movimenti e le capacità delle articolazioni e dei gripper, assicurandosi che tutte le componenti siano operative e rispondano correttamente ai comandi.

3.4.2. Analisi dei risultati dei test

In questo paragrafo, analizziamo i risultati ottenuti dai test eseguiti sui due script Jupyter Notebook forniti da Pollen Robotics per verificare il funzionamento del simulatore di Reachy. Gli script, denominati *'Testing the head.ipynb'* e *'Testing the arm.ipynb'*, sono stati prelevati dalla repository GitHub *'reachy-sdk'* nel percorso `reachy-sdk/reachy_sdk/examples`.

Risultati dello script ‘*Testing the head.ipynb*’

Obiettivo dello script

Lo script ‘*Testing the head.ipynb*’ è progettato per testare il funzionamento della testa di Reachy, verificando la capacità del robot di orientare la testa verso diverse direzioni e di utilizzare le antenne per esprimere emozioni.

Dettagli dei risultati

1. Orientamento della testa:

- La testa di Reachy si orienta correttamente nelle direzioni specificate, rispondendo in modo fluido ai comandi.
- Sono stati eseguiti movimenti della testa verso destra, giù, sinistra e di nuovo davanti, con successo.

2. Seguimento del braccio:

- La testa di Reachy segue correttamente i movimenti delle braccia, sia destra che sinistra, utilizzando la cinematica diretta.
- Se un braccio non è presente, lo script gestisce l'errore stampando un messaggio appropriato.

3. Movimenti delle antenne:

- Le antenne di Reachy si muovono correttamente nelle posizioni specificate.
- Le funzioni definite per esprimere emozioni di felicità e tristezza attraverso il movimento delle antenne funzionano come previsto.

4. Visualizzazione delle immagini:

- Le immagini catturate dalle telecamere sinistra e destra di Reachy vengono visualizzate correttamente, confermando il funzionamento delle telecamere.

Risultati dello script ‘*Testing the arm.ipynb*’

Obiettivo dello script

Lo script ‘*Testing the arm.ipynb*’ è progettato per testare il funzionamento delle braccia di Reachy, verificando la capacità del robot di eseguire movimenti complessi utilizzando la cinematica inversa.

Dettagli dei risultati

1. Accensione e spegnimento:

- Le operazioni di accensione e spegnimento dei motori di Reachy funzionano correttamente, inclusa la modalità di spegnimento graduale per evitare movimenti bruschi.

2. Posizioni angolari delle braccia:

- Le posizioni angolari delle articolazioni delle braccia destra e sinistra sono state impostate correttamente.
- Le braccia si muovono alle posizioni specificate utilizzando un'interpolazione di tipo `MINIMUM_JERK`.

3. Cinematica inversa:

- Le posizioni articolari necessarie per raggiungere le pose definite dalle matrici di trasformazione sono calcolate correttamente.
- I movimenti delle braccia tra le posizioni calcolate sono stati eseguiti con successo.

4. Sensori di forza:

- La presenza e lo stato dei sensori di forza di Reachy sono verificati correttamente.
- I sensori di forza del gripper destro e sinistro funzionano come previsto.

5. Modalità compliant dei gripper:

La modalità compliant dei gripper è stata attivata e disattivata correttamente, permettendo movimenti flessibili e rigidi per la manipolazione degli oggetti.

Conclusione

L'analisi dei risultati dei test ha confermato che gli script '*Testing the head.ipynb*' e '*Testing the arm.ipynb*' funzionano correttamente, permettendo di controllare i movimenti della testa e delle braccia di Reachy, rispettivamente. Le operazioni di accensione e spegnimento, la cinematica inversa, i movimenti delle antenne e la verifica dei sensori di forza sono state eseguite con successo, dimostrando l'operatività delle componenti del robot e la loro risposta corretta ai comandi.

4. SVILUPPO DI SCRIPT PYTHON PER IL ROBOT ANTROPOMORFO REACHY

Questo capitolo si concentra sullo sviluppo di script Python per il controllo avanzato del robot antropomorfo Reachy, illustrando le metodologie e le tecniche utilizzate per implementare movimenti complessi e funzionalità di riconoscimento di oggetti. L'obiettivo è quello di sfruttare al massimo le potenzialità offerte dall'SDK di Reachy e dalle librerie Python.

In primo luogo, viene trattata la pianificazione dei movimenti del robot, descrivendo come la programmazione degli script consenta di ottenere movimenti fluidi e coordinati, adattabili a vari scenari applicativi. Si analizzano le tecniche di controllo delle articolazioni e degli attuatori, illustrando come sia possibile modulare le azioni di Reachy per eseguire compiti specifici, quali l'afferramento e la manipolazione di oggetti. Particolare attenzione è dedicata all'ottimizzazione dei parametri di movimento per garantire un'elevata precisione e reattività del robot, riducendo al minimo errori e tempi di latenza.

Successivamente, viene approfondito lo sviluppo di script per il riconoscimento di oggetti, utilizzando algoritmi di visione artificiale e tecniche di elaborazione delle immagini. Si esplorano le modalità con cui il robot è in grado di identificare oggetti in scenari complessi e dinamici, integrando le informazioni visive con i dati provenienti dai sensori per migliorare la capacità di interazione con l'ambiente. Questo approccio consente a Reachy di riconoscere e manipolare con accuratezza una vasta gamma di oggetti, potenziando le sue capacità operative in contesti applicativi quali la robotica assistiva, la logistica e la ricerca scientifica.

4.1. SVILUPPO DI SCRIPT PYTHON PER L'ESECUZIONE DEI MOVIMENTI

Il movimento rappresenta una delle funzionalità più affascinanti e complesse per un robot antropomorfo come Reachy. Sviluppare script Python per controllare i movimenti di Reachy richiede una combinazione di conoscenze di cinetica, dinamica e programmazione avanzata. Questo processo non solo consente al robot di eseguire movimenti fluidi e naturali, ma anche di interagire efficacemente con l'ambiente circostante.

4.1.1. Pianificazione dei movimenti

La pianificazione dei movimenti per un robot antropomorfo come Reachy è un aspetto cruciale che richiede una comprensione approfondita della cinetica e della dinamica applicate alla robotica. Questo processo è fondamentale per assicurare che i movimenti del robot siano fluidi, naturali e sicuri. Di seguito viene descritta l'importanza della pianificazione dei movimenti, i principi teorici su cui si basa, gli obiettivi da raggiungere e le principali sfide da affrontare.

La pianificazione dei movimenti è vitale per garantire che Reachy possa eseguire compiti complessi in modo efficiente e preciso. Una pianificazione accurata consente di:

- **Ottimizzare le traiettorie:** Minimizzare il consumo di energia e ridurre l'usura dei componenti meccanici.
- **Evitare collisioni:** Prevenire danni al robot e all'ambiente circostante.
- **Garantire la sicurezza:** Assicurare che i movimenti del robot siano sicuri per le persone che interagiscono con esso.
- **Migliorare l'interazione umana:** Rendere i movimenti del robot più intuitivi e naturali per facilitare l'interazione con gli esseri umani.

La pianificazione dei movimenti si basa su principi di cinetica e dinamica, che sono utilizzati per modellare e controllare il movimento del robot. I principali aspetti teorici includono:

- **Cinetica:** Studia il movimento dei corpi senza considerare le forze che lo causano. Nel contesto di Reachy, si tratta di determinare le posizioni, le velocità e le accelerazioni delle sue articolazioni.
- **Dinamica:** Esamina le forze e i momenti che causano il movimento. Questo implica la modellazione delle forze generate dai motori, le resistenze passive e le interazioni con l'ambiente.

L'uso combinato di questi principi permette di sviluppare modelli matematici che descrivono il comportamento del robot e di progettare algoritmi di controllo avanzati.

Gli obiettivi principali della pianificazione dei movimenti per Reachy includono:

- **Precisione:** Garantire che Reachy possa raggiungere e manipolare oggetti con elevata precisione.
- **Fluidità:** Assicurare che i movimenti siano fluidi e senza scatti, migliorando l'imitazione dei movimenti umani.
- **Efficienza energetica:** Minimizzare il consumo energetico per prolungare la durata operativa del robot.
- **Robustezza:** Creare movimenti che possano adattarsi a variazioni impreviste nell'ambiente o a incertezze nei modelli dinamici.

La pianificazione dei movimenti presenta diverse sfide tecniche:

- **Complessità computazionale:** I calcoli richiesti per pianificare i movimenti in tempo reale possono essere molto intensivi dal punto di vista computazionale.
- **Incertezze ambientali:** Variazioni impreviste nell'ambiente, come ostacoli mobili, richiedono algoritmi di pianificazione che possano adattarsi dinamicamente.
- **Limitazioni hardware:** La precisione e la velocità dei movimenti sono limitate dalle capacità dei motori e dei sensori di Reachy.
- **Interazione sicura:** Garantire che i movimenti siano sicuri per le persone che interagiscono con Reachy, evitando collisioni e movimenti bruschi.

4.1.2. Implementazione degli script di movimento

Nella seguente sezione, verrà descritta l'implementazione degli script Python utilizzati per programmare i movimenti di Reachy all'interno del simulatore Unity. Lo scenario simulato è stato modificato, posizionando tre palloni da basket in posizioni diverse sul tavolo, con l'obiettivo di creare script che permettano a Reachy di afferrare ogni pallone (grasping) e compiere un'azione di movimento specifica.

La scena Unity di partenza, denominata '*Reachy in OfficeScene*' è stata modificata per aggiungere tre palloni da basket posizionati in punti diversi sul tavolo. La posizione precisa dei palloni è stata determinata e visualizzata utilizzando l'Inspector di Unity. Tuttavia, è fondamentale notare che le coordinate ottenute nell'Inspector di Unity sono riferite al sistema di coordinate globale della scena (il cui centro è posto a $[0,0,0]$). Per programmare correttamente i movimenti di Reachy, è necessario convertire queste coordinate nel sistema di riferimento locale di Reachy.

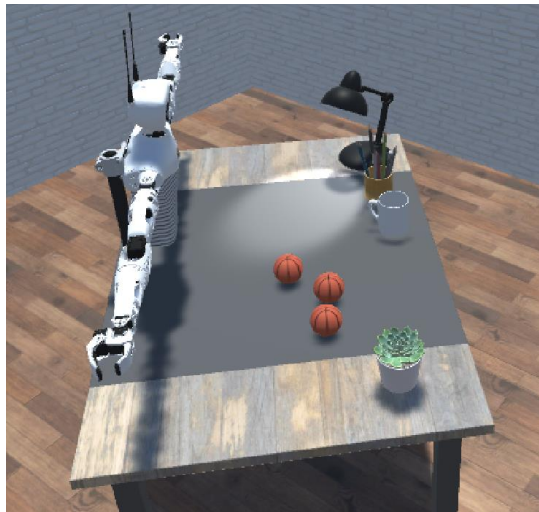


Figura 4.1. Scena Unity '*Reachy in OfficeScene*' modificata.

Per effettuare questa conversione, è stato utilizzato un codice C# fornito da Pollen Robotics, denominato '*ObjInReachyFrame.cs*'. Questo script si trova nel percorso `Packages/com.pollenrobotics.reachysimulator/Scripts/ObjInReachyFrame.cs` all'interno del progetto Unity.

Lo script C# converte le coordinate dell'oggetto nel sistema di riferimento locale di Reachy, tenendo conto della rotazione e della scala del modello di Reachy rispetto al sistema globale. Le coordinate risultanti sono poi utilizzate negli script Python per eseguire i movimenti precisi necessari per il grasping.

Ecco il codice C# con commenti esplicativi:

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class ObjInReachyFrame : MonoBehaviour
{
    public Transform obj; // Variabile pubblica che contiene il
    riferimento all'oggetto di cui calcolare la posizione

    // Metodo chiamato all'inizio del ciclo di vita dello script
    void Start()
    {
        objInReachyFrame(obj); // Chiama il metodo per calcolare
        la posizione dell'oggetto nel frame di riferimento di Reachy
    }

    // Metodo che viene chiamato una volta per frame
    void Update()
    {
        // Potenzialmente, possiamo aggiungere codice qui se
        vogliamo aggiornare la posizione ad ogni frame
    }

    // Metodo per calcolare la posizione dell'oggetto rispetto al
    frame di riferimento di Reachy
    void objInReachyFrame(Transform obj)
    {
        // Trasforma la posizione dell'oggetto dalle coordinate
        globali alle coordinate locali di Reachy
        Vector3 localPos =
        transform.InverseTransformPoint(obj.position);

        // Dato che il prefab di Reachy è ruotato di 180° attorno
        all'asse Y, applichiamo una rotazione aggiuntiva
        Quaternion rotation = Quaternion.Euler(0, 180, 0);
        Matrix4x4 m = Matrix4x4.Rotate(rotation);

        // Moltiplica il punto per la matrice di rotazione per
        ottenere la posizione corretta nel frame di Reachy
        Vector3 posrot = m.MultiplyPoint3x4(localPos);

        // Invertiamo gli assi Y e Z per adattarli al sistema di
        coordinate di Reachy
        Debug.Log("Object in Reachy frame: "
            + "-posrot.y * transform.localScale.x + ", "
            + posrot.x * transform.localScale.y + ", "
            + "-posrot.z * transform.localScale.z);
    }
}
```

Ecco una descrizione delle principali operazioni eseguite dallo script C#:

- **Trasformazione delle coordinate:** Lo script utilizza la funzione `'transform.InverseTransformPoint(obj.position)'` per ottenere la posizione dell'oggetto nel sistema di coordinate di Reachy.
- **Rotazione:** Dato che il prefab di Reachy è ruotato di 180° attorno all'asse Y, lo script applica una rotazione aggiuntiva utilizzando una matrice di rotazione.
- **Scalatura:** Infine, lo script regola le coordinate per tener conto della scala locale di Reachy, garantendo che le coordinate siano corrette nel sistema di riferimento locale.

Una volta ottenute le coordinate corrette dei palloni da basket rispetto a Reachy, si passa all'implementazione degli script Python per il grasping.

Per ognuna delle tre palle, è stato sviluppato uno script Python simile, basato sul seguente approccio generale:

```
from reachy_sdk import ReachySDK
from reachy_sdk.trajectory import goto
from reachy_sdk.trajectory import InterpolationMode
import time
import cv2 as cv
import numpy as np

from threading import Thread

# Variabile globale per tenere traccia se il thread è in
# esecuzione
thread_running = False

# Funzione per far guardare il robot verso un obiettivo
def look_at(target):
    reachy.head.look_at(
        target[0], target[1], target[2], 1.0)

# Funzione per muovere il braccio del robot verso una posizione
# target con una durata specificata
def move_arm(target, duration):

    # Creazione della matrice target per la cinematica inversa
    target_matrix = np.array([
        [0, 0, -1, target[0]],
        [0, 1, 0, target[1]],
        [1, 0, 0, target[2]],
        [0, 0, 0, 1],
    ])
    1)
```

```

    # Calcolo delle posizioni delle giunture usando la cinematica
    inversa
    joint_pos_ball =
reachy.r_arm.inverse_kinematics(target_matrix)

    # Accensione del braccio destro
reachy.turn_on('r_arm')

    # Movimento del braccio verso le posizioni delle giunture
    calcolate
    goto({joint: pos for joint, pos in zip(
        reachy.r_arm.joints.values(), joint_pos_ball)},
        duration=duration)

# Funzione per eseguire il movimento di lancio
def throw_movement():

    # Apertura della pinza
reachy.r_arm.r_gripper.goal_position = -60

    # Posizione angolare del braccio destro per il lancio
    right_angled_position = {
        reachy.r_arm.r_shoulder_pitch: -90,
        reachy.r_arm.r_shoulder_roll: 0,
        reachy.r_arm.r_arm_yaw: 0,
        reachy.r_arm.r_elbow_pitch: 0,
        reachy.r_arm.r_forearm_yaw: 0,
        reachy.r_arm.r_wrist_pitch: 0,
        reachy.r_arm.r_wrist_roll: 10,
    }

    # Movimento del braccio verso la posizione di lancio
    goto(
        goal_positions=right_angled_position,
        duration=0.2,
        interpolation_mode=InterpolationMode.MINIMUM_JERK
    )

# Funzione principale per il movimento di basket
def basketball():

    # Position Basket Ball
    # ball_position = np.array([xBC, yBC, zBC]) # In riferimento al
    centro della scena (0,0,0)
    ball_position = np.array([xBR, yBR, zBR]) # In riferimento a
    Reachy

    # Far guardare il robot verso la palla
    look_at(ball_position)

    # Muovere il braccio allungabile sopra la palla evitando la
    palla
    avoid_ball = np.array([ball_position[0]-0.05,
        ball_position[1]-0.10, 0])
    move_arm(avoid_ball, 1.0)

    # Posizionare il braccio sopra la palla
    above_ball = np.array([ball_position[0]-0.05,
        ball_position[1], 0])

```

```

move_arm(above_ball, 1.0)

# Apertura della pinza
reachy.r_arm.r_gripper.goal_position = -50
time.sleep(1.0)

# Raggiungere la palla
move_arm(ball_position, 4.0)
time.sleep(2.0)

# Chiusura della pinza
reachy.r_arm.r_gripper.goal_position = -40
time.sleep(0.5)

# Alzare il braccio e lanciare la palla
move_arm(above_ball, 4.0)

# Eseguire il movimento di lancio
throw_movement()
time.sleep(1)

# Aggiornare lo stato del thread
global thread_running
thread_running = False

# Codice eseguibile principale
if __name__ == "__main__":

    # Connessione al robot Reachy
    reachy = ReachySDK(host='localhost')

    # Creare e avviare un thread per eseguire la funzione
    basketball
    t = Thread(target=lambda: basketball())
    t.daemon = True
    t.start()

    # Impostare lo stato del thread come in esecuzione
    thread_running = True

    # Loop per mostrare le immagini della telecamera destra del
    robot
    while (thread_running):
        cv.imshow('Right camera', reachy.right_camera.last_frame)
        cv.waitKey(30)

    # Chiudere tutte le finestre di OpenCV
    cv.destroyAllWindows()

    # Spegner il braccio destro del robot in modo graduale
    reachy.turn_off_smoothly('r_arm')

    # Uscire dal programma
    exit()

```

Ecco una descrizione delle principali operazioni eseguite dallo script Python:

- **Connessione a Reachy:** Ogni script inizia con la connessione al robot Reachy tramite `ReachySDK`, specificando l'host come `localhost`, dato che stiamo lavorando in un ambiente simulato.
- **Posizionamento della testa:** La testa di Reachy viene orientata verso il pallone utilizzando la funzione `look_at`, che riceve come input le coordinate corrette della palla.
- **Movimento del braccio:** Il braccio destro di Reachy viene mosso in diverse fasi:
- **Evitamento dell'ostacolo:** Prima di tutto, il braccio si muove in una posizione "sopra" la palla, evitando di colpire l'oggetto durante il movimento.
- **Posizionamento sopra la palla:** Il braccio si posiziona direttamente sopra la palla, pronto per afferrarla.
- **Afferraggio:** Il gripper del braccio viene chiuso per afferrare la palla.
- **Movimento di lancio:** Infine, il braccio si solleva e si prepara a lanciare la palla, eseguendo il movimento di lancio.
- **Esecuzione del codice:** Gli script vengono eseguiti in un loop gestito da un thread, che mostra contemporaneamente le immagini catturate dalla telecamera destra di Reachy utilizzando OpenCV.

L'implementazione di questi script dimostra come, partendo da una scena Unity modificata e utilizzando una combinazione di codici C# e Python, sia possibile programmare movimenti complessi per un robot antropomorfo come Reachy. La corretta conversione delle coordinate degli oggetti è fondamentale per garantire movimenti precisi e affidabili, mentre l'integrazione tra Unity e Python offre un potente strumento per lo sviluppo e la simulazione di azioni robotiche complesse.

4.1.3. Test dei movimenti

Nella seguente sezione, viene analizzato il processo di test degli script di movimento implementati per Reachy, con particolare riferimento all'operazione di

grasping delle tre palline da basket posizionate sulla scrivania nella scena ‘*Reachy in OfficeScene*’ nel simulatore Unity.

Obiettivo del test

L'obiettivo del test è stato verificare l'efficacia e la precisione degli script Python sviluppati per l'operazione di grasping, ossia l'afferraggio delle tre palline da basket. Ogni script è stato progettato per guidare Reachy in modo che possa individuare, avvicinarsi, afferrare e, infine, eseguire un movimento di rilascio della palla. La riuscita di questo test rappresenta un indicatore chiave delle capacità di Reachy di eseguire compiti di manipolazione in un ambiente simulato.

Metodologia di test

Il processo di test ha seguito i seguenti passaggi:

1. Impostazione iniziale:

- Prima di eseguire gli script, la scena Unity è stata preparata e caricata correttamente, assicurandosi che le tre palline fossero posizionate nei punti desiderati.
- È stata verificata la connessione tra il simulatore Unity e gli script Python tramite `ReachySDK`.

2. Esecuzione degli script:

- Ogni script è stato eseguito individualmente, guidando Reachy verso ciascuna delle tre palline.
- I movimenti di Reachy sono stati monitorati in tempo reale utilizzando le telecamere integrate nel simulatore.

3. Verifica dell'operazione di grasping:

- La riuscita dell'operazione di grasping è stata valutata osservando se Reachy riuscisse a individuare correttamente la palla, avvicinarsi ad essa senza collisioni, chiudere il gripper sulla palla e sollevarla dal tavolo.
- Per ogni palla, è stato eseguito un movimento di rilascio controllato, per verificare la stabilità del braccio durante l'operazione.

4. Ripetibilità:

Per assicurare la ripetibilità, ogni script è stato eseguito più volte, in condizioni identiche, per confermare la consistenza dei risultati.

Risultati del test

L'operazione di grasping delle tre palline da basket è avvenuta con successo in tutti i test eseguiti. Di seguito, una sintesi dei risultati:

- **Precisione del movimento:**

Reachy ha dimostrato un'alta precisione nel localizzare e afferrare ciascuna delle tre palline. La conversione delle coordinate da Unity al sistema di riferimento di Reachy ha funzionato correttamente, consentendo a Reachy di posizionarsi esattamente sopra ogni pallina prima di eseguire il grasping.

- **Affidabilità del gripper:**

Il gripper di Reachy si è comportato in modo affidabile, chiudendosi correttamente su ciascuna palla e mantenendola saldamente durante il movimento di sollevamento. Non sono stati osservati problemi di presa, come la perdita della palla o la chiusura incompleta del gripper.

- **Fluidità dei movimenti:**

I movimenti del braccio sono stati eseguiti in modo fluido e naturale, senza scatti o movimenti bruschi. L'interpolazione dei movimenti, impostata su `MINIMUM_JERK`, ha contribuito a garantire questa fluidità, migliorando la sicurezza del robot e la protezione degli oggetti manipolati.

- **Robustezza:**

Gli script hanno mostrato robustezza e ripetibilità, eseguendo con successo l'operazione di grasping in tutte le iterazioni del test. Non sono stati riscontrati errori o comportamenti anomali, e il sistema ha risposto correttamente a tutti i comandi impartiti.

- **Valutazione complessiva:**

I risultati indicano che Reachy è in grado di eseguire con successo operazioni di manipolazione complesse, come il grasping di oggetti posizionati in punti specifici, con un alto grado di precisione e affidabilità.

Questi risultati confermano l'efficacia degli script sviluppati e la corretta integrazione tra il simulatore Unity e il sistema di controllo di Reachy.



Figura 4.2. Grasping della basketball da parte di Reachy

Conclusione

Il processo di test degli script di movimento ha confermato che l'operazione di grasping delle tre palline da basket è avvenuta con successo, dimostrando le capacità avanzate di Reachy di manipolare oggetti in un ambiente simulato. La precisione dei movimenti, l'affidabilità del gripper e la fluidità delle operazioni sono indicatori positivi della qualità degli script Python implementati, e suggeriscono che Reachy è pronto per affrontare scenari più complessi di manipolazione e interazione con l'ambiente circostante.

4.2. SVILUPPO DI SCRIPT PYTHON PER IL RICONOSCIMENTO DI OGGETTI

Il riconoscimento di oggetti è una delle capacità più cruciali per i robot antropomorfi, permettendo loro di interagire in modo intelligente e autonomo con

l'ambiente circostante. Per un robot come Reachy, essere in grado di identificare e reagire agli oggetti è fondamentale per compiti che vanno dalla manipolazione di oggetti alla navigazione in ambienti complessi. Lo sviluppo di script Python per il riconoscimento di oggetti sfrutta tecniche avanzate di visione artificiale e apprendimento automatico.

4.2.1. Introduzione al riconoscimento di oggetti

Il riconoscimento di oggetti rappresenta una delle competenze fondamentali per i robot antropomorfi come Reachy. Questa capacità permette al robot di interagire efficacemente con l'ambiente circostante, identificando e manipolando oggetti con precisione. L'importanza del riconoscimento di oggetti risiede nella sua applicazione in vari contesti, dall'automazione industriale all'assistenza sanitaria, dove l'accuratezza e l'affidabilità sono essenziali.

Le tecniche di visione artificiale costituiscono la base del riconoscimento di oggetti. Queste tecniche utilizzano sensori visivi, come telecamere stereoscopiche, per acquisire immagini dell'ambiente. Le immagini raccolte vengono poi elaborate attraverso algoritmi di intelligenza artificiale per identificare e classificare gli oggetti. Tra le tecniche più comuni troviamo:

- **Rilevamento degli oggetti:** Utilizza algoritmi come YOLO (You Only Look Once) e SSD (Single Shot MultiBox Detector) per localizzare gli oggetti all'interno di un'immagine.
- **Riconoscimento delle immagini:** Utilizza reti neurali convoluzionali (CNN) per classificare gli oggetti presenti nelle immagini. Le CNN sono addestrate su grandi dataset per riconoscere una vasta gamma di oggetti.
- **Segmentazione delle immagini:** Tecnica che suddivide un'immagine in segmenti, identificando le regioni corrispondenti a diversi oggetti.

L'apprendimento automatico gioca un ruolo cruciale nel migliorare le capacità di riconoscimento di oggetti. I modelli di apprendimento automatico, come le deep neural networks, sono addestrati su enormi quantità di dati per riconoscere schemi complessi. In particolare, i large language model (LLMs) come OWL-ViT (Object-World Language Vision Transformer) hanno dimostrato eccellenti capacità nel

riconoscimento di oggetti attraverso la combinazione di visione artificiale e comprensione del linguaggio naturale.

- **OWL-ViT:** utilizza una combinazione di transformer neural networks per analizzare sia l'immagine che il contesto linguistico, migliorando l'accuratezza del riconoscimento. Questo modello è particolarmente efficace nel gestire situazioni complesse dove gli oggetti devono essere identificati non solo visivamente ma anche in relazione al contesto descritto.

Per integrare queste tecniche di riconoscimento di oggetti con Reachy, vengono utilizzate librerie specifiche come Pollen-Vision. Queste librerie facilitano l'implementazione di algoritmi di visione artificiale sui sensori di Reachy, consentendo al robot di riconoscere e interagire con gli oggetti in tempo reale.

- **Pollen-Vision:** Una libreria Python sviluppata per lavorare con i sensori di Reachy, ottimizzando le operazioni di riconoscimento degli oggetti. La libreria supporta l'integrazione con modelli avanzati come OWL-ViT, permettendo una facile implementazione degli script di riconoscimento.

4.2.2. Implementazione degli script di riconoscimento

Il codice fornito in un notebook Jupyter implementa una pipeline di visione artificiale per il riconoscimento e la segmentazione di oggetti specifici, come una tazza, utilizzando il robot Reachy nella scena di Unity denominata Office Scene. Di seguito viene fornita un'analisi dettagliata del codice, suddivisa per celle, spiegando le funzioni e le librerie utilizzate.

- **Prima cella: importazione delle librerie e classi necessarie**

In questa cella, vengono importate diverse librerie e classi necessarie per la connessione con Reachy, l'elaborazione delle immagini e l'implementazione dei modelli di rilevamento e segmentazione degli oggetti. Le librerie includono `numpy` per l'elaborazione numerica, `'time'` per la gestione delle tempistiche, `'PIL'` per l'elaborazione delle immagini e `pollen_vision` per i modelli di visione artificiale.

```
# Si importa la classe ReachySDK dal modulo reachy_sdk
from reachy_sdk import ReachySDK
```

```

# Si importa la libreria numpy
import numpy as np

# Si importa la libreria time
import time

# Si importa la funzione goto dal modulo reachy_sdk
from reachy_sdk.trajectory import goto

# Si importa la classe InterpolationMode dal modulo
reachy_sdk
from reachy_sdk.trajectory.interpolation import
InterpolationMode

# Si importa il modulo Image dal pacchetto PIL
from PIL import Image

# Si importa la classe OwlVitWrapper dal modulo
object_detection all'interno del pacchetto vision_models
della libreria pollen_vision
from pollen_vision.vision_models.object_detection import
OwlVitWrapper

# Si importa le classe get_bboxes dal modulo utils
all'interno del pacchetto vision_models della libreria
pollen_vision
from pollen_vision.vision_models.utils import get_bboxes

# Si importa le classi Annotator dal modulo utils
all'interno del pacchetto vision_models della libreria
pollen_vision
from pollen_vision.vision_models.utils import Annotator

# Si importa la classe MobileSamWrapper dal modulo
object_segmentation all'interno del pacchetto vision_models
della libreria pollen_vision
from pollen_vision.vision_models.object_segmentation import
MobileSamWrapper

```

- **Seconda cella: connessione con Reachy**

In questa cella, si stabilisce una connessione con Reachy utilizzando la classe `ReachySDK`. Il parametro `host='localhost'` indica che la connessione viene effettuata localmente.

```

# Si stabilisce una connessione con Reachy
reachy = ReachySDK(host='localhost')

```

- **Terza cella: verifica delle articolazioni**

Qui, il codice verifica se tutte le articolazioni di Reachy sono rilevate correttamente. Questo passaggio è cruciale per assicurarsi che il robot possa eseguire i movimenti necessari senza problemi.

```
# Si controllano se tutte le articolazioni di Reachy sono
rilevate
reachy.joints
```

- **Quarta cella: verifica dei sensori di forza**

In questa cella, si verifica la rilevazione di tutti i sensori di forza di Reachy. I sensori di forza sono importanti per garantire che il robot possa interagire in modo sicuro con gli oggetti nell'ambiente.

```
# Si controlla se tutti i sensori di forza di Reachy sono
rilevati
reachy.force_sensors
```

- **Quinta cella: movimento della testa di Reachy**

Questa cella comanda la testa di Reachy di abbassarsi e guardare verso la tazza posizionata sul tavolo. La funzione `look_at` utilizza coordinate spaziali per orientare la testa e include una durata specifica per il movimento.

```
# Si fa abbassare la testa di Reachy per fare in modo che
rivolga lo sguardo davanti la tazza riposta nel tavolo
look_down = reachy.head.look_at(
    x=0.5,
    y=0,
    z=-0.4,
    duration=1.0,
    starting_positions={
        reachy.head.neck_roll:
reachy.head.neck_roll.goal_position,
        reachy.head.neck_pitch:
reachy.head.neck_pitch.goal_position,
        reachy.head.neck_yaw:
reachy.head.neck_yaw.goal_position
    })
```

- **Sesta cella: acquisizione dell'immagine**

In questa cella, viene acquisita un'immagine dall'occhio destro di Reachy. La funzione `Image.fromarray` converte l'array `numpy` dell'immagine in un oggetto immagine utilizzabile per ulteriori elaborazioni.

```
# Si assegna alla variabile "img" l'ultimo frame dall'occhio
destro di Reachy
```

```
img = Image.fromarray(reachy.right_camera.last_frame[:,:,:-1])
img
```

- **Settima cella: inizializzazione del modello di rilevazione degli oggetti**

Qui viene creata un'istanza del modello di rilevazione degli oggetti OwlVitWrapper, che verrà utilizzata per identificare gli oggetti nell'immagine acquisita.

```
# Si crea un'istanza della classe OwlVitWrapper e si assegna
alla variabile "object_detection_wrapper"
object_detection_wrapper = OwlVitWrapper()
```

- **Ottava cella: rilevazione degli oggetti**

In questa cella, il modello `object_detection_wrapper` viene utilizzato per rilevare oggetti nell'immagine. Si cercano specificamente "mug", utilizzando una soglia di rilevazione di 0.06.

```
'''
Si utilizza il modello object_detection_wrapper per eseguire
la rilevazione degli oggetti sull'immagine im,
cercando "cups" utilizzando la zero-shot object detection.
I risultati della rilevazione vengono assegnati alla
variabile "predictions".
'''
predictions = object_detection_wrapper.infer(
    im=np.array(img), candidate_labels=["mug"],
    detection_threshold=0.06
)
Predictions
```

- **Nona cella: ottenimento delle bounding boxes**

Questa cella utilizza la funzione `get_bboxes` per estrarre le bounding boxes dai risultati della rilevazione degli oggetti. Le bounding boxes vengono utilizzate per delimitare visivamente gli oggetti rilevati nell'immagine.

```
'''
Si utilizza il modello get_bboxes per ottenere le caselle
delimitatrici (bounding boxes)
dai risultati della rilevazione degli oggetti contenuti nella
variabile "predictions".

Le bounding boxes vengono assegnate alla variabile "bboxes".
'''
bboxes = get_bboxes(predictions)
```


- **Decima cella: inizializzazione dell'annotatore**

In questa cella, viene creata un'istanza della classe `Annotator`, che sarà utilizzata per aggiungere annotazioni visive all'immagine.

```
# Si crea un'istanza della classe Annotator e si assegna
alla variabile "annotator"
annotator = Annotator()
```

- **Undicesima cella: annotazione dell'immagine**

Questa cella utilizza l'oggetto `annotator` per aggiungere annotazioni all'immagine originale, evidenziando gli oggetti rilevati. L'immagine annotata viene visualizzata per verificare i risultati della rilevazione.

```
'''
Si utilizza l'oggetto "annotator" per aggiungere annotazioni
all'immagine "im",
utilizzando i risultati della rilevazione degli oggetti
contenuti nella variabile "prediction".
L'immagine risultante viene assegnata alla variabile
"img_annotated"
'''
img_annotated = annotator.annotate(im=img,
detection_predictions=predictions)
Image.fromarray(img_annotated)
```

- **Dodicesima cella: inizializzazione del modello di segmentazione degli oggetti**

In questa cella, viene creata un'istanza del modello di segmentazione degli oggetti `MobileSamWrapper`, che verrà utilizzata per segmentare gli oggetti rilevati nell'immagine.

```
# Si crea un'istanza della classe MobileSamWrapper e si
assegna alla variabile "object_segmentation_wrapper"
object_segmentation_wrapper = MobileSamWrapper()
```

- **Tredicesima cella: segmentazione degli oggetti**

Questa cella utilizza il modello `MobileSamWrapper` per segmentare gli oggetti rilevati nell'immagine. Le maschere di segmentazione risultanti vengono assegnate alla variabile `masks`.

```
'''
Si utilizza il modello sam per eseguire la segmentazione
degli oggetti sull'immagine im,
utilizzando le bounding boxes ottenute dalla rilevazione
degli oggetti
utilizzando la zero-shot object segmentation.
I risultati della segmentazione vengono assegnati alla
variabile "masks".
'''
masks = object_segmentation_wrapper.infer(im=img,
bboxes=bboxes)
```

- **Quattordicesima cella: annotazione finale con segmentazione**

In questa cella finale, l'oggetto `annotator` viene utilizzato per aggiungere annotazioni all'immagine originale, includendo sia i risultati della rilevazione degli oggetti che le maschere di segmentazione. L'immagine annotata risultante viene visualizzata per verificare l'accuratezza della segmentazione.

```
'''
utilizzando i risultati della rilevazione degli oggetti
contenuti nella variabile "prediction"
e le maschere di segmentazione contenute nella variabile
"masks".
L'immagine risultante viene assegnata alla variabile
"img_annotated"
'''

img_annotated = annotator.annotate(im=img,
detection_predictions=predictions, masks=masks)
Image.fromarray(img_annotated)
```

4.2.3. Test del riconoscimento di oggetti

Il processo di test degli script di riconoscimento di oggetti è essenziale per garantire che i modelli utilizzati siano accurati e affidabili. Questo paragrafo descrive i metodi utilizzati per valutare le prestazioni del riconoscimento di oggetti e gli strumenti utilizzati per monitorare e migliorare le capacità di riconoscimento di Reachy.

Il processo di test coinvolge l'esecuzione di diversi script in condizioni controllate per osservare il comportamento del modello di riconoscimento degli oggetti. Il test si svolge in più fasi, ognuna delle quali mira a valutare diversi aspetti delle prestazioni del modello.

1. Acquisizione dell'immagine:

- La sesta cella del notebook acquisisce un'immagine dalla telecamera destra di Reachy. L'immagine mostra una scena dell'ufficio con vari oggetti sul tavolo, inclusa una tazza.
- **Risultato:** La prima foto allegata mostra l'immagine acquisita, evidenziando la disposizione degli oggetti.



Figura 4.3. Ultimo frame dell'occhio destro di Reachy

2. Rilevazione degli oggetti:

- L'ottava cella utilizza il modello OwlVitWrapper per rilevare oggetti nell'immagine. La rilevazione è configurata per identificare specificamente una "mug" (tazza).
- **Risultato:** La rilevazione degli oggetti produce una previsione con un punteggio di 0.112 per l'oggetto "mug", indicando una probabile

identificazione della tazza nella scena. Le coordinate della bounding box sono specificate per delimitare visivamente l'oggetto rilevato.

```
[{'score': 0.11224572360515594,  
  
  'label': 'mug',  
  
  'box': {'xmin': 314, 'ymin': 704, 'xmax': 452,  
          'ymax': 855}}]
```

3. Annotazione dell'immagine:

- L'undicesima cella aggiunge annotazioni visive all'immagine originale utilizzando le bounding box ottenute dalla rilevazione degli oggetti. Questa fase aiuta a visualizzare chiaramente quali oggetti sono stati rilevati e dove sono posizionati nell'immagine.
- **Risultato:** La seconda foto allegata mostra l'immagine annotata con la tazza correttamente identificata e delimitata.



Figura 4.4. Risultato dell'object detection

4. Segmentazione degli oggetti:

- La quattordicesima cella esegue la segmentazione degli oggetti utilizzando il modello `MobileSamWrapper`, creando maschere che identificano i contorni precisi degli oggetti rilevati.
- **Risultato:** La terza foto allegata mostra l'immagine finale con le annotazioni e le maschere di segmentazione, evidenziando chiaramente la tazza nel contesto della scena.



Figura 4.5. Risultato dell'object segmentation

Il processo di test del riconoscimento di oggetti per Reachy è strutturato e rigoroso, utilizzando strumenti avanzati di monitoraggio. Attraverso un'attenta valutazione e iterazione, le capacità di riconoscimento di Reachy possono essere ottimizzate per garantire prestazioni elevate in vari scenari operativi, contribuendo a una maggiore efficacia e sicurezza nelle interazioni robotiche. Tuttavia, durante i test, è stato osservato che il modello OWL-ViT riesce a riconoscere soltanto le tazze presenti nella scena del simulatore, limitando l'accuratezza del sistema nella distinzione di oggetti diversi.

5. ADDESTRAMENTO DI ChatGPT-4o E VALUTAZIONE DEGLI SCRIPT GENERATI PER REACHY

Questo capitolo è dedicato all'approfondimento delle metodologie e delle tecniche impiegate per l'addestramento di ChatGPT-4 al fine di generare script Python specifici per il robot antropomorfo Reachy. L'obiettivo principale è dimostrare come l'intelligenza artificiale possa essere utilizzata per automatizzare la creazione di codice efficiente e funzionale, capace di far compiere a Reachy una varietà di task, da quelli più semplici a quelli più complessi.

Il processo di addestramento è suddiviso in due fasi distinte: il test pilota e i test sistematici. La fase del test pilota prevede l'esecuzione di un task complesso, selezionato per valutare la capacità iniziale del modello di comprendere e risolvere un problema articolato. Questa fase funge da base di riferimento per calibrare il modello e identificare eventuali aree di miglioramento.

Successivamente, durante i test sistematici, il livello di complessità dei task viene gradualmente incrementato. Si parte con task semplici, come movimenti di base o interazioni elementari con oggetti, per poi procedere con task sempre più articolati, includendo movimenti complessi e l'integrazione di input visivi provenienti dal simulatore. Questa progressione graduale consente di valutare il miglioramento delle prestazioni del modello e la sua capacità di adattarsi a compiti di diversa natura e difficoltà.

La valutazione degli script generati si basa su una serie di metriche predefinite che permettono di misurare la qualità degli script prodotti e di identificare le aree in cui il modello può essere ulteriormente ottimizzato.

5.1. ADDESTRAMENTO DI ChatGPT-4o CON SCRIPT PYTHON

L'addestramento di ChatGPT-4o con script Python è una fase cruciale per permettere al modello di comprendere e generare codice funzionale per il robot Reachy. L'obiettivo di questa sezione è fornire al modello una serie di esempi ben strutturati e rappresentativi delle operazioni che Reachy deve compiere, utilizzando un approccio graduale e incrementale. Il processo di addestramento avviene tramite prompt dettagliati e l'allegato di file contenenti script Python, che consentono a ChatGPT-4o di apprendere come strutturare e sviluppare nuovi script in modo autonomo.

5.1.1. Processo di addestramento tramite prompt e dati in input

L'addestramento di ChatGPT-4o per la generazione di codice specifico per il robot Reachy si basa su una strategia metodologica che combina l'utilizzo di prompt ben strutturati con l'allegato di file script preesistenti. L'obiettivo principale di questo processo è consentire al modello di acquisire una comprensione approfondita delle operazioni richieste e di generare codice funzionale per compiti specifici legati al controllo e alla programmazione del robot antropomorfo.

Il processo di addestramento inizia con un **test pilota** mirato a valutare le capacità iniziali del modello nel generare codice per compiere un'azione di grasping. In questa fase, viene utilizzato un prompt dettagliato, che fornisce al modello tutte le specifiche necessarie per eseguire correttamente il compito, inclusi parametri come la posizione della pallina da afferrare, le coordinate spaziali del tavolo e la sequenza di operazioni da seguire. Al prompt viene allegata una serie di script esistenti che implementano già un'azione di grasping per Reachy. Questi script fungono da riferimento per il modello, facilitandone l'apprendimento e la generazione di codice appropriato.

Il test pilota rappresenta un punto di partenza essenziale, in quanto consente di valutare se il modello è in grado di comprendere e replicare le operazioni indicate nel prompt. I risultati di questo test saranno utilizzati per calibrare il processo di

addestramento nelle fasi successive, garantendo che il modello riceva un feedback strutturato e utile per il miglioramento continuo.

Dopo il test pilota, il processo di addestramento procede con una serie di Test Sistematici. Questi test sono progettati per incrementare gradualmente la complessità dei task e la quantità di dati forniti. Ogni test rappresenta un passo successivo nell'addestramento del modello, aumentando la sua capacità di comprendere e generare codice più complesso e meno vincolato da istruzioni specifiche.

La progressione dei **test sistematici** avviene secondo i seguenti stadi:

- **Primo stadio:** In questa fase, al modello viene fornito esclusivamente il set di script che implementano l'azione di grasping, senza ulteriori dettagli. Il prompt è più generico e con meno vincoli rispetto al Test Pilota. L'obiettivo è generare degli script che permettano a Reachy di compiere movimenti semplici.
- **Secondo stadio:** Nel secondo stadio oltre agli script di grasping, viene fornito anche lo script che permette di controllare l'inclinazione della testa di Reachy. Il prompt in questa fase rimane sempre generico, lasciando maggiore libertà al modello di decidere come integrare le diverse funzionalità degli script forniti. L'obiettivo è generare degli script che permettano a Reachy di compiere movimenti più complessi come la manipolazione di oggetti.
- **Terzo stadio:** Nell'ultimo stadio dei test sistematici, al modello viene fornito un set completo di script, inclusi quelli per il grasping e il controllo della testa, insieme a un'immagine della scena di Reachy nel simulatore. Questo stadio richiede al modello di utilizzare informazioni visive per eseguire il task, integrando dati provenienti da fonti diverse (codice e immagine) per generare degli script che siano in grado di interagire correttamente con l'ambiente simulato. Il prompt è formulato in maniera generica, per valutare la capacità del modello di gestire input complessi e multi-modalità. L'obiettivo è generare degli script che permettano a Reachy di compiere movimenti più complessi come la manipolazione di oggetti.

Lo scopo dell'intero processo di addestramento è quello di migliorare progressivamente la capacità del modello di generare codice efficiente, affidabile e flessibile per compiti specifici di robotica. I test sistematici sono strutturati in modo da affinare le capacità di ChatGPT-4o nel comprendere e applicare concetti di programmazione robotica a diversi livelli di astrazione. Inoltre, fornire al modello una combinazione di prompt dettagliati e generici consente di testare la sua abilità di adattamento a diverse situazioni, aumentando la robustezza e la versatilità degli script generati.

Il metodo di addestramento descritto in questo sottoparagrafo mira a sviluppare un sistema che non solo sia in grado di replicare operazioni standard, ma che possa anche adattarsi a nuove istruzioni con una minima necessità di supervisione umana.

Utilizzando script predefiniti e prompt mirati, ChatGPT-4o viene addestrato a comprendere il contesto operativo di Reachy, acquisendo la capacità di generare codice che rispetti i requisiti funzionali e operativi richiesti.

5.1.2. Configurazione dell'ambiente di addestramento

Per l'addestramento di ChatGPT-4o con script Python relativi al controllo del robot antropomorfo Reachy, è necessario configurare un ambiente di sviluppo appropriato che permetta di interagire con l'API di OpenAI in modo efficiente. Questo ambiente consente di gestire diverse tipologie di input, inclusi file di testo e file binari, come immagini, per arricchire il contesto della conversazione con il modello e ottenere risposte più precise e mirate. Il processo di configurazione prevede l'installazione della libreria `openai`, la gestione delle chiavi API e l'implementazione di uno script che permette di gestire file allegati e risposte multiple.

Ecco il codice completo utilizzato per interagire con l'API di ChatGPT-4o:

```
import openai
import os

# Imposta la tua chiave API
openai.api_key = "sk-example"

# Inizializza il contesto della conversazione
messages = [
```

```

        {"role": "system", "content": "Sei un esperto di algoritmi di
programmazione per robot antropomorfi."},
    ]

# Funzione per leggere il contenuto di un file
def leggi_file(file_path):
    try:
        # Controlla l'estensione del file per decidere se leggerlo
        # come testo o binario
        _, estensione = os.path.splitext(file_path)
        if estensione.lower() in ['.jpg', '.png', '.jpeg', '.bmp',
'.gif']: # File binari (immagini)
            with open(file_path, "rb") as file:
                return f"Contenuto binario del file {file_path}:
{len(file.read())} byte"
        else: # File di testo
            with open(file_path, "r", encoding="utf-8") as file:
                return file.read()
    except FileNotFoundError:
        return f"File non trovato: {file_path}"
    except UnicodeDecodeError:
        return f"Errore di decodifica del file di testo:
{file_path}"

# Ciclo per inviare messaggi all'assistente
while True:
    message = input("User: ")

    # Esci dal ciclo se l'utente digita "esci"
    if message.lower() == "esci":
        print("Chat terminata.")
        break

    # Se l'utente digita "allega file", chiedi uno o più percorsi
    # dei file e leggi i contenuti
    if message.lower() == "allega file":
        file_paths = input("Inserisci i percorsi dei file separati
da virgola: ").split(',')
        file_contents = []

        # Leggi ciascun file e aggiungi il contenuto
        for file_path in file_paths:
            file_content = leggi_file(file_path.strip())
            file_contents.append(file_content)

        # Combina i contenuti dei file
        message = f"Ecco i contenuti dei file:\n\n" +
"\n\n".join(file_contents)

        # Aggiungi il messaggio dell'utente
        messages.append({"role": "user", "content": message})

        # Invio della richiesta al modello GPT-4o mini con una
        # singola risposta (nessuna risposta multipla)
        response = openai.ChatCompletion.create(
            model="gpt-4o-mini",
            messages=messages,
            temperature=0.7 # Parametro di variabilità
        )

```

```

# Ottieni la singola risposta
reply = response['choices'][0]['message']['content']
print(f"ChatGPT-4o mini: {reply}")

# Aggiungi la risposta dell'assistente al contesto
messages.append({"role": "assistant", "content": reply})

else:
    # Per altri messaggi (non allega file), invia una
    richiesta con 5 risposte multiple
    messages.append({"role": "user", "content": message})

    # Invio della richiesta al modello GPT-4o mini con 5
    risposte multiple
    response = openai.ChatCompletion.create(
        model="gpt-4o-mini",
        messages=messages,
        n=5, # Genera 5 risposte multiple
        temperature=0.7, # Parametro di variabilità
        top_p=0.9
    )

    # Itera su ciascuna risposta
    for i, choice in enumerate(response['choices']):
        reply = choice['message']['content']
        print(f"\nRisposta {i + 1}:\n{reply}\n")

        # Aggiungi ciascuna risposta dell'assistente al
        contesto
        messages.append({"role": "assistant", "content":
reply})

```

L'implementazione del codice è stata suddivisa in diverse fasi:

- **Installazione della libreria openai:** Il primo passaggio consiste nell'installare la libreria openai, passaggio spiegato nei capitoli precedenti. Questa libreria fornisce le funzionalità necessarie per interagire con l'API di OpenAI e inviare richieste al modello GPT. È una componente fondamentale per il corretto funzionamento dello script di addestramento.



Figura 5.1. Logo di OpenAI

- **Impostazione della chiave API:** Dopo l'installazione, è necessario configurare la chiave API personale, che garantisce l'accesso autorizzato al servizio. La chiave viene memorizzata nella variabile `openai.api_key`. Questo passaggio è cruciale per permettere all'applicazione di inviare richieste all'API di OpenAI e ricevere risposte appropriate.
- **Inizializzazione del contesto di conversazione:** Viene definito un contesto iniziale per la conversazione con il modello. Questo contesto include un messaggio di sistema che stabilisce il ruolo dell'assistente, come ad esempio: "Sei un esperto di algoritmi di programmazione per robot antropomorfi." Questo permette al modello di comprendere meglio il tipo di risposte da fornire e mantenere un focus specifico durante le interazioni.
- **Funzione di lettura dei file:** Lo script include una funzione denominata `leggi_file()`, che è in grado di gestire sia file di testo che file binari, come immagini. Questa funzione verifica l'estensione del file per determinarne il tipo e legge il contenuto di conseguenza. Per i file di testo, il contenuto viene letto e restituito in formato leggibile. Per i file binari, invece, viene restituito il numero di byte, indicativo delle dimensioni del file. Questa funzionalità consente di allegare diversi tipi di file al contesto della conversazione, arricchendo le informazioni disponibili per il modello.
- **Ciclo di richiesta-risposta con gestione file:** Lo script è progettato per interagire dinamicamente con l'utente. Quando viene richiesto di allegare uno o più file, la funzione `leggi_file()` viene utilizzata per leggere e combinare i contenuti dei file in un unico messaggio, che viene quindi inviato al modello. Questo consente al modello di basare le proprie risposte sul contenuto dei file caricati, migliorando l'accuratezza delle risposte generate.
- **Generazione di risposte multiple:** Se l'utente invia messaggi che non includono file, lo script è programmato per richiedere fino a 5 risposte multiple al modello. Queste risposte vengono generate utilizzando parametri di variabilità come `temperature`, che controllano il livello di creatività e diversificazione delle risposte. Questo permette di ottenere una

gamma di soluzioni diverse per lo stesso problema, facilitando la selezione della risposta migliore.

- **Personalizzazione dei parametri di risposta:** Lo script utilizza parametri configurabili come `temperature` per controllare la diversità delle risposte. Il parametro `temperature` regola l'imprevedibilità delle risposte: valori più alti rendono le risposte più varie, mentre valori più bassi le rendono più mirate.

5.2. TEST PILOTA E VALUTAZIONE INZIALE

Il test pilota rappresenta il primo passo per valutare le capacità di ChatGPT-4o nella generazione di script Python per il robot Reachy. Durante questa fase, verrà utilizzato un prompt specifico e dettagliato, insieme a script di riferimento allegati, per testare la capacità del modello di comprendere e replicare task di grasping. Questo test iniziale è fondamentale per stabilire un punto di riferimento sulle capacità attuali del modello e identificare eventuali aree di miglioramento prima di procedere con test più complessi.

5.2.1. Formulazione delle richieste del task specifico

Il prompt selezionato per questo task è stato progettato per chiedere la generazione di uno script Python specifico, basato sugli script precedentemente forniti e utilizzando la libreria Reachy SDK. Il prompt definisce chiaramente un'operazione di grasping (presa di un oggetto) e richiede al modello di seguire una sequenza di azioni ordinate e precise, tenendo conto della posizione spaziale della pallina da basket e delle coordinate del tavolo rispetto al sistema di riferimento di Reachy.

La formulazione del prompt è suddivisa in diverse componenti essenziali:

1. **Descrizione del task generale:** La prima parte del prompt introduce il task generale, ossia l'obiettivo di far sviluppare a ChatGPT-4o uno script Python che permetta a Reachy di afferrare una pallina da basket situata sul tavolo. Si specifica l'uso della libreria Reachy SDK (versione 0.5.4), fondamentale per il controllo del robot. In questa fase, è importante includere il contesto,

specificando la posizione della pallina da basket e il suo rapporto rispetto alla visuale di Reachy.

2. **Sequenza dettagliata delle azioni:** Successivamente, viene fornita una lista dettagliata delle azioni che Reachy dovrà eseguire per completare il task. La sequenza è ordinata in modo logico, partendo dalla rotazione della testa di Reachy verso la pallina, fino al grasping (presa) e al rilascio finale della pallina sul tavolo. Questa lista sequenziale garantisce che il modello possa produrre un codice che rispetti l'ordine corretto delle operazioni, senza saltare passaggi critici.
3. Le azioni specifiche includono:
 - **Rotazione della testa:** Per orientare Reachy verso la pallina.
 - **Movimento del braccio destro:** Per posizionare la pinza destra sopra la pallina.
 - **Apertura e chiusura della pinza:** Per afferrare e rilasciare la pallina.
 - **Sollevamento e abbassamento del braccio:** Per completare l'azione di grasping.
 - **Coordinate del tavolo:** Un altro elemento chiave nella formulazione del prompt riguarda la specifica delle coordinate del tavolo rispetto al sistema di riferimento di Reachy. Le coordinate $(x, y, z) = (0.3187999, -0.01479991, -1.204)$ vengono fornite per consentire al modello di posizionare correttamente il braccio e la pinza di Reachy in relazione all'oggetto da afferrare. La specifica delle coordinate è fondamentale per garantire che il codice generato tenga conto della posizione fisica degli oggetti nello spazio tridimensionale e che le operazioni vengano eseguite con precisione.

Il prompt formulato, in base agli elementi discussi, è il seguente:

Sulla base degli script Python che ti ho lasciato in allegato, sviluppa uno script Python che permetta a Reachy di afferrare una pallina da basket sul tavolo (quindi che compia un'azione di grasping).

In particolare, la pallina da basket è situata un po' di centimetri più a destra rispetto alla visuale frontale iniziale di Reachy, un po' di centimetri sopra il centro del tavolo.

Esegui le seguenti azioni in ordine:

1. rivolgere gli occhi davanti alla pallina da basket da afferrare;
2. posizionare la pinza destra sopra la pallina da basket spostando il braccio destro;
3. aprire la pinza destra;
4. scendere il braccio destro fino a farlo poggiare sul tavolo;
5. afferrare la palla;
6. chiudere la pinza destra;
7. sollevare il braccio destro con la pallina nella pinza destra;
8. riscendere il braccio destro fino a farlo poggiare sul tavolo;
9. aprire la pinza destra;
10. rilasciare la pallina sul tavolo;
11. riportare il braccio destro alla posizione iniziale.

Le coordinate del tavolo rispetto al sistema di riferimento di Reachy sono le seguenti: $(x, y, z) = (0, 3187999, -0, 01479991, -1, 204)$.

Utilizzare la libreria Reachy SDK (versione 0.5.4).

La formulazione del prompt presenta alcune caratteristiche fondamentali:

- **Precisione:** Ogni passaggio è descritto in modo dettagliato per evitare ambiguità e fornire a ChatGPT-4o istruzioni chiare e sequenziali. Questo approccio aiuta il modello a generare codice che segue una logica operativa coerente.
- **Specificità delle coordinate:** L'inclusione delle coordinate del tavolo garantisce che il modello utilizzi valori di riferimento precisi per posizionare il braccio e la pinza di Reachy in relazione alla pallina da basket. Ciò è fondamentale per assicurare l'accuratezza dell'interazione fisica del robot con l'ambiente circostante.
- **Modularità e flessibilità:** Sebbene la richiesta si concentri su un'operazione specifica di grasping, il prompt è strutturato in modo modulare, permettendo al modello di riconoscere e riutilizzare singole operazioni, come il movimento della testa o l'apertura della pinza, in altri task futuri.

5.2.2. Test degli script generati

Il processo di test degli script generati dall'API di ChatGPT-4o per il controllo del robot Reachy è stato condotto con l'obiettivo di valutare l'efficacia e la precisione degli script prodotti in risposta a un task specifico. In questo caso, il compito

richiesto a ChatGPT era quello di generare uno script Python che consentisse a Reachy di eseguire con successo l'operazione di grasping su una pallina da basket posizionata su un tavolo.

Metodologia di test

1. **Caricamento ed esecuzione degli script:** Ogni script generato è stato caricato nell'ambiente di simulazione Unity, dove Reachy ha eseguito i movimenti programmati. L'esecuzione di ciascuno script è stata attentamente monitorata per valutare la conformità dei movimenti alle istruzioni specificate nel prompt.
2. **Osservazione e valutazione del comportamento:** Durante l'esecuzione, sono stati osservati i seguenti parametri chiave:
 - Orientamento della testa di Reachy.
 - Movimenti del braccio destro in direzione della pallina.
 - Corretta esecuzione delle azioni di apertura e chiusura della pinza per afferrare la pallina.
 - Capacità di Reachy di rilasciare la pallina dopo averla afferrata.
3. **Validazione dei risultati:** Il comportamento di Reachy è stato confrontato con le azioni richieste nel prompt per verificare la coerenza dell'esecuzione con le aspettative. Sono stati annotati i successi parziali e le discrepanze, al fine di comprendere le aree in cui gli script richiedono miglioramenti.

Risultati dei Test

- **Script generato 1:**
 - **Esecuzione generale:** Lo script è stato eseguito senza errori di compilazione o runtime, indicando che il codice generato è tecnicamente corretto.
 - **Orientamento della testa:** La testa di Reachy si è voltata verso il lato sinistro del tavolo, senza concentrare lo sguardo sulla pallina.
 - **Movimento del braccio:** Il braccio ha eseguito un movimento verso la pallina, ma non è riuscito a raggiungerla con precisione, mancando di poco l'afferramento.

- **Apertura e chiusura della pinza:** La pinza si è aperta e chiusa correttamente nei tempi previsti, anche se non è riuscita ad afferrare la pallina.

Codice generato per lo script 1:

```

from reachy_sdk import ReachySDK
from reachy_sdk.trajectory import goto
from reachy_sdk.trajectory import InterpolationMode
import time
import numpy as np

# Funzione per far guardare il robot verso un obiettivo
def look_at(target):
    reachy.head.look_at(target[0], target[1], target[2],
1.0)

# Funzione per muovere il braccio del robot verso una
posizione target
def move_arm(target, duration):
    target_matrix = np.array([
        [0, 0, -1, target[0]],
        [0, 1, 0, target[1]],
        [1, 0, 0, target[2]],
        [0, 0, 0, 1],
    ])
    joint_pos =
reachy.r_arm.inverse_kinematics(target_matrix)
    reachy.turn_on('r_arm')
    goto({joint: pos for joint, pos in
zip(reachy.r_arm.joints.values(), joint_pos)},
duration=duration)

# Funzione per aprire la pinza
def open_gripper():
    reachy.r_arm.r_gripper.goal_position = -60 # Valore per
aprire la pinza
    time.sleep(1) # Attendere che la pinza si apra

# Funzione per chiudere la pinza
def close_gripper():
    reachy.r_arm.r_gripper.goal_position = -10 # Valore per
chiudere la pinza
    time.sleep(1) # Attendere che la pinza si chiuda

# Funzione principale per afferrare la pallina
def grasp_ball():
    # Coordinate della pallina da basket (leggermente a
destra e sopra il tavolo)
    ball_position = np.array([0.10, 0.3187999, -0.01479991])
    # Aggiustare x per la posizione a destra

    # 1. Far guardare il robot verso la pallina
    look_at(ball_position)

```

```

# 2. Posizionare la pinza sopra la pallina
above_ball = np.array([ball_position[0],
ball_position[1], ball_position[2] + 0.1]) # Posizionare
sopra la palla
move_arm(above_ball, 2.0)

# 3. Aprire la pinza
open_gripper()

# 4. Scendere il braccio fino a farlo poggiare sul
tavolo
on_table = np.array([ball_position[0], ball_position[1],
ball_position[2] - 0.1]) # Poggiato sul tavolo
move_arm(on_table, 2.0)

# 5. Afferrare la palla
close_gripper()

# 6. Sollevare il braccio con la pallina
move_arm(above_ball, 2.0)

# 7. Riscendere il braccio fino a farlo poggiare sul
tavolo
move_arm(on_table, 2.0)

# 8. Aprire la pinza per rilasciare la pallina
open_gripper()

# 9. Riportare il braccio alla posizione iniziale
initial_position = np.array([0.4, 0.3187999, -
0.01479991]) # Posizione iniziale del braccio
move_arm(initial_position, 2.0)

# Codice eseguibile principale
if __name__ == "__main__":
    # Connessione al robot Reachy
    reachy = ReachySDK(host='localhost')

    # Eseguire la funzione di afferrare la pallina
    grasp_ball()

    # Spegner il braccio destro del robot in modo graduale
    reachy.turn_off_smoothly('r_arm')

    # Uscire dal programma
    exit()

```

- **Script generato 2:**

- **Esecuzione generale:** Anche questo script è stato eseguito senza errori.
- **Orientamento della testa:** La testa si è nuovamente voltata verso il lato sinistro, non centrando la visuale sul tavolo.

- **Movimento del braccio:** Il braccio non ha eseguito il movimento richiesto verso la pallina, limitandosi a restare fermo.
- **Apertura e chiusura della pinza:** Non è stata presa in considerazione, poiché il braccio non si è mosso in direzione della pallina.

Codice generato per lo script 2:

```

from reachy_sdk import ReachySDK
from reachy_sdk.trajectory import goto
from reachy_sdk.trajectory import InterpolationMode
import time
import numpy as np

# Funzione per far guardare il robot verso un obiettivo
def look_at(target):
    reachy.head.look_at(target[0], target[1], target[2],
1.0)

# Funzione per muovere il braccio del robot verso una
posizione target
def move_arm(target, duration):
    target_matrix = np.array([
        [0, 0, -1, target[0]],
        [0, 1, 0, target[1]],
        [1, 0, 0, target[2]],
        [0, 0, 0, 1],
    ])

    joint_pos =
reachy.r_arm.inverse_kinematics(target_matrix)
    reachy.turn_on('r_arm')
    goto({joint: pos for joint, pos in
zip(reachy.r_arm.joints.values(), joint_pos)},
duration=duration)

# Funzione per aprire la pinza
def open_gripper():
    reachy.r_arm.r_gripper.goal_position = -60 # Imposta la
posizione di apertura della pinza
    time.sleep(1)

# Funzione per chiudere la pinza
def close_gripper():
    reachy.r_arm.r_gripper.goal_position = -40 # Imposta la
posizione di chiusura della pinza
    time.sleep(1)

# Funzione principale per afferrare la pallina da basket
def grasp_basketball():
    # Coordinate della pallina da basket
    ball_position = np.array([0.05, 0.3187999, -0.01479991])
    # Aggiusta x per posizionare la pallina a destra

```

```

    table_position = np.array([0.05, 0.3187999, -0.1]) #
    Posizione sul tavolo per poggiare il braccio

    # 1. Rivolgere gli occhi verso la pallina
    look_at(ball_position)

    # 2. Posizionare la pinza sopra la pallina
    above_ball_position = np.array([ball_position[0],
    ball_position[1], ball_position[2] + 0.1]) # Posizione
    sopra la pallina
    move_arm(above_ball_position, 2.0)

    # 3. Aprire la pinza
    open_gripper()

    # 4. Scendere il braccio fino a farlo poggiare sul
    tavolo
    move_arm(table_position, 2.0)

    # 5. Afferrare la palla
    close_gripper()

    # 7. Sollevare il braccio con la pallina
    move_arm(above_ball_position, 2.0)

    # 8. Riscendere il braccio fino a farlo poggiare sul
    tavolo
    move_arm(table_position, 2.0)

    # 9. Aprire la pinza per rilasciare la pallina
    open_gripper()

    # 10. Rilasciare la pallina sul tavolo
    # (Non è necessario alcun movimento aggiuntivo poiché il
    braccio è già poggiato sul tavolo)

    # 11. Riportare il braccio alla posizione iniziale
    initial_position = np.array([0, 0.3, 0]) # Adatta
    questa posizione alla posizione iniziale desiderata
    move_arm(initial_position, 2.0)

# Codice eseguibile principale
if __name__ == "__main__":
    reachy = ReachySDK(host='localhost')
    grasp_basketball()
    reachy.turn_off_smoothly('r_arm')
    exit()

```

- **Script generato 3:**

- **Esecuzione generale:** Lo script si è avviato correttamente.
- **Orientamento della testa:** Ancora una volta, la testa si è orientata erroneamente verso il lato sinistro.
- **Movimento del braccio:** Il braccio non si è mosso in direzione della pallina e non ha eseguito il compito richiesto.

- **Apertura e chiusura della pinza:** Non è stata presa in considerazione, poiché il braccio non si è mosso in direzione della pallina.

Codice generato per lo script 3:

```
from reachy_sdk import ReachySDK
from reachy_sdk.trajectory import goto
from reachy_sdk.trajectory import InterpolationMode
import time
import numpy as np

# Coordinate della pallina da basket
basketball_position = np.array([0.1, 0.3187999, -
0.01479991]) # x leggermente a destra, y e z sul tavolo

# Funzione per far guardare il robot verso un obiettivo
def look_at(target):
    reachy.head.look_at(target[0], target[1], target[2],
1.0)

# Funzione per muovere il braccio del robot verso una
posizione target
def move_arm(target, duration):
    target_matrix = np.array([
        [0, 0, -1, target[0]],
        [0, 1, 0, target[1]],
        [1, 0, 0, target[2]],
        [0, 0, 0, 1],
    ])
    joint_positions =
reachy.r_arm.inverse_kinematics(target_matrix)
    reachy.turn_on('r_arm')
    goto({joint: pos for joint, pos in
zip(reachy.r_arm.joints.values(), joint_positions)},
duration=duration)

# Funzione per eseguire il grasping della pallina
def grasp_ball():
    # 1. Rivolgere gli occhi davanti alla pallina da basket
    look_at(basketball_position)

    # 2. Posizionare la pinza destra sopra la pallina da
basket
    above_ball_position = np.array([basketball_position[0],
basketball_position[1], basketball_position[2] + 0.1])
    move_arm(above_ball_position, 1.0)

    # 3. Aprire la pinza destra
    reachy.r_arm.r_gripper.goal_position = -60 # Posizione
aperta
    time.sleep(1.0)

    # 4. Scendere il braccio destro fino a farlo poggiare
sul tavolo
    move_arm(basketball_position, 1.0)
```

```

time.sleep(1.0)

# 5. Afferrare la palla
reachy.r_arm.r_gripper.goal_position = -40 # Posizione
chiusa
time.sleep(0.5)

# 6. Sollevare il braccio destro con la pallina nella
pinza destra
move_arm(above_ball_position, 1.0)

# 7. Riscendere il braccio destro fino a farlo poggiare
sul tavolo
move_arm(basketball_position, 1.0)
time.sleep(1.0)

# 8. Aprire la pinza destra
reachy.r_arm.r_gripper.goal_position = -60 # Posizione
aperta
time.sleep(1.0)

# 9. Rilasciare la pallina sul tavolo
# Non serve fare nulla di speciale qui, la pinza è già
aperta.

# 10. Riportare il braccio destro alla posizione
iniziale
initial_position = np.array([0.0, 0.3187999, 0.1]) #
Posizione iniziale del braccio
move_arm(initial_position, 1.0)

# Codice eseguibile principale
if __name__ == "__main__":
    reachy = ReachySDK(host='localhost')

    grasp_ball()

    # Spegnerne il braccio destro del robot in modo graduale
    reachy.turn_off_smoothly('r_arm')

    # Uscire dal programma
    exit()

```

- **Script generato 4:**

- **Esecuzione generale:** Lo script ha funzionato senza errori.
- **Orientamento della testa:** La testa di Reachy è rimasta frontale, come richiesto dal prompt, con gli occhi rivolti verso il tavolo.
- **Movimento del braccio:** Tuttavia, il braccio non si è mosso correttamente verso la pallina, impedendo il completamento del grasping.

- **Apertura e chiusura della pinza:** Non è stata presa in considerazione, poiché il braccio non si è mosso in direzione della pallina.

Codice generato per lo script 4:

```

from reachy_sdk import ReachySDK
from reachy_sdk.trajectory import goto
from reachy_sdk.trajectory import InterpolationMode
import time
import numpy as np

# Funzione per far guardare il robot verso un obiettivo
def look_at(target):
    reachy.head.look_at(target[0], target[1], target[2],
1.0)

# Funzione per muovere il braccio del robot verso una
posizione target con una durata specificata
def move_arm(target, duration):
    target_matrix = np.array([
        [0, 0, -1, target[0]],
        [0, 1, 0, target[1]],
        [1, 0, 0, target[2]],
        [0, 0, 0, 1],
    ])
    joint_pos =
reachy.r_arm.inverse_kinematics(target_matrix)
    reachy.turn_on('r_arm')
    goto({joint: pos for joint, pos in
zip(reachy.r_arm.joints.values(), joint_pos)},
duration=duration)

# Funzione principale per afferrare la pallina da basket
def grasp_basketball():
    # Coordinate del tavolo
    table_position = np.array([0.3187999, 0, -1.204]) # (x,
y, z)

    # 1. Rivolgere gli occhi davanti alla pallina da basket
    ball_position = np.array([0.3187999 + 0.1, 0, -1.204 +
0.1]) # Un po' a destra e sopra il tavolo
    look_at(ball_position)

    # 2. Posizionare la pinza destra sopra la pallina
    above_ball_position = ball_position + np.array([0, 0,
0.1]) # Posizione sopra la palla
    move_arm(above_ball_position, 1.0)
    time.sleep(1.0)

    # 3. Aprire la pinza destra
    reachy.r_arm.r_gripper.goal_position = -60 # Apertura
della pinza
    time.sleep(1.0)

```

```

# 4. Scendere il braccio destro fino a farlo poggiare
sul tavolo
move_arm(table_position, 1.0)
time.sleep(1.0)

# 5. Afferrare la palla (chiusura della pinza)
reachy.r_arm.r_gripper.goal_position = -40 # Chiusura
della pinza
time.sleep(0.5)

# 7. Sollevare il braccio destro con la pallina nella
pinza
move_arm(above_ball_position, 1.0)
time.sleep(1.0)

# 8. Riscendere il braccio destro fino a farlo poggiare
sul tavolo
move_arm(table_position, 1.0)
time.sleep(1.0)

# 9. Aprire la pinza destra
reachy.r_arm.r_gripper.goal_position = -60 # Apertura
della pinza
time.sleep(1.0)

# 10. Rilasciare la pallina sul tavolo
# (già rilasciata aprendo la pinza)

# 11. Riportare il braccio destro alla posizione
iniziale
initial_position = np.array([0.5, 0, 0]) # Sostituisci
con le coordinate iniziali desiderate
move_arm(initial_position, 1.0)
time.sleep(1.0)

if __name__ == "__main__":
    # Connessione al robot Reachy
    reachy = ReachySDK(host='localhost')

    # Eseguire l'operazione di afferrare la pallina da
    basket
    grasp_basketball()

    # Spegnerne il braccio destro del robot in modo graduale
    reachy.turn_off_smoothly('r_arm')

```

- **Script generato 5:**

- **Esecuzione generale:** Lo script ha funzionato correttamente e, rispetto agli altri, ha mostrato un miglioramento significativo.
- **Orientamento della testa:** La testa si è orientata verso il lato sinistro, ma il movimento del braccio è stato più preciso rispetto agli altri script.

- **Movimento del braccio:** Anche se il braccio ha mancato di poco l'afferramento, la traiettoria è stata più accurata, arrivando a compiere un contatto con la pallina e quindi mostrando un miglioramento rispetto agli altri tentativi.
- **Apertura e chiusura della pinza:** La pinza si è aperta e chiusa nei tempi corretti, ma non è riuscita a completare l'afferramento.

Codice generato per lo script 5:

```
from reachy_sdk import ReachySDK
from reachy_sdk.trajectory import goto
from reachy_sdk.trajectory import InterpolationMode
import time
import numpy as np

# Funzione per far guardare il robot verso un obiettivo
def look_at(target):
    reachy.head.look_at(target[0], target[1], target[2],
1.0)

# Funzione per muovere il braccio del robot verso una
posizione target
def move_arm(target, duration):
    target_matrix = np.array([
        [0, 0, -1, target[0]],
        [0, 1, 0, target[1]],
        [1, 0, 0, target[2]],
        [0, 0, 0, 1],
    ])
    joint_pos =
reachy.r_arm.inverse_kinematics(target_matrix)
    reachy.turn_on('r_arm')
    goto({joint: pos for joint, pos in
zip(reachy.r_arm.joints.values(), joint_pos)},
duration=duration)

# Funzione per aprire la pinza
def open_gripper():
    reachy.r_arm.r_gripper.goal_position = -60 # Valore per
aprire la pinza
    time.sleep(1) # Attendere che la pinza si apra

# Funzione per chiudere la pinza
def close_gripper():
    reachy.r_arm.r_gripper.goal_position = -10 # Valore per
chiudere la pinza
    time.sleep(1) # Attendere che la pinza si chiuda

# Funzione principale per afferrare la pallina
def grasp_ball():
    # Coordinate della pallina da basket (leggermente a
destra e sopra il tavolo)
```

```

    ball_position = np.array([0.10, 0.3187999, -0.01479991])
# Aggiustare x per la posizione a destra

    # 1. Far guardare il robot verso la pallina
    look_at(ball_position)

    # 2. Posizionare la pinza sopra la pallina
    above_ball = np.array([ball_position[0],
ball_position[1], ball_position[2] + 0.1]) # Posizionare
sopra la palla
    move_arm(above_ball, 2.0)

    # 3. Aprire la pinza
    open_gripper()

    # 4. Scendere il braccio fino a farlo poggiare sul
tavolo
    on_table = np.array([ball_position[0], ball_position[1],
ball_position[2] - 0.1]) # Poggiato sul tavolo
    move_arm(on_table, 2.0)

    # 5. Afferrare la palla
    close_gripper()

    # 6. Sollevare il braccio con la pallina
    move_arm(above_ball, 2.0)

    # 7. Riscendere il braccio fino a farlo poggiare sul
tavolo
    move_arm(on_table, 2.0)

    # 8. Aprire la pinza per rilasciare la pallina
    open_gripper()

    # 9. Riportare il braccio alla posizione iniziale
    initial_position = np.array([0.4, 0.3187999, -
0.01479991]) # Posizione iniziale del braccio
    move_arm(initial_position, 2.0)

# Codice eseguibile principale
if __name__ == "__main__":
    # Connessione al robot Reachy
    reachy = ReachySDK(host='localhost')

    # Eseguire la funzione di afferrare la pallina
    grasp_ball()

    # Spegner il braccio destro del robot in modo graduale
    reachy.turn_off_smoothly('r_arm')

    # Uscire dal programma
    exit()

```

Analisi dei risultati

Di seguito è riportata una tabella che sintetizza i risultati ottenuti per ogni movimento richiesto, utilizzando i criteri di valutazione analizzato sopra.

Tabella 5.1. Risultati del test pilota

	Esecuzione generale	Orientamento della testa	Movimento del braccio	Apertura e chiusura della pinza	Aderenza al prompt
Script 1	100%	25%	100%	100%	75%
Script 2	100%	25%	50%	100%	75%
Script 3	100%	25%	75%	100%	25%
Script 4	100%	100%	50%	50%	75%
Script 5	100%	25%	100%	100%	75%

Il test ha dimostrato che ChatGPT-4o è in grado di generare script Python che possono essere eseguiti correttamente senza errori, ma con una certa variabilità nell'esecuzione delle operazioni richieste. In particolare:

- **Precisione del movimento:** Mentre la maggior parte degli script ha eseguito correttamente i movimenti di base, come il posizionamento della testa e il movimento del braccio, pochi sono riusciti a completare con successo l'intera sequenza richiesta per afferrare la pallina.
- **Risultati parziali:** Alcuni script hanno mostrato risultati parziali positivi, come la corretta apertura e chiusura della pinza, ma con imprecisioni nella localizzazione e afferramento dell'oggetto.
- **Miglioramento progressivo:** Il confronto tra gli script ha evidenziato miglioramenti incrementali nelle performance, con lo Script 5 che ha mostrato una maggiore precisione nella realizzazione del task rispetto agli altri.

5.3. TEST SISTEMATICI CON INCREMENTO DI COMPLESSITA' E DEI DATI DI INPUT

Dopo il test pilota iniziale, il passo successivo consiste nell'eseguire una serie di test sistematici per valutare le capacità di ChatGPT-4o nella generazione di script Python per Reachy, con un incremento graduale della complessità dei task e dei dati forniti. Questi test sono progettati per migliorare progressivamente le capacità del modello, permettendo di affrontare task sempre più complessi e diversificati.

5.3.1. Movimenti semplici

Nel processo di addestramento di ChatGPT-4o, una delle fasi iniziali ha riguardato la generazione e l'analisi di script per eseguire movimenti semplici con il robot Reachy. Questa fase ha avuto l'obiettivo di valutare la capacità del modello di generare codice efficace e corretto per compiti elementari, come la rotazione della testa o il movimento delle braccia. Il prompt utilizzato è stato strutturato in modo meno dettagliato rispetto al Test Pilota, con un focus sull'uso della libreria Reachy SDK (versione 0.5.4).

Il prompt utilizzato per questa fase ha la seguente struttura:

```
Sulla base degli script Python che ti ho lasciato in allegato,
sviluppa uno script Python che permetta a Reachy di [movimento
richiesto].
```

```
Utilizzare la libreria Reachy SDK (versione 0.5.4).
```

Questo prompt è stato mantenuto volutamente generico, sostituendo ogni volta la dicitura `[movimento richiesto]` con il movimento specifico richiesto, come:

- Rotazione testa verso destra
- Rotazione testa verso sinistra
- Posizionamento braccio destro in alto
- Posizionamento braccio sinistro in alto
- Simulare il gesto di salutare
- Simulare il movimento di bere

Rispetto al Test Pilota, dove il prompt forniva specifiche dettagliate su ogni singola operazione, qui è stata scelta una struttura più sintetica e generica per testare diverse capacità del modello:

- **Capacità di generalizzazione:** L'uso di un prompt meno dettagliato permette di valutare quanto il modello sia in grado di comprendere e applicare i concetti appresi dai file allegati per eseguire compiti simili, ma con variazioni nelle specifiche.
- **Autonomia nella generazione di codice:** Riducendo la quantità di informazioni nel prompt, si osserva se il modello riesce a integrare le conoscenze acquisite dagli script allegati per generare nuovi script senza un'eccessiva dipendenza da istruzioni minuziose.
- **Risposta a prompt simili ai comandi reali:** Nella pratica, le richieste fatte a un assistente AI sono spesso formulate in modo sintetico. Testare il modello con prompt generici simula meglio un contesto di utilizzo reale, dove le istruzioni potrebbero non essere sempre estremamente dettagliate.
- **Valutazione delle capacità di adattamento:** Un prompt generico richiede al modello di inferire e adattarsi a compiti nuovi, utilizzando le conoscenze esistenti e creando soluzioni che non sono basate su comandi espliciti ma su deduzioni logiche.

I file allegati contenevano script Python già funzionanti per l'azione di grasping, sviluppati precedentemente. Questi script rappresentavano un riferimento utile per ChatGPT-4o, permettendogli di comprendere la struttura di base del codice per eseguire movimenti specifici. Fornendo questi script come contesto, l'intenzione era quella di verificare se il modello fosse in grado di replicare o adattare il comportamento degli script allegati per compiti diversi, come la rotazione della testa o il movimento delle braccia.

Per ogni movimento semplice richiesto, ChatGPT-4o ha generato cinque versioni dello script, utilizzando il contesto fornito dagli script allegati e il prompt generico.

Una volta generati, gli script sono stati testati nel simulatore di Reachy per verificarne il comportamento.

Di seguito è riportata una tabella che sintetizza i risultati ottenuti per ogni movimento richiesto, utilizzando i seguenti criteri di valutazione:

- **Esecuzione generale:** Capacità del robot di eseguire il compito complessivo richiesto.
- **Orientamento della testa:** Accuratezza e fluidità dell'orientamento della testa di Reachy.
- **Movimento del braccio:** Precisione e stabilità del movimento del braccio.
- **Aderenza al prompt:** La capacità dello script di seguire le istruzioni fornite nel prompt, rispettando i requisiti specificati.

Tabella 5.2. Risultati del test di movimenti semplici per la testa di Reachy

	Esecuzione generale	Orientamento della testa	Aderenza al prompt
Rotazione testa verso destra	0%	0%	0%
Rotazione testa verso sinistra	0%	0%	0%

Tabella 5.3. Risultati del test di movimenti semplici per le braccia di Reachy

	Esecuzione generale	Movimento del braccio	Aderenza al prompt
Posizionamento braccio destro in alto	80%	35%	20%
Posizionamento braccio sinistro in alto	100%	40%	20%
Simulare il gesto di salutare	100%	40%	20%
Simulare il movimento di bere	90%	60%	60%

5.3.2. Movimenti complessi

In questa fase, è stato utilizzato un prompt generico per valutare la capacità di ChatGPT-4o di generare script Python in grado di eseguire movimenti complessi con il robot Reachy. Rispetto al test pilota, in cui venivano fornite istruzioni dettagliate per ogni passaggio, qui si è preferito un approccio meno vincolante per permettere al modello di interpretare e gestire autonomamente le informazioni disponibili.

Il prompt utilizzato per questa fase ha la seguente struttura:

```
Sulla base degli script Python che ti ho lasciato in allegato,
sviluppa uno script Python che permetta a Reachy di [movimento
richieste].
```

In particolare, la pallina da basket è situata un po' di centimetri più a destra (o a sinistra nel caso del task 4) rispetto alla visuale frontale iniziale di Reachy, un po' di centimetri sopra il centro del tavolo.

Le coordinate del tavolo rispetto al sistema di riferimento di Reachy sono le seguenti: $(x, y, z) = (0,3187999, -0,01479991, -1,204)$. Utilizzare la libreria Reachy SDK (versione 0.5.4).

Questo prompt è stato mantenuto volutamente generico, sostituendo ogni volta la dicitura `[movimento richiesto]` con il movimento specifico richiesto, come:

- Afferrare una pallina da basket
- Mettere una pallina da basket su una tazza
- Spostare una pallina da basket da destra verso sinistra
- Spostare una pallina da Basket da sinistra verso destra

Rispetto al test pilota, questo prompt più generico è stato scelto per diverse ragioni:

- **Valutare la generalizzazione:** Un prompt generico consente di osservare come il modello utilizzi le conoscenze acquisite in contesti simili per generare script validi senza avere istruzioni specifiche per ogni dettaglio. Questo è utile per capire se il modello riesce a creare soluzioni nuove basate su informazioni limitate.

- **Flessibilità e adattamento:** Fornendo solo le informazioni di base e il contesto generale, il modello viene incentivato a sviluppare strategie autonome per completare il task. Questo è importante per testare l'abilità del modello di adattarsi a situazioni diverse senza essere guidato passo-passo.
- **Riduzione della dipendenza da istruzioni specifiche:** L'obiettivo è far sì che ChatGPT-4o non dipenda eccessivamente da dettagli minuziosi, ma che possa inferire correttamente le azioni da compiere basandosi su un livello più alto di astrazione.
- **Simulazione di richieste reali:** Nella pratica, le richieste fatte a un assistente AI potrebbero essere meno dettagliate e più generiche. Questo tipo di prompt simula meglio un contesto reale di utilizzo, dove le informazioni potrebbero essere limitate.

Gli script allegati per il training comprendevano esempi già funzionanti per compiti di grasping e per l'orientamento della testa. Questi script sono stati utilizzati come base di riferimento, fornendo a ChatGPT-4o i modelli di codice da cui apprendere le sequenze di operazioni necessarie.

Per ogni task richiesto, ChatGPT-4o ha generato cinque diverse versioni dello script, utilizzando il contesto fornito dagli script allegati e il prompt generico. Gli script sono stati testati nel simulatore di Reachy per verificare il comportamento e la correttezza del codice.

Di seguito è riportata una tabella che sintetizza i risultati ottenuti per ogni movimento richiesto, utilizzando i seguenti criteri di valutazione:

- **Esecuzione generale:** Capacità del robot di eseguire il compito complessivo richiesto.
- **Orientamento della testa:** Accuratezza e fluidità dell'orientamento della testa di Reachy.
- **Movimento del braccio:** Precisione e stabilità del movimento del braccio.
- **Apertura e chiusura della pinza:** Correttezza e tempismo nell'apertura e chiusura della pinza per la manipolazione dell'oggetto.

- **Apertura e chiusura della pinza:** Correttezza e tempismo nell'apertura e chiusura della pinza per la manipolazione dell'oggetto.
- **Aderenza al prompt:** Capacità dello script di seguire le istruzioni fornite nel prompt, rispettando i requisiti specificati.

Tabella 5.4. Risultati del test per movimenti complessi

	Esecuzione generale	Orientamento della testa	Movimento del braccio	Apertura e chiusura della pinza	Aderenza al prompt
Afferrare una pallina da Basket	100%	20%	70%	80%	50%
Mettere una pallina da basket su una Tazza	100%	10%	40%	100%	15%
Spostare una pallina da basket da destra verso sinistra	100%	20%	15%	100%	5%
Spostare una pallina da basket da sinistra verso destra	100%	45%	5%	100%	0%

5.3.3. Movimenti complessi con input visivo

In quest'ultima fase, l'obiettivo era testare la capacità di ChatGPT-4o di generare script Python in grado di eseguire movimenti complessi, utilizzando non solo le istruzioni testuali ma anche un input visivo sotto forma di immagine della scena del simulatore. L'aggiunta di un riferimento visivo consente al modello di comprendere

meglio il contesto spaziale in cui il robot Reachy deve operare, migliorando potenzialmente la precisione dei movimenti e l'interazione con gli oggetti.

Il prompt utilizzato per questa fase ha la seguente struttura:

Sulla base degli script Python che ti ho lasciato in allegato, sviluppa uno script Python che permetta a Reachy di [movimento richieste].

In particolare, la pallina da basket è situata un po' di centimetri più a destra (o a sinistra nel caso del task 4) rispetto alla visuale frontale iniziale di Reachy, un po' di centimetri sopra il centro del tavolo.

Le coordinate del tavolo rispetto al sistema di riferimento di Reachy sono le seguenti: $(x, y, z) = (0,3187999, -0,01479991, -1,204)$. Utilizzare la libreria Reachy SDK (versione 0.5.4).

Questo prompt è stato mantenuto volutamente generico, sostituendo ogni volta la dicitura [movimento richiesto] con il movimento specifico richiesto, come:

- Afferrare una pallina da basket
- Mettere una pallina da basket su una tazza
- Spostare una pallina da basket da destra verso sinistra
- Spostare una pallina da basket da sinistra verso destra

Per questa fase, oltre agli script di base per il grasping e l'orientamento della testa, è stata allegata un'immagine della scena del simulatore che mostra la disposizione della pallina da basket e degli oggetti circostanti. Questo input visivo è stato utilizzato per fornire a ChatGPT-4o un contesto spaziale più dettagliato, con l'obiettivo di migliorare la precisione e l'efficacia degli script generati.

L'aggiunta di un'immagine della scena è stata motivata da diverse ragioni:

1. **Supporto contestuale visivo:** L'immagine fornisce un riferimento visivo chiaro su dove si trovano gli oggetti e sulla disposizione del tavolo rispetto a Reachy. Questo permette al modello di integrare meglio le istruzioni testuali con un contesto spaziale chiaro e immediato.

2. **Miglioramento della precisione:** L'uso di input visivi può ridurre le ambiguità presenti nei soli input testuali, facilitando il posizionamento accurato della testa e delle braccia di Reachy durante l'interazione con gli oggetti.
3. **Verifica dell'orientamento:** Con l'immagine della scena, il modello può avere un'idea più precisa dell'orientamento del robot e degli oggetti, migliorando la coordinazione tra i vari componenti del robot.
4. **Apprendimento multimodale:** Integrare informazioni visive e testuali permette di testare la capacità del modello di lavorare in un ambiente multimodale, simulando situazioni più vicine alla realtà dove le informazioni visive sono cruciali per eseguire compiti complessi.

Per ogni task richiesto, ChatGPT-4o ha generato cinque diverse versioni dello script, utilizzando il contesto fornito dagli script allegati e l'immagine della scena. Gli script sono stati testati nel simulatore di Reachy per verificare il comportamento del robot e la correttezza del codice. Gli script generati includevano comandi per orientare la testa, dirigere il braccio e azionare la pinza, integrando tutte queste azioni in un unico script coordinato.

I risultati ottenuti dagli script generati sono sintetizzati nella seguente tabella. La tabella valuta le prestazioni di ogni task in base ai seguenti criteri:

- **Esecuzione generale:** Capacità del robot di eseguire il compito complessivo richiesto.
- **Orientamento della testa:** Accuratezza e fluidità dell'orientamento della testa di Reachy.
- **Movimento del braccio:** Precisione e stabilità del movimento del braccio.
- **Apertura e chiusura della pinza:** Correttezza e tempismo nell'apertura e chiusura della pinza per la manipolazione dell'oggetto.

Tabella 5.5. Risultati del test per movimenti complessi con input visivo

	Esecuzione generale	Orientamento della testa	Movimento del braccio	Apertura e chiusura della pinza	Aderenza al prompt
Afferrare una pallina da Basket	100%	20%	55%	100%	5%
Mettere una pallina da basket su una Tazza	100%	5%	20%	100%	0%
Spostare una pallina da basket da destra verso sinistra	100%	5%	45%	100%	40%
Spostare una pallina da basket da sinistra verso destra	100%	10%	30%	100%	5%

5.4. ANALISI DELLE DIFFERENZE E DELLE LIMITAZIONI DEI RISULTATI

Le prestazioni degli script generati durante i test sistematici mostrano una serie di differenze e limitazioni che meritano un'attenta analisi. L'esecuzione dei compiti programmati e l'integrazione tra variabili complesse hanno evidenziato aspetti critici relativi alla consistenza e all'affidabilità delle soluzioni proposte.

5.4.1. Differenze osservate nei risultati dei test

Nell'ambito della sperimentazione condotta, sono stati eseguiti diversi test sistematici per valutare la capacità di ChatGPT-4o di generare script Python efficaci per il controllo del robot antropomorfo Reachy. Questi test hanno permesso di analizzare le prestazioni dei cinque script generati in diverse situazioni, partendo da task semplici fino a compiti più complessi con input visivo.

Il test pilota ha rappresentato il punto di partenza per valutare l'efficacia degli script generati, utilizzando un prompt specifico e dettagliato. I risultati, riportati nella Tabella 5.1, hanno evidenziato una significativa variabilità nelle performance dei cinque script ottenuti. Mentre tutti gli script sono riusciti a completare il task di base, si sono osservate differenze sostanziali nella precisione del movimento del braccio e nell'orientamento della testa, con alcune imprecisioni nella localizzazione e nel grasping dell'oggetto. Il confronto tra gli script ha mostrato un miglioramento incrementale, con lo Script 5 che ha raggiunto la migliore aderenza al prompt e una maggiore accuratezza nei movimenti rispetto agli altri .

I test relativi ai movimenti semplici, riportati nelle Tabelle 5.2 e 5.3, hanno mostrato un comportamento non uniforme tra i cinque script generati per ciascun task. Ad esempio, nei test di rotazione della testa e posizionamento del braccio, si è osservata una significativa variabilità nell'esecuzione generale e nella capacità degli script di aderire alle istruzioni fornite. In particolare, gli script hanno evidenziato difficoltà nell'interpretare correttamente il comando di "rotazione della testa", con esecuzioni non sempre coerenti e precise. Tuttavia, negli script relativi al movimento delle braccia, è emerso un miglioramento nella coordinazione dei movimenti, soprattutto

nello Script 5, che ha registrato una maggiore precisione e fluidità nell'esecuzione del task di posizionamento e saluto .

Nel caso dei movimenti complessi, riportati nella Tabella 5.4, le differenze tra gli script sono diventate ancora più evidenti. Le prestazioni sono risultate variabili, con alcuni script che hanno eseguito correttamente solo una parte delle operazioni richieste. Ad esempio, nei task di spostamento della pallina da basket, si è verificato che alcuni script hanno eseguito con successo il movimento del braccio, ma non l'orientamento della testa, compromettendo così la precisione complessiva del task. Questo ha messo in luce le difficoltà del modello nel gestire compiti che richiedono un coordinamento complesso tra più azioni, e ha evidenziato che l'uso di prompt generici potrebbe non essere sufficiente per guidare il modello verso l'esecuzione corretta di task avanzati .

L'introduzione dell'input visivo nel quarto test ha permesso di verificare se la disponibilità di un contesto visivo potesse migliorare le prestazioni complessive del robot. Come mostrato nella Tabella 5.5, i risultati sono stati contrastanti. In alcuni casi, l'aggiunta dell'immagine ha facilitato una migliore comprensione spaziale, migliorando la precisione del posizionamento del braccio e l'interazione con l'oggetto. Tuttavia, in altri task, l'integrazione del contesto visivo ha generato confusione, portando a errori di orientamento e una minore aderenza al prompt rispetto al test senza input visivo. Questo risultato suggerisce che, sebbene l'input visivo possa potenzialmente migliorare la performance, la capacità del modello di utilizzare efficacemente queste informazioni è ancora limitata e richiede un'ulteriore ottimizzazione del processo di training .

5.4.2. Analisi delle limitazioni e dei vincoli riscontrati

L'analisi dei test sistematici condotti ha messo in luce diverse limitazioni intrinseche sia nel processo di generazione degli script sia nell'esecuzione dei task da parte del robot Reachy. Nonostante i progressi ottenuti, alcune problematiche persistono e limitano l'efficacia complessiva degli script generati. In questo paragrafo verranno analizzate le principali limitazioni osservate, basandosi sui risultati ottenuti e riportati nelle tabelle dei test.

Una delle principali limitazioni emerse è la significativa variabilità delle prestazioni tra i cinque script generati per ciascun test. Questa differenza è particolarmente evidente nei task più complessi, come l'afferramento e il posizionamento di oggetti. Nonostante la coerenza strutturale degli script, le loro performance variano notevolmente a seconda della capacità del modello di interpretare correttamente il prompt fornito. Ad esempio, nei test di grasping e movimento complesso (Tabella 5.4), si è osservato che alcuni script riescono a eseguire movimenti fluidi e precisi, mentre altri presentano errori significativi nell'orientamento della testa o nella chiusura della pinza. Questa variabilità indica che il modello, pur essendo in grado di generare codice formalmente corretto, non sempre riesce a garantire una prestazione uniforme e affidabile.

Un'altra limitazione importante riguarda la capacità di generalizzazione del modello, soprattutto nei test in cui il prompt fornito era meno dettagliato. Nei test per movimenti semplici (Tabella 5.2 e Tabella 5.3), gli script generati mostrano difficoltà nell'interpretare correttamente istruzioni generiche, portando a esecuzioni non ottimali. In particolare, i movimenti della testa e del braccio spesso non rispettano le aspettative in termini di precisione e fluidità, evidenziando una scarsa capacità del modello di adattarsi a variazioni non esplicitamente specificate nel prompt. Questo limite risulta particolarmente evidente quando si richiede al modello di integrare azioni che coinvolgono più componenti del robot, come l'orientamento della testa e il movimento del braccio in coordinazione.

L'introduzione dell'input visivo nei test per movimenti complessi con input visivo (Tabella 5.5) ha mostrato che, nonostante un lieve miglioramento della precisione in alcuni task, il modello ha ancora difficoltà a utilizzare efficacemente le informazioni visive. In molti casi, l'immagine fornita sembra non essere stata sfruttata in modo ottimale, con errori di orientamento della testa e imprecisioni nel movimento del braccio. Questo indica che il modello non è ancora in grado di integrare completamente le informazioni multimodali (testuali e visive) per generare script che rispecchino fedelmente il contesto operativo. La capacità di interpretare e applicare correttamente le informazioni visive rimane limitata,

soprattutto quando queste sono essenziali per determinare la posizione relativa degli oggetti rispetto al robot.

Nei task più complessi, che richiedono una maggiore coordinazione tra diverse parti del robot, gli script generati spesso non riescono a garantire una performance soddisfacente. Ad esempio, nei task di afferramento e spostamento di oggetti in movimenti complessi (Tabella 5.4), il modello mostra difficoltà nel coordinare il movimento del braccio con l'orientamento della testa e l'apertura della pinza. Questo problema è dovuto in parte alla complessità intrinseca del task, che richiede una gestione precisa del timing e delle sequenze operative, e in parte alla struttura del prompt, che non fornisce indicazioni sufficientemente dettagliate per guidare il modello attraverso tutte le fasi del task.

L'analisi comparativa tra i test con e senza input visivo ha evidenziato che, nonostante l'inclusione di informazioni aggiuntive, il modello non è sempre in grado di utilizzarle per migliorare significativamente le prestazioni degli script. In alcuni casi, l'input visivo ha addirittura introdotto ambiguità, portando a esecuzioni meno accurate rispetto a quelle ottenute senza immagine. Questo suggerisce che l'approccio basato esclusivamente sulla variazione dei prompt potrebbe non essere sufficiente per ottimizzare le prestazioni del modello in compiti complessi e che sarebbe necessario esplorare ulteriori strategie di addestramento e ottimizzazione.

Le limitazioni emerse durante i test sistematici indicano che, sebbene il modello ChatGPT-4o sia in grado di generare script formalmente corretti e, in alcuni casi, anche funzionali, persistono problematiche significative legate alla consistenza delle prestazioni e alla capacità di interpretare e integrare input complessi. Ulteriori sviluppi potrebbero includere un affinamento dei prompt e una maggiore integrazione tra dati testuali e visivi, nonché l'adozione di tecniche di addestramento più avanzate per migliorare la generalizzazione e l'adattabilità del modello.

CONCLUSIONI

La sinergia tra Intelligenza Artificiale (IA) e robotica antropomorfa rappresenta una delle aree di ricerca più promettenti e in rapida evoluzione nel panorama tecnologico odierno. In questa tesi, è stato esplorato l'impiego di modelli linguistici avanzati come ChatGPT-4o per la generazione automatica di script Python destinati al robot antropomorfo Reachy, con l'obiettivo di migliorare le sue capacità operative e interattive.

I risultati ottenuti con gli script generati dall'IA sono stati complessivamente positivi, dimostrando la capacità di eseguire task complessi con un livello di efficienza e precisione accettabile. Gli script prodotti hanno permesso a Reachy di eseguire movimenti sofisticati e interazioni dinamiche con l'ambiente, confermando l'efficacia del modello nel rispondere a esigenze operative diversificate. Sebbene gli script non siano privi di difetti e richiedano ancora perfezionamenti per raggiungere prestazioni ottimali in tutti i contesti, i successi ottenuti mostrano chiaramente il potenziale di questa tecnologia.

È emerso che ulteriori miglioramenti potrebbero essere raggiunti attraverso l'ottimizzazione dei prompt utilizzati per la generazione degli script o con l'integrazione di dati input più dettagliati e specifici. Questo approccio consentirebbe di affinare ulteriormente le capacità del modello, migliorando la precisione e la robustezza degli script generati. Inoltre, l'uso di dati più ricchi e contestualizzati potrebbe aumentare la capacità del robot di adattarsi a situazioni nuove e complesse.

La ricerca condotta ha dimostrato che l'utilizzo dell'IA per la generazione di script rappresenta una valida alternativa alla programmazione tradizionale, contribuendo a esplorare nuove possibilità per lo sviluppo di robot sempre più autonomi e intelligenti. Gli esperimenti realizzati costituiscono una solida base per future ricerche, che potrebbero concentrarsi sull'integrazione di tecniche di apprendimento più avanzate e sull'uso di input multimodali per espandere ulteriormente le capacità cognitive e operative dei robot antropomorfi.

RINGRAZIAMENTI

Desidero esprimere la mia più sincera gratitudine a tutti coloro che hanno contribuito al completamento di questo percorso di studi e alla realizzazione della mia tesi.

Tengo a manifestare la mia più sentita riconoscenza alla Prof.ssa Daniela Giordano, mia relatrice, per avermi dato la possibilità di lavorare in un campo tanto affascinante come quello della robotica e dell'intelligenza artificiale. Il suo supporto e la sua guida sono stati fondamentali per la realizzazione di questo lavoro.

Un sentito ringraziamento va anche ai miei correlatori, il Dott. Salvatore Calcagno e il Dott. Raffaele Mineo, per il loro supporto prezioso e la loro costante disponibilità durante tutte le fasi di realizzazione di questa tesi. La loro esperienza e i loro consigli hanno rappresentato un contributo fondamentale.

Rivolgo un ringraziamento ai miei colleghi universitari, con i quali ho condiviso questo percorso. Il loro supporto è stato essenziale per superare le sfide accademiche e affrontare con determinazione ogni difficoltà.

Un ringraziamento speciale va a Corinna, per avermi sempre sostenuto e incoraggiato con amore e per aver creduto in me in ogni momento di questo viaggio.

Infine, un grazie profondo è rivolto alla mia famiglia, che mi ha sempre supportato e spronato con affetto e dedizione. Senza di loro, nulla di tutto questo sarebbe stato possibile.

INDICE DELLE FIGURE

Figura 2.1. Logo di Pollen Robotics	24
Figura 2.2. Torso frontale di Reachy	29
Figura 2.3. Torso posteriore di Reachy	30
Figura 2.4. Interfaccia hardware di Reachy	31
Figura 2.5. Testa di Reachy	31
Figura 2.6. Braccia di Reachy	32
Figura 2.7. Reachy con base mobile	33
Figura 2.8. Specifiche generali di Reachy	34
Figura 2.9. Specifiche del braccio di Reachy	35
Figura 2.10. Specifiche della pinza di Reachy	36
Figura 2.11. Specifiche della testa di Reachy	37
Figura 2.12. Specifiche del torso di Reachy	38
Figura 2.13. Specifiche di orbita di Reachy	39
Figura 2.14. Base mobile di Reachy	40
Figura 2.15. Specifiche della base mobile di Reachy	40
Figura 4.1. Scena Unity 'Reachy in OfficeScene' modificata	70
Figura 4.2. Grasping della basketball da parte di Reachy	78
Figura 4.3. Ultimo frame dell'occhio destro di Reachy	86
Figura 4.4. Risultato dell'object detection	87
Figura 4.5. Risultato dell'object segmentation	88
Figura 5.1. Logo di OpenAI	94

INDICE DELLE TABELLE

Tabella 3.1. Componenti necessari per l'ambiente di sviluppo	44
Tabella 3.2. Risorse necessarie per l'ambiente di sviluppo	46
Tabella 5.1. Risultati del test pilota	110
Tabella 5.2. Risultati del test di movimenti semplici per la testa di Reachy	113
Tabella 5.3. Risultati del test di movimenti semplici per le braccia di Reachy...	113
Tabella 5.4. Risultati del test per movimenti complessi	116
Tabella 5.5. Risultati del test per movimenti complessi con input visivo	119

BIBLIOGRAFIA

1. Wikipedia. (2024, September 18). *History of robots*. Wikipedia.
https://en.wikipedia.org/wiki/History_of_robots
Nota: La data di archiviazione può essere soggetta a modifiche future.
2. Wikipedia. (2024, September 20). *History of artificial intelligence*.
Wikipedia. https://en.wikipedia.org/wiki/History_of_artificial_intelligence
Nota: La data di archiviazione può essere soggetta a modifiche future.
3. Pollen Robotics. (n.d.-a). *About us Pollen Robotics*. Pollen Robotics.
<https://www.pollen-robotics.com/about>
4. Pollen Robotics. (n.d.-b). *Reachy*. Pollen Robotics.
<https://www.pollen-robotics.com/reachy>
5. Pollen Robotics. (n.d.-c). *VR introduction*. Pollen Robotics.
<https://docs.pollen-robotics.com/vr/introduction/introduction>
6. Pollen Robotics. (n.d.-d). *Open Source*. Pollen Robotics.
<https://www.pollen-robotics.com/opensource>
7. Pollen Robotics. (n.d.-e). *Full/Starter kit*. Pollen Robotics.
<https://docs.pollen-robotics.com/advanced/overview/full-kit>
8. Pollen Robotics. (n.d.-f). *Arm kit*. Pollen Robotics.
<https://docs.pollen-robotics.com/advanced/overview/arm-kit>
9. Pollen Robotics. (n.d.-g). *Reachy + mobile base*. Pollen Robotics.
<https://docs.pollen-robotics.com/advanced/overview/mobile-base>
10. Pollen Robotics. (n.d.-h). *General specifications*. Pollen Robotics.
<https://docs.pollen-robotics.com/advanced/specifications/general-specs>
11. Pollen Robotics. (n.d.-i). *Arm specifications*. Pollen Robotics.
<https://docs.pollen-robotics.com/advanced/specifications/arm-specs>
12. Pollen Robotics. (n.d.-j). *Gripper specifications*. Pollen Robotics.
<https://docs.pollen-robotics.com/advanced/specifications/gripper-specs>
13. Pollen Robotics. (n.d.-k). *Head specifications*. Pollen Robotics.
<https://docs.pollen-robotics.com/advanced/specifications/head-specs>

14. Pollen Robotics. (n.d.-l). *Torso specifications*. Pollen Robotics.
<https://docs.pollen-robotics.com/advanced/specifications/torso-specs>
15. Pollen Robotics. (n.d.-m). *Orbita specifications*. Pollen Robotics.
<https://docs.pollen-robotics.com/advanced/specifications/orbita-specs>
16. Pollen Robotics. (n.d.-n). *Mobile Base specifications*. Pollen Robotics.
<https://docs.pollen-robotics.com/advanced/specifications/mobile-base-specs>
17. Pollen Robotics. (n.d.-o). *SDK Introduction*. Pollen Robotics.
<https://docs.pollen-robotics.com/sdk/getting-started/introduction>
18. Pollen Robotics. (n.d.-p). *SDK Installation*. Pollen Robotics.
<https://docs.pollen-robotics.com/sdk/getting-started/install>
19. Pollen Robotics. (n.d.-q). *SDK Hello World*. Pollen Robotics.
<https://docs.pollen-robotics.com/sdk/getting-started/hello-world>
20. Pollen Robotics. (n.d.-r). *SDK Once you're connected*. Pollen Robotics.
<https://docs.pollen-robotics.com/sdk/first-moves/intro>
21. Pollen Robotics. (n.d.-s). *SDK Safety first*. Pollen Robotics.
<https://docs.pollen-robotics.com/sdk/first-moves/safety>
22. Pollen Robotics. (n.d.-t). *SDK Arms kinematics*. Pollen Robotics.
<https://docs.pollen-robotics.com/sdk/first-moves/kinematics>
23. Pollen Robotics. (n.d.-u). *SDK Controlling the arm*. Pollen Robotics.
<https://docs.pollen-robotics.com/sdk/first-moves/arm>
24. Pollen Robotics. (n.d.-v). *SDK Record and play trajectories*. Pollen Robotics.
<https://docs.pollen-robotics.com/sdk/first-moves/record>
25. Pollen Robotics. (n.d.-w). *SDK Controlling the head*. Pollen Robotics.
<https://docs.pollen-robotics.com/sdk/first-moves/head>
26. Pollen Robotics. (n.d.-x). *SDK Reachy's cameras*. Pollen Robotics.
<https://docs.pollen-robotics.com/sdk/first-moves/cameras>
27. Onshape. (n.d.-a). *Documento 1*. Onshape.
<https://cad.onshape.com/documents/5a9e7c88f6e1068df540bf7a/w/a0a08cc5a2a1d3ad8b96785c/e/a661d6a7df21bf59ff5454f5>

28. Onshape. (n.d.-b). *Documento 2*. Onshape.
<https://cad.onshape.com/documents/0306aa0a644dba7cf10899a8/w/d9693e2e31f7a51b36371b90/e/e5e6361d5aeff2eea89d7bf2>
29. Onshape. (n.d.-c). *Documento 3*. Onshape.
<https://cad.onshape.com/documents/4456e4d7aa9833296dc141b2/w/57a0c3cdfc5fa6bcb14dc18f/e/1d9a5954d213403af8e6551d>
30. Onshape. (n.d.-d). *Documento 4*. Onshape.
<https://cad.onshape.com/documents/2cd541150588bd17e3473399/w/99559d8755e7af1fef990efc/e/c6a9220cdc95295819319026>
31. Onshape. (n.d.-e). *Documento 5*. Onshape.
<https://cad.onshape.com/documents/5ca7f684d33fbcace89ad4d3/w/29fb37b24eb49ed098217242/e/aa193690db5622f1e37493db>
32. Python Software Foundation. (n.d.). *Python*. Python Software Foundation.
<https://www.python.org/>
33. Microsoft. (n.d.). *Visual Studio Code*. Microsoft.
<https://code.visualstudio.com/>
34. Git SCM. (n.d.). *Git*. Git SCM.
<https://git-scm.com/>
35. Unity Technologies. (n.d.). *Unity*. Unity Technologies.
<https://unity.com/>
36. Pollen Robotics. (n.d.-a). *Reachy SDK*. GitHub.
<https://github.com/pollen-robotics/reachy-sdk>
37. Pollen Robotics. (n.d.-b). *Pollen Vision*. GitHub.
<https://github.com/pollen-robotics/pollen-vision>
38. Pollen Robotics. (n.d.-c). *Reachy 2021 Unity Package*. GitHub.
<https://github.com/pollen-robotics/reachy2021-unity-package>
39. gRPC. (2022, April 15). *Artifacts for gRPC*. gRPC.
<https://packages.grpc.io/archive/2022/04/67538122780f8a081c774b66884289335c290cbe-f15a2c1c-582b-4c51-acf2-ab6d711d2c59/index.xml>
Nota: La data di archiviazione può essere soggetta a modifiche future.
40. OpenAI. (n.d.-a). *API Reference: Introduction*. OpenAI.
<https://platform.openai.com/docs/api-reference/introduction>

41. OpenAI. (n.d.-b). *API Reference: Authentication*. OpenAI.
<https://platform.openai.com/docs/api-reference/authentication>
42. OpenAI. (n.d.-c). *API Reference: Making Requests*. OpenAI.
<https://platform.openai.com/docs/api-reference/making-requests>
43. OpenAI. (n.d.-d). *API Reference: Streaming*. OpenAI.
<https://platform.openai.com/docs/api-reference/streaming>
44. OpenAI. (n.d.-e). *API Reference: Debugging Requests*. OpenAI.
<https://platform.openai.com/docs/api-reference/debugging-requests>